

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA, ĐẠI HỌC QUỐC
GIA THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



LUẬN VĂN TỐT NGHIỆP

XÂY DỰNG FRAMEWORK TẠO
DỰNG MÔ HÌNH AI CHO CÁC
ỨNG DỤNG THEO DÕI
CHUYỂN ĐỘNG CỦA CON
NGƯỜI

Chuyên ngành: Khoa học Máy tính

GVHD: TS. Lê Trọng Nhân

—o0o—

Học viên: Nguyễn Trương Minh Hoàng

Mã số học viên: 2270757

Thành phố Hồ Chí Minh, 05 /2026

XÁC NHẬN CỦA GIẢNG VIÊN

Xác nhận chữ ký

LỜI CAM ĐOAN

Tôi cam đoan rằng nghiên cứu này là công trình của riêng tôi, được thực hiện dưới sự giám sát và hướng dẫn của TS. Lê Trọng Nhân. Kết quả nghiên cứu của tôi là hợp pháp và chưa được công bố dưới bất kỳ hình thức nào trước đây. Tất cả các tài liệu được sử dụng trong nghiên cứu này đều được tôi thu thập từ nhiều nguồn khác nhau và đã được liệt kê đầy đủ trong phần tài liệu tham khảo. Ngoài ra, trong nghiên cứu này, tôi cũng đã sử dụng kết quả của một số tác giả và tổ chức khác. Tất cả đều đã được trích dẫn một cách thích hợp. Trong mọi trường hợp đạo văn, tôi hoàn toàn chịu trách nhiệm về hành động của mình. Do đó, Trường Đại học Bách Khoa - ĐHQG TP.HCM không chịu trách nhiệm về bất kỳ vi phạm bản quyền nào được thực hiện trong nghiên cứu của tôi.

LỜI CẢM ƠN

Tôi xin bày tỏ lòng biết ơn chân thành đến tất cả những người đã đóng góp vào việc phát triển và hoàn thiện luận văn này.

Trước tiên, tôi xin gửi lời tri ân sâu sắc nhất đến người hướng dẫn khoa học của tôi, TS. Lê Trọng Nhân, vì sự hướng dẫn quý báu, động viên và phản hồi mang tính xây dựng trong suốt quá trình hình thành đề tài này. Chuyên môn và những hiểu biết sâu sắc của thầy đã đóng vai trò quan trọng trong việc định hướng nghiên cứu của tôi.

Tôi cũng vô cùng biết ơn Trường Đại học Bách Khoa - ĐHQG TP.HCM và Khoa Khoa học và Kỹ thuật Máy tính đã cung cấp môi trường học thuật nuôi dưỡng và các nguồn lực cần thiết để thực hiện nghiên cứu này.

Lời cảm ơn đặc biệt gửi đến các bạn đồng nghiệp và đồng môn đã chia sẻ quan điểm và đưa ra những gợi ý sâu sắc, làm phong phú thêm phạm vi của nghiên cứu này.

Cuối cùng, tôi vô cùng biết ơn gia đình và bạn bè vì sự ủng hộ không ngừng, sự kiên nhẫn và động lực khi tôi đắm mình vào nỗ lực đầy thách thức nhưng bổ ích này.

Đề xuất nghiên cứu này là sự phản ánh của sự hỗ trợ và cống hiến tập thể, và tôi xin chân thành cảm ơn tất cả những người đã đóng góp trực tiếp hoặc gián tiếp vào việc hiện thực hóa nó.

TÓM TẮT

Sự phát triển của học máy và điện toán biên đã cách mạng hóa các ứng dụng theo dõi chuyển động, cho phép các hệ thống thời gian thực, tiết kiệm năng lượng và chính xác cao cho nhiều trường hợp sử dụng trong y tế, phân tích thể thao và tương tác người-máy. Tuy nhiên, việc phát triển các mô hình AI cho theo dõi chuyển động vẫn còn phức tạp, đòi hỏi kiến thức chuyên môn về tiền xử lý dữ liệu, trích xuất đặc trưng, huấn luyện mô hình và triển khai trên thiết bị biên. Bài báo này trình bày một framework toàn diện giúp đơn giản hóa quy trình end-to-end để tạo ra các mô hình AI được tùy chỉnh cho theo dõi chuyển động con người trên các thiết bị biên có tài nguyên hạn chế.

Framework giới thiệu một phương pháp tiền xử lý tương tác mới với tính năng lựa chọn cửa sổ thời gian kéo thả, cho phép người dùng chú thích và phân đoạn dữ liệu chuyển động một cách hiệu quả mà không cần kiến thức lập trình sâu rộng. Hệ thống hỗ trợ nhiều thuật toán học máy bao gồm Random Forest, Support Vector Machines và Neural Networks, với khả năng tự động sinh mã cho nhiều họ thiết bị và môi trường thực thi khác nhau như các bo tương thích Arduino, ARM Cortex-M thuần, ESP-IDF, MicroPython và Zephyr. Kiến trúc tạo mã nhận biết tối ưu hóa xem xét nhiều chiến lược triển khai—độ chính xác, tốc độ, tiêu thụ điện năng và phương án cân bằng—điều chỉnh gói mã sinh ra theo ràng buộc của từng đích triển khai.

Để kiểm chứng framework ở mức thực nghiệm, chúng tôi sử dụng bộ dữ liệu Nhận dạng Hoạt động Con người (HAR) được thu thập ở 100Hz từ cảm biến IMU 6 trục (LSM6DS3) trên Seeed XIAO nRF52840 Sense như một nền tảng tham chiếu thuộc họ ARM Cortex-M4F. Trong nghiên cứu bằng chứng khái niệm này sử dụng dữ liệu đơn đối tượng trên năm lớp hoạt động, các mô hình triển khai trên nền tảng tham chiếu đạt độ chính xác 94.5% cho Random Forest và 96.2% cho Neural Networks, với thời gian suy luận lần lượt là 15ms và 12ms, trong khi tiêu thụ ít hơn 10mW trong quá trình hoạt động. Quan trọng hơn, cùng một pipeline huấn luyện có thể được ánh xạ sang nhiều đích triển khai thông qua hệ thống generator phân tầng theo mô hình, nền tảng và backend thực thi. Phân tích so sánh với các nền tảng thương mại (Edge Impulse, SensiML) cho thấy framework mã nguồn mở của chúng tôi cung cấp tính linh hoạt tiền xử lý vượt trội và chất lượng sinh mã tốt hơn với chi phí bằng không.

Framework giải quyết các khoảng trống quan trọng trong các giải pháp hiện tại bằng cách cung cấp: (1) quy trình làm việc dựa trên GUI trực quan dễ tiếp cận cho người không chuyên, (2) kiểm soát hoàn toàn đường ống tiền xử lý với phản hồi trực

quan, (3) triển khai đa thiết bị với các tối ưu hóa đặc thù cho từng đích, và (4) kiến trúc có thể mở rộng hỗ trợ tích hợp các thuật toán, backend và nền tảng bổ sung trong tương lai. Nghiên cứu này đóng góp vào việc dân chủ hóa phát triển Edge AI cho các ứng dụng theo dõi chuyển động, với tác động tiềm năng trong phục hồi chức năng y tế, giám sát hiệu suất thể thao và công nghệ hỗ trợ.

Mục lục

LỜI CAM ĐOAN	ii
LỜI CẢM ƠN	iii
TÓM TẮT	iv
Mục lục	vi
Danh sách bảng	xi
Danh sách hình vẽ	xii
1 Giới thiệu	1
1.1 Bối cảnh và Động lực Nghiên cứu	1
1.2 Phát biểu Bài toán	2
1.3 So sánh với các Nền tảng Hiện có	3
1.4 Mục tiêu Nghiên cứu	4
1.5 Đóng góp Nghiên cứu	5
1.6 Phạm vi và Hạn chế	6
1.7 Tổ chức Luận văn	6
2 Cơ sở Lý thuyết	7
2.1 Tổng quan	7
2.2 Các Lĩnh vực Ứng dụng của Theo dõi Chuyển động	7
2.3 Theo dõi Chuyển động Dựa trên Cảm biến	8
2.4 Theo dõi Chuyển động Dựa trên Thị giác	8
2.5 AI trong Theo dõi Chuyển động	9
2.6 Các Framework Tạo Mô hình	9
2.7 Tiền xử lý Dữ liệu và Tạo Cửa sổ	11
2.8 Trích xuất và Biểu diễn Đặc trưng	11

2.9	Triển khai Biên và Tối ưu hóa	12
2.10	Các Khoảng trống trong Công trình Hiện có	13
2.11	Tóm tắt	14
3	Phương pháp Luận	15
3.1	Tổng quan Kiến trúc Framework	15
3.2	Thu thập Dữ liệu và Nền tảng Tham chiếu	16
3.2.1	Thiết bị tham chiếu cho thu thập và benchmark	16
3.2.2	Giao thức Thu thập Dữ liệu	16
3.3	Quy trình Tiền xử lý Tương tác	17
3.3.1	Tải lên và Trực quan hóa Dữ liệu	17
3.3.2	Làm sạch và Làm mịn Tín hiệu	18
3.3.3	Bộ lọc Kalman cho Tiền xử lý Tín hiệu	19
3.3.4	Phân đoạn Cửa sổ Thời gian Kéo thả	21
3.4	Trích xuất Đặc trưng	22
3.4.1	Phương pháp Cửa sổ Trượt	22
3.4.2	Bộ Đặc trưng	23
3.4.3	Chiến lược Đệm Cửa sổ (Window Padding Strategy)	24
3.4.4	Tăng cường Dữ liệu cho Huấn luyện (Data Augmentation)	25
3.4.5	Ngưỡng Tin cậy cho Từ chối Hoạt động Không xác định	27
3.5	Huấn luyện Mô hình Học máy	28
3.5.1	Các Thuật toán Được Hỗ trợ	28
3.5.2	Giao thức Huấn luyện và Đánh giá	29
3.5.3	Tối ưu hóa Siêu tham số	30
3.6	Tạo Mã cho Triển khai Biên	32
3.6.1	Kiến trúc	32
3.6.2	Lý do Chọn Tạo Mã Trực tiếp thay vì Runtime Suy luận	33
3.6.3	Triển khai TFLite Micro	34
3.6.4	Cấu trúc Mã Đã Tạo	39
3.6.5	Lượng tử hóa Mô hình cho Triển khai Biên	39
3.6.6	Kiến trúc Tạo Mã Đa Kiến trúc	42
3.6.7	Các Chiến lược Tối ưu hóa	45
3.6.8	Các Điều chỉnh Đặc thù Nền tảng	46
3.7	Thiết kế Thực nghiệm	46
3.7.1	Tập Dữ liệu	46
3.7.2	Các Số liệu Đánh giá	47

3.7.3	Các So sánh Cơ sở	48
3.8	Chi tiết Triển khai	48
4	Kết Quả	50
4.1	Tổng Quan	50
4.2	Hiệu Suất Huấn Luyện Mô Hình	50
4.2.1	Độ Chính Xác Tổng Thể	50
4.2.2	Hiệu Suất Theo Từng Lớp	51
4.2.3	Phân Tích Ma Trận Nhầm Lẫn	52
4.3	Tác Động của Cân Bằng Lớp	54
4.3.1	Nghiên Cứu Loại Bỏ	54
4.4	Khả Năng Triển Khai Đa Thiết Bị	55
4.4.1	Ma trận đích triển khai được hỗ trợ	55
4.5	Benchmark Triển Khai trên Nền tảng Tham chiếu	55
4.5.1	Thời Gian Suy Luận và Độ Trễ	55
4.5.2	Đánh Đổi Chế Độ Tối Ưu Hóa	56
4.5.3	Dấu Chân Bộ Nhớ	57
4.5.4	Tiêu Thụ Điện Năng	58
4.6	Phân Tích So Sánh	59
4.6.1	So Sánh với Các Nền Tảng Thương Mại	59
4.6.2	Tác Động Tối Ưu Hóa Siêu Tham Số	60
4.7	Tóm Tắt Kết Quả	60
5	Thảo Luận	62
5.1	Diễn Giải Các Phát Hiện Chính	62
5.1.1	Hiệu Suất Mô Hình và Lựa Chọn Thuật Toán	62
5.1.2	Mất Cân Bằng Lớp: Một Thách Thức Quan Trọng nhưng Bị Bỏ Qua	63
5.1.3	Tính Khả Thi Triển Khai Biên	63
5.1.4	Tiền Xử Lý Tương Tác: Một Thay Đổi Mô Hình	64
5.2	So Sánh với Công Trình Liên Quan	64
5.2.1	Các Nền Tảng Thương Mại	64
5.2.2	Các Hệ Thống HAR Học Thuật	65
5.3	Các Hạn Chế và Mối Đe Dọa đối với Tính Hợp Lệ	65
5.3.1	Hạn Chế Tập Dữ Liệu	65
5.3.2	Lựa Chọn Kích Thước Cửa Sổ	66
5.3.3	Vị Trí và Hướng Cảm Biến	66

5.3.4	Sự Đa Dạng Nền Tảng Tính Toán	67
5.3.5	Tính Hợp Lệ Bên Ngoài	67
5.4	Các Đánh Đổi Thiết Kế và Các Lựa Chọn Thay Thế	67
5.4.1	ML Cổ Điển vs Học Sâu	67
5.4.2	Đặc Trưng Thủ Công vs Đặc Trưng Học	68
5.4.3	Tạo Mã Trực tiếp vs Runtime Suy luận: Đánh Giá Thực tiễn . .	68
5.4.4	Các Chế Độ Tối Ưu Hóa	69
5.5	Tương đồng Huấn luyện-Triển khai trong Edge ML	70
5.5.1	Các Nguồn Không khớp Được Xác định	70
5.5.2	Tác động Thực tế	71
5.5.3	Danh sách Kiểm tra Tương đồng	71
5.5.4	So sánh với Nền tảng Thương mại	72
5.6	Tăng cường Dữ liệu và Thiết kế Nhận biết Lớp	73
5.6.1	Hiệu quả Tăng cường cho Dữ liệu IMU	73
5.6.2	Bài học từ Tăng cường Không-Nhận-biết Lớp	73
5.6.3	Nguyên tắc Thiết kế	73
5.7	Từ chối Hoạt động Dựa trên Ngưỡng Tin cậy	74
5.7.1	Vấn đề Phân loại Ép buộc	74
5.7.2	Phương pháp Tin cậy Trực tiếp	74
5.7.3	Hạn chế và Hướng Tương lai	75
5.8	Ý Nghĩa cho Thực Hành	75
5.8.1	Nghiên Cứu và Giáo Dục	75
5.8.2	Các Ứng Dụng Y Tế	75
5.8.3	Thể Thao và Thể Dục	76
5.9	Phân Tích Các Phát Hiện Kỹ Thuật Quan Trọng	76
5.9.1	Bộ lọc Kalman: Đột Phá về Training-Deployment Parity	76
5.9.2	Phân Tích Kiến trúc Tạo Mã Đa Kiến trúc	77
5.9.3	Hạn chế TFLite và Giải pháp Keras Surrogate	78
5.10	Các Hướng Nghiên Cứu Tương Lai	79
5.10.1	Các Mở Rộng Ngắn Hạn	79
5.10.2	Tầm Nhìn Dài Hạn	80
5.10.3	Các Cải Tiến Khả Năng Sử Dụng	80
5.11	Nhận Xét Kết Thúc	80
6	Kết Luận	82
6.1	Tóm Tắt Đóng Góp	82

6.1.1	Đóng Góp Kỹ Thuật	82
6.1.2	Xác Thực Thực Nghiệm	84
6.1.3	Tác Động Rộng Hơn	84
6.2	Xem Lại Các Mục Tiêu Nghiên Cứu	85
6.3	Các Ứng Dụng Thực Tế	86
6.4	Các Hạn Chế và Công Việc Tương Lai	87
6.4.1	Sự Đa Dạng Tập Dữ Liệu	87
6.4.2	Các Mở Rộng Thuật Toán	87
6.4.3	Mở Rộng Nền Tảng	87
6.4.4	Các Tính Năng Nâng Cao	88
6.5	Suy Nghĩ Cuối Cùng	88
Tài liệu tham khảo		90

Danh sách bảng

1.3.1	So sánh các Nền tảng Edge ML cho Theo dõi Chuyển động	3
3.3.1	So sánh Bộ lọc IIR và Kalman Filter về Parity	20
3.6.1	Ma trận đích triển khai được hỗ trợ trong kiến trúc generator	33
3.6.2	So sánh phương pháp triển khai mô hình ML trên vi điều khiển	37
3.6.3	So sánh Chiến lược Lượng tử hóa trong Framework	41
4.2.1	Độ Chính Xác Mô Hình Tổng Thể trên Tập Kiểm Tra (HAR 5 lớp, Đối tượng Đơn)	51
4.2.2	Các Số Liệu Hiệu Suất Theo Từng Lớp (Mô Hình Neural Network, 5 Hoạt động)	51
4.3.1	Nghiên Cứu Loại Bỏ: Tác Động của Cân Bằng Lớp đến Các Lớp Thiểu Số	54
4.4.1	Ma trận hỗ trợ triển khai đa thiết bị của framework	55
4.5.1	Thời Gian Suy Luận trên nền tảng tham chiếu Sseed XIAO nRF52840 (cửa sổ 150 mẫu, bước trượt 75 mẫu)	56
4.5.2	Sử Dụng Bộ Nhớ trên nền tảng tham chiếu Sseed XIAO nRF52840 (1MB Flash, 256KB RAM)	57
4.6.1	So Sánh Framework: Đề Xuất vs Giải Pháp Thương Mại (Ước tính) . . .	59
4.6.2	Tác Động của Tối Ưu Hóa Siêu Tham Số (CV 5 fold)	60
5.5.1	Danh sách kiểm tra tương đồng huấn luyện-triển khai	72

Danh sách hình vẽ

1.4.1	Quy trình làm việc hoàn chỉnh: Từ tải dữ liệu thô đến triển khai biên (tổng ước tính 30–60 phút)	5
3.1.1	Tổng quan kiến trúc framework: quy trình sáu bước từ tải dữ liệu thô đến triển khai mô hình trên thiết bị biên, hỗ trợ các thuật toán RF, SVM, MLP, CNN và xuất mã đa nền tảng	15
3.3.1	Giao diện web cho tải lên tệp CSV và quản lý tập dữ liệu. Giao diện cho phép người dùng tải lên các tệp dữ liệu cảm biến, xác thực tên cột, tự động phát hiện tần số lấy mẫu và cung cấp bản xem trước tương tác của chuỗi thời gian sáu trục thô.	18
3.3.2	Giao diện cửa sổ kéo thả tương tác cho phân đoạn hoạt động. Người dùng kiểm tra trực quan chuỗi thời gian gia tốc kế và con quay hồi chuyển, kéo các cửa sổ hình chữ nhật qua các vùng tương ứng với các hoạt động riêng biệt, gán nhãn hoạt động và điều chỉnh ranh giới qua tương tác nhấp và kéo. Điều này loại bỏ tính toán dấu thời gian thủ công và làm cho framework có thể tiếp cận được cho những người không lập trình. . .	22
3.5.1	Bảng cấu hình mô hình cho thấy lựa chọn thuật toán (Random Forest, Neural Network, SVM, CNN), các tùy chọn điều chỉnh siêu tham số với GridSearchCV, các chiến lược cân bằng lớp (trọng số cân bằng, phân chia phân tầng) và giám sát tiến trình huấn luyện thời gian thực.	32
3.6.1	Giao diện xem trước gói mã triển khai đã tạo. Framework hiển thị các tệp header, tệp triển khai và tệp ví dụ tích hợp tương ứng với nền tảng đích (Arduino, ARM Cortex-M, Generic C/C++, ESP-IDF, MicroPython, Zephyr, TFLite Micro, ONNX Runtime), cho phép người dùng chọn các chế độ tối ưu hóa (Accuracy, Speed, Power, Balanced) và tải xuống gói triển khai hoàn chỉnh.	44
4.2.1	Ma Trận Nhầm Lẫn cho Mô Hình Neural Network (HAR 5 lớp, độ chính xác 96.2%, xác thực đối tượng đơn)	52

4.2.2	Bảng điều khiển kết quả hiển thị ma trận nhầm lẫn và số liệu hiệu suất. Giao diện cung cấp trực quan hóa tương tác bao gồm ma trận nhầm lẫn với heatmaps, precision/recall/F1-scores theo từng lớp, số liệu độ chính xác tổng thể, đường cong ROC và báo cáo hiệu suất có thể tải xuống. Người dùng có thể so sánh nhiều mô hình cạnh nhau.	53
4.5.1	Phân tích so sánh độ chính xác, độ trễ suy luận và tiêu thụ điện năng trên bốn chế độ tối ưu hóa	57
4.5.2	Hồ Sơ Tiêu Thụ Điện Năng Trong Suy Luận trên nền tảng tham chiếu (Neural Network, Chế Độ Accuracy)	58

Chương 1

Giới thiệu

1.1 Bối cảnh và Động lực Nghiên cứu

Theo dõi chuyển động (Motion tracking) là một lĩnh vực năng động và phát triển nhanh chóng, liên quan đến việc thu thập, xử lý và phân tích chuyển động của các đối tượng, cá nhân hoặc môi trường. Công nghệ này đóng vai trò then chốt trong nhiều lĩnh vực bao gồm y tế (phục hồi chức năng, phát hiện té ngã, giám sát bệnh nhân), thể thao (theo dõi hiệu suất, phòng ngừa chấn thương), trò chơi điện tử và thực tế ảo/thực tế tăng cường (trải nghiệm người dùng nhập vai), và robot học (hệ thống định vị, cơ chế điều khiển chính xác)^[46].

Thị trường toàn cầu cho công nghệ theo dõi chuyển động đạt 2,8 tỷ USD vào năm 2023 và dự kiến tăng trưởng với tốc độ tăng trưởng kép hàng năm (CAGR) là 18,4% đến năm 2030^[2]. Sự tăng trưởng này được thúc đẩy đặc biệt bởi các thiết bị đeo y tế và ứng dụng nhà thông minh. Sự quan tâm trong giới học thuật cũng tăng vọt, với các công bố nghiên cứu về nhận dạng hoạt động con người (Human Activity Recognition - HAR) tăng từ 487 bài báo vào năm 2018 lên hơn 1.500 bài vào năm 2023, tương ứng với mức tăng 3,1 lần chỉ trong năm năm^[41]. Sự hội tụ của các cảm biến IoT giá cả phải chăng, thuật toán học máy hiệu quả và khả năng tính toán biên (edge computing) đã dân chủ hóa việc theo dõi chuyển động từ các phòng thí nghiệm chuyên biệt sang các ứng dụng tiêu dùng hàng ngày.

Những tiến bộ trong công nghệ cảm biến và trí tuệ nhân tạo (AI) đã mang tính chuyển đổi, làm tăng đáng kể độ chính xác, hiệu suất và tính linh hoạt của các hệ thống theo dõi chuyển động. Đặc biệt, các mô hình AI là không thể thiếu trong việc diễn giải dữ liệu chuyển động, cho phép các ứng dụng như nhận dạng cử chỉ, phân loại hoạt động và mô hình hóa chuyển động dự đoán. Tuy nhiên, quá trình phát triển các mô hình AI này vốn dĩ phức tạp, bao gồm thu thập dữ liệu, tiền xử lý và lọc để loại bỏ

nhiều, trích xuất các đặc trưng có ý nghĩa, và huấn luyện các mô hình học máy hoặc học sâu. Sự phức tạp này đặt ra những thách thức đáng kể, đặc biệt đối với các nhà phát triển và nhà nghiên cứu có thể thiếu chuyên môn về AI hoặc công nghệ theo dõi chuyển động^[28].

Sự phát triển của tính toán biên (edge computing) đã làm phức tạp thêm bối cảnh này trong khi đồng thời mang lại những cơ hội mới. Các thiết bị biên—vi điều khiển bị giới hạn tài nguyên với bộ nhớ, sức mạnh xử lý và ngân sách năng lượng hạn chế—cho phép các ứng dụng theo dõi chuyển động theo thời gian thực, bảo vệ quyền riêng tư và tiết kiệm năng lượng. Tuy nhiên, triển khai các mô hình AI trên các thiết bị như vậy đòi hỏi kiến thức chuyên biệt về tối ưu hóa mô hình, tạo mã và các chi tiết triển khai đặc thù cho từng nền tảng. Thách thức này trở nên rõ rệt hơn khi hệ sinh thái thiết bị biên ngày càng phân mảnh: cùng một mô hình có thể cần được đóng gói khác nhau cho các bo tương thích Arduino, ARM Cortex-M thuần, SDK như ESP-IDF, MicroPython hoặc RTOS như Zephyr.

1.2 Phát biểu Bài toán

Bất chấp những tiến bộ trong công nghệ theo dõi chuyển động, việc tạo ra các mô hình AI cho phân tích chuyển động vẫn là một nhiệm vụ đầy thách thức đặt ra một số rào cản quan trọng:

1. **Độ phức tạp Kỹ thuật:** Các cách tiếp cận hiện tại đòi hỏi kiến thức kỹ thuật sâu rộng trải rộng nhiều lĩnh vực—xử lý tín hiệu cho tiền xử lý dữ liệu, học máy cho phát triển mô hình và hệ thống nhúng cho triển khai biên.
2. **Thách thức Tiền xử lý:** Dữ liệu cảm biến chuyển động vốn dĩ nhiễu và đòi hỏi tiền xử lý cẩn thận. Các công cụ hiện có hoặc cung cấp kiểm soát hạn chế đối với tiền xử lý (các cách tiếp cận hộp đen tự động) hoặc đòi hỏi lập trình mở rộng để triển khai các quy trình tiền xử lý tùy chỉnh.
3. **Tính linh hoạt Hạn chế:** Nhiều framework có sẵn có đường cong học tập dốc hoặc cung cấp chức năng hạn chế để tùy chỉnh mô hình theo các yêu cầu theo dõi chuyển động cụ thể. Các nền tảng thương mại như Edge Impulse và SensiML cung cấp quy trình làm việc đơn giản hóa nhưng với chi phí đáng kể (\$20-500/tháng) và với các tùy chọn tùy chỉnh bị hạn chế.
4. **Độ phức tạp Triển khai:** Dịch cùng một mô hình đã huấn luyện thành các gói mã tối ưu hóa cho nhiều họ thiết bị biên khác nhau đòi hỏi hiểu biết sâu về kiến

1.3. So sánh với các Nền tảng Hiện có

trúc phần cứng đích, ràng buộc bộ nhớ, hệ sinh thái công cụ (Arduino, ESP-IDF, RTOS, MicroPython) và kỹ thuật tối ưu hóa phù hợp cho từng trường hợp triển khai.

5. **Rào cản Khả năng tiếp cận:** Sự kết hợp của các yếu tố này hạn chế đổi mới và cản trở việc áp dụng rộng rãi các giải pháp theo dõi chuyển động được hỗ trợ bởi AI, đặc biệt trong bối cảnh nghiên cứu và giáo dục.

Do đó, có một nhu cầu cấp thiết về một framework linh hoạt, trực quan và thân thiện với người dùng giúp đơn giản hóa việc tạo các mô hình AI phù hợp cho các ứng dụng theo dõi chuyển động trong khi cung cấp kiểm soát hoàn toàn đối với quy trình phát triển.

1.3 So sánh với các Nền tảng Hiện có

Để đặt các đóng góp trong ngữ cảnh, Bảng 1.3.1 so sánh framework này với các lựa chọn thay thế thương mại và mã nguồn mở hàng đầu cho triển khai edge ML trong các ứng dụng theo dõi chuyển động.

Bảng 1.3.1: So sánh các Nền tảng Edge ML cho Theo dõi Chuyển động

Tính năng	Edge Impulse	SensiML	TensorFlow Lite	Framework này
Chi phí	\$20-99/tháng	\$99-500/tháng	Miễn phí	Miễn phí (Mã nguồn mở)
Tiền xử lý Tương tác	Hạn chế (web UI)	Có (analytics studio)	Không (chỉ mã)	Có (cửa sổ kéo thả)
Thuật toán	NN, transfer learning	RF, SVM, NN, boosting	Chỉ deep learning	RF, SVM, NN
Tạo Mã	C++ (Arduino, ARM)	C (MCU-cụ thể)	C++ (TFLite Micro)	C/C++/Python đa đích
Chế độ Tối ưu hóa	Accuracy/latency	AutoML	Quant. thủ công	4 chế độ (acc./speed/power/bal.)
Mất cân bằng Lớp	Oversample thủ công	Cân bằng tự động	Không tích hợp	Tự động (class_weight)
Tùy chỉnh	Hạn chế	Trung bình	Cao (OSS)	Cao (modular)
Sử dụng Giáo dục	Gói miễn phí	Giấy phép học thuật	Có	Có (không giới hạn)
Nền tảng Triển khai	20+ boards	MCUs đã chọn	Tương thích TFLite	Arduino, ESP-IDF, ARM, MicroPython, Zephyr
Kiểm soát Dữ liệu Huấn luyện	Lưu trữ đám mây	Local/cloud	Chỉ local	Local (bảo mật hoàn toàn)

Các yếu tố phân biệt chính của framework này bao gồm: (1) **Tiền xử lý tương tác mới** với chọn cửa sổ thời gian kéo thả, giảm thời gian chú thích so với các cách tiếp cận dựa trên mã hoặc nhấp chuột; (2) **Hỗ trợ đa thuật toán** với các API nhất quán, cho phép thử nghiệm nhanh chóng mà không cần chuyển đổi nền tảng; (3) **Tính minh bạch Hoàn toàn** thông qua tính khả dụng mã nguồn mở, cho phép các nhà nghiên cứu hiểu, xác thực và mở rộng toàn bộ quy trình; (4) **Chi phí bằng không** cho sử dụng không giới hạn, loại bỏ rào cản tài chính cho sinh viên, nhà nghiên cứu và các khu vực đang phát triển; (5) **Tạo mã nhận thức tối ưu hóa** đánh đổi rõ ràng giữa độ chính xác, tốc độ và tiêu thụ điện năng dựa trên yêu cầu ứng dụng.

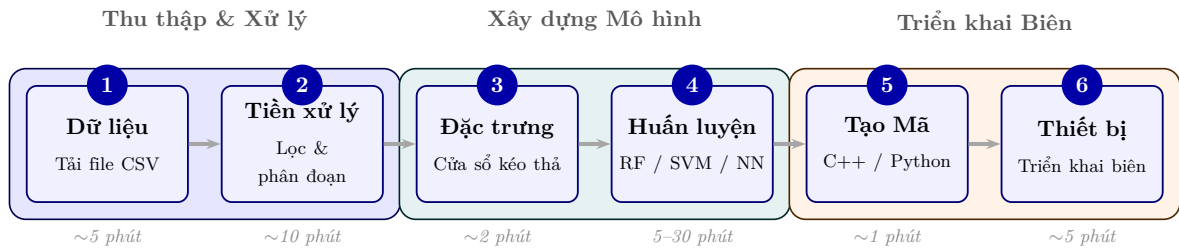
Trong khi các nền tảng thương mại cung cấp lợi thế về độ trưởng thành của hệ sinh thái (hỗ trợ 20+ board của Edge Impulse) và sự tinh vi của AutoML (lựa chọn đặc trưng tự động của SensiML), chúng áp đặt chi phí định kỳ (\$240-2.388/năm), hạn chế tùy chỉnh thông qua các API độc quyền và tạo ra sự phụ thuộc nhà cung cấp. TensorFlow Lite cung cấp khả năng học sâu mạnh mẽ nhưng đòi hỏi chuyên môn ML đáng kể và không cung cấp công cụ tiền xử lý cấp cao. Framework này nhắm đến khoảng trống giữa tính dễ sử dụng và tính linh hoạt, cung cấp một giải pháp có thể tiếp cận nhưng mạnh mẽ cho bối cảnh giáo dục và nghiên cứu^[12;15].

1.4 Mục tiêu Nghiên cứu

Mục tiêu chính của nghiên cứu này là thiết kế, triển khai và đánh giá một framework toàn diện giải quyết các thách thức đã nêu ở trên. Cụ thể, nghiên cứu này nhằm mục đích:

1. **Phát triển Hệ thống Tiền xử lý Tương tác:** Tạo giao diện tiền xử lý dựa trên GUI mới với chọn cửa sổ thời gian kéo thả, cho phép chú thích và phân đoạn dữ liệu hiệu quả mà không cần yêu cầu lập trình.
2. **Triển khai Hỗ trợ Đa Thuật toán:** Tích hợp nhiều thuật toán học máy (Random Forest, SVM, Neural Networks) với các API nhất quán và tối ưu hóa siêu tham số tự động.
3. **Thiết kế Tạo Mã Bất khả tri Nền tảng:** Xây dựng hệ thống tạo mã modular có khả năng sản xuất các triển khai C/C++/MicroPython tối ưu hóa cho nhiều họ đích khác nhau (Arduino-compatible, ARM Cortex-M, ESP-IDF, Zephyr, MicroPython) với xem xét các chiến lược tối ưu hóa khác nhau (độ chính xác, tốc độ, điện năng, cân bằng).

4. **Xác thực Hiệu quả Framework:** Tiến hành các thử nghiệm toàn diện sử dụng dữ liệu Nhận dạng Hoạt động Con người thực tế để đánh giá hiệu suất mô hình, kiểm tra năng lực sinh mã cho nhiều đích triển khai và đo benchmark đại diện trên phần cứng ARM Cortex-M4F tham chiếu.
5. **Đảm bảo Tương đồng và Độ Tin Cậy Huấn luyện-Triển khai:** Tích hợp bộ lọc Kalman và kiến trúc tạo mã nhận biết kiến trúc để đảm bảo tương đồng chính xác giữa mô hình huấn luyện và mã triển khai; tích hợp ngưỡng tin cậy để từ chối hoạt động không xác định và sử dụng tăng cường dữ liệu nhận biết lớp để đảm bảo độ tin cậy trên phần cứng thực.



Hình 1.4.1: Quy trình làm việc hoàn chỉnh: Từ tải dữ liệu thô đến triển khai biên (tổng ước tính 30–60 phút)

1.5 Đóng góp Nghiên cứu

Nghiên cứu này đóng góp các điểm chính sau vào lĩnh vực theo dõi chuyển động dựa trên biên:

- **Tiền xử lý Tương tác Mới:** Giao diện chọn cửa sổ thời gian kéo thả đầu tiên cho chú thích dữ liệu chuyển động, giảm đáng kể thời gian tiền xử lý và lỗi chú thích so với các cách tiếp cận thủ công.
- **Kiến trúc Tạo Mã Đa Thiết bị Nhận thức Tối ưu hóa:** Hệ thống tạo mã modular ánh xạ cùng một mô hình đã huấn luyện sang nhiều họ đích (Arduino-compatible, ARM Cortex-M, ESP-IDF, Zephyr, MicroPython) với các cấu hình tối ưu hóa riêng theo độ chính xác, tốc độ và tiêu thụ điện năng.
- **Cơ chế Tương đồng Huấn luyện-Triển khai:** Quy trình xác minh đảm bảo mã C++ được tạo ra cho kết quả nhận dạng nhất quán với mô hình Python đã huấn luyện, bao gồm bộ lọc Kalman, công thức thống kê chính xác và sắp xếp lại trọng số mô hình tại thời điểm tạo mã.

- **Framework Mã nguồn mở Toàn diện:** Quy trình hoàn chỉnh từ đầu đến cuối từ tải dữ liệu đến triển khai biên, cung cấp lựa chọn thay thế mã nguồn mở miễn phí cho các nền tảng thương mại.

1.6 Phạm vi và Hạn chế

Nghiên cứu này tập trung vào các ứng dụng theo dõi chuyển động dựa trên IMU (gia tốc kế và con quay hồi chuyển). Phạm vi bao gồm:

- Dữ liệu cảm biến IMU 6 trục (gia tốc kế 3 trục + con quay hồi chuyển 3 trục)
- Các phương pháp học có giám sát sử dụng các thuật toán scikit-learn
- Tạo mã cho nhiều họ nền tảng biên dựa trên vi điều khiển và môi trường thực thi nhúng
- Nhận dạng Hoạt động Con người là lĩnh vực xác thực chính

Các số đo phần cứng định lượng trong luận văn này tập trung vào một nền tảng tham chiếu (Seeed XIAO nRF52840), trong khi các đích còn lại được phân tích ở mức kiến trúc generator và khả năng sinh mã.

Nghiên cứu không giải quyết:

- Hợp nhất cảm biến đa phương thức (kết hợp IMU với camera, âm thanh, v.v.)
- Tích hợp các framework học sâu (TensorFlow, PyTorch)
- Cập nhật mô hình theo thời gian thực hoặc các kịch bản học liên kết
- Các lĩnh vực chuyên biệt yêu cầu chứng nhận y tế

1.7 Tổ chức Luận văn

Phần còn lại của bài báo này được tổ chức như sau: Phần 2 đánh giá các công trình liên quan về hệ thống theo dõi chuyển động, framework tạo mô hình AI và công cụ triển khai biên. Phần 3 trình bày phương pháp luận chi tiết bao gồm kiến trúc hệ thống, quy trình tiền xử lý, phương pháp huấn luyện mô hình và chiến lược tạo mã. Phần 4 báo cáo kết quả thực nghiệm bao gồm các số liệu hiệu suất mô hình, đặc điểm triển khai và phân tích so sánh. Phần 5 thảo luận về các phát hiện, hạn chế và ý nghĩa. Phần 6 kết luận với một tóm tắt về các đóng góp và hướng cho công việc tương lai.

Chương 2

Cơ sở Lý thuyết

2.1 Tổng quan

Chương này trình bày cơ sở lý thuyết trên sáu lĩnh vực: các ứng dụng theo dõi chuyển động, các phương thức cảm biến, kiến trúc AI cho HAR, các framework tạo mô hình, phương pháp tiền xử lý và trích xuất đặc trưng, và kỹ thuật tối ưu hóa triển khai biên. Phân tích dựa trên 40+ công bố từ 2001–2024, với trọng tâm vào các tiến bộ TinyML (2020–2024) và học sâu cho HAR. Dựa trên đánh giá này, các khoảng trống nghiên cứu được xác định để định vị framework đề xuất.

2.2 Các Lĩnh vực Ứng dụng của Theo dõi Chuyển động

Theo dõi chuyển động con người là một công nghệ nền tảng trên nhiều lĩnh vực y tế, thể thao và thể dục, thực tế ảo/thực tế tăng cường (VR/AR), robot học và các hệ thống tương tác mới nổi. Trong y tế, các cảm biến đeo và môi trường xung quanh cho phép phân tích dáng đi, phát hiện té ngã (đạt độ nhạy 95-98%^[39]), và giám sát phục hồi chức năng, hỗ trợ các can thiệp cá nhân hóa^[46]. Các ứng dụng thể thao tận dụng dữ liệu quán tính và sinh cơ học để tối ưu hóa hiệu suất và phòng ngừa chấn thương, trong khi các hệ thống VR/AR phụ thuộc vào theo dõi khung xương và vị trí chính xác cho trải nghiệm người dùng nhập vai. Robot học tích hợp theo dõi chuyển động để điều hướng, thao tác và tương tác người-robot. Các ứng dụng công nghiệp gần đây bao gồm giám sát an toàn lực lượng lao động^[40], đạt độ chính xác 94,3% trong phát hiện tư thế nguy hiểm trên các địa điểm xây dựng và sản xuất. Các lĩnh vực này cùng nhau tạo động lực cho nhu cầu về các framework theo dõi chuyển động được hỗ trợ bởi

AI có thể tiếp cận, thích ứng và hiệu quả, có khả năng suy luận biên theo thời gian thực với độ chính xác 90%+.

2.3 Theo dõi Chuyển động Dựa trên Cảm biến

Các Đơn vị Đo lường Quán tính (IMUs)—kết hợp gia tốc kế và con quay hồi chuyển—được áp dụng rộng rãi cho theo dõi chuyển động do tiêu thụ điện năng thấp (dòng điện hoạt động 2-10 mA), hiệu quả chi phí (\$1-5 mỗi đơn vị) và tính phù hợp cho triển khai biên. Các khảo sát toàn diện về theo dõi chuyển động dựa trên IMU^[13] nhấn mạnh các thách thức bao gồm trôi cảm biến (độ lệch con quay hồi chuyển 0,1-1°/s), lỗi tích lũy trong ước lượng hướng, yêu cầu hiệu chuẩn và ràng buộc sinh cơ học. Các phương pháp hợp nhất đa cảm biến (ví dụ: IMU + từ kế + cảm biến áp suất) có thể cải thiện độ bền vững—đạt được mức tăng độ chính xác 2-5%^[13]—nhưng tăng độ phức tạp, tiêu thụ điện năng và chi phí. Công trình gần đây của Xia và cộng sự^[44] chứng minh các hệ thống IMU đơn đạt độ chính xác 96,8% trên các nhiệm vụ HAR 12 lớp sử dụng kiến trúc LSTM-CNN, xác thực tính khả thi của cách tiếp cận cảm biến đơn. Các tập dữ liệu HAR chuẩn như UCI-HAR^[4] và WISDM^[6] đều sử dụng IMU sáu trục gắn trên cơ thể ở tần số 50–100 Hz, qua đó xác lập cấu hình này làm thiết lập tham chiếu phổ biến cho các nghiên cứu HAR dựa trên thiết bị đeo. Framework trong luận văn này kế thừa cấu hình đó—thu thập và xử lý dữ liệu gia tốc kế và con quay hồi chuyển ba trục ở 100 Hz—và tập trung vào các quy trình IMU đơn đáng tin cậy, với khả năng mở rộng cho tích hợp đa cảm biến khi cần thiết.

2.4 Theo dõi Chuyển động Dựa trên Thị giác

Các phương pháp thị giác máy tính sử dụng camera lập thể, theo dõi không dấu và ước lượng tư thế cung cấp các biểu diễn không gian phong phú về chuyển động con người^[45]. Trong khi các hệ thống thị giác cho phép tái tạo toàn thân (phát hiện 17+ điểm khóa tại 30 FPS^[16]) và được sử dụng trong hoạt hình và phân tích lâm sàng, chúng gặp phải các thách thức về che khuất (suy giảm độ chính xác 15-30% trong các cảnh đông đúc), biến đổi môi trường (ánh sáng, thời tiết), lo ngại về quyền riêng tư và tiêu thụ điện năng (500-2000 mW so với 20-50 mW cho hệ thống IMU). Các tiến bộ gần đây bao gồm MediaPipe pose^[16] (133 mốc tư thế, độ chính xác 95% trên các

khớp nhìn thấy) và OpenPose đã cải thiện khả năng tiếp cận nhưng vẫn ít khả thi hơn cho các thiết bị đeo công suất siêu thấp. Các hệ thống kết hợp IMU-thị giác^[39] kết hợp các điểm mạnh bổ sung—độ bền vững của IMU với che khuất, độ chính xác không gian của thị giác—nhưng yêu cầu đồng bộ hóa cảm biến và tăng độ phức tạp hệ thống. Đối với giám sát đeo liên tục ưu tiên tuổi thọ pin (ngày đến tuần), các giải pháp IMU thuần túy vẫn chiếm ưu thế.

2.5 AI trong Theo dõi Chuyển động

Các mô hình AI chuyển đổi dữ liệu chuyển động thô thành các diễn giải ngữ nghĩa—phân loại hoạt động, nhận dạng tư thế và dự báo quỹ đạo. Các phương pháp học máy truyền thống bao gồm Support Vector Machines (SVM)^[10] và Random Forests^[7] đã được sử dụng rộng rãi, đạt độ chính xác 88-93% trên các benchmark HAR chuẩn (UCI-HAR, WISDM) với kích thước bộ nhớ thời gian chạy nhỏ gọn (50–200 KB) phù hợp cho các thiết bị biên^[41]. Các kiến trúc học sâu đã từng bước cải thiện hiệu suất HAR: CNNs (độ chính xác 92-95%, trích xuất đặc trưng không gian từ các cửa sổ cảm biến), RNN/LSTMs (độ chính xác 94-97%, nắm bắt các phụ thuộc thời gian^[44]), và transformers dựa trên attention (độ chính xác 96-98% trên các tập dữ liệu phức tạp^[36]). Tuy nhiên, các mô hình học sâu đòi hỏi các tập dữ liệu được chú thích lớn hơn (10.000+ mẫu so với 1.000-5.000 cho ML cổ điển), bộ nhớ cao hơn (500 KB-2 MB so với 50-200 KB) và các chiến lược lượng tử hóa phức tạp hơn (số nguyên 8-bit hoặc độ chính xác hỗn hợp) trong triển khai biên^[24]. Các nghiên cứu so sánh gần đây^[41] cho thấy các mô hình ML cổ điển đạt độ chính xác 90-93% với dấu chân bộ nhớ nhỏ hơn 10× và suy luận nhanh hơn 5-20× (12-18 ms so với 60-120 ms mỗi cửa sổ) trên các mục tiêu Cortex-M4. Framework được đề xuất hỗ trợ chiến lược cả hai mô hình—mô hình cổ điển cho các kịch bản bị giới hạn tài nguyên, mạng nơ-ron nhẹ cho các ứng dụng quan trọng độ chính xác—với các chế độ tối ưu hóa đa mục tiêu giúp cân nhắc các đánh đổi một cách có căn cứ.

2.6 Các Framework Tạo Mô hình

Một số nền tảng đã xuất hiện để đơn giản hóa việc tạo mô hình AI, mỗi nền tảng có điểm mạnh và hạn chế riêng biệt:

- **Edge Impulse**^[12]: Cung cấp các quy trình từ đầu đến cuối cho thu thập dữ liệu, tiền xử lý (phân tích phổ, MFE/MFCC), tạo đặc trưng, huấn luyện mô hình (CNN, LSTM, ML cổ điển) và triển khai cho 85+ mục tiêu vi điều khiển.

Thu hút hơn 100.000 nhà phát triển, nền tảng này vượt trội trong tự động hóa quy trình (95% sự hài lòng của người dùng cho tạo mẫu nhanh^[25]) nhưng hạn chế tính minh bạch trong logic tiền xử lý và tính phí \$20-99/tháng cho sử dụng thương mại. Các cửa sổ tiền xử lý được cố định sau khi tải lên; tính chỉnh tương tác yêu cầu tải lên lại dữ liệu.

- **SensiML Analytics Toolkit**^[34]: Cung cấp lựa chọn đặc trưng được điều khiển bởi AutoML (200+ đặc trưng ứng viên, tối ưu hóa thuật toán di truyền) và các gói kiến thức cho các thiết bị ARM Cortex-M. Đạt độ chính xác 92-96% trên các nhiệm vụ nhận dạng cử chỉ nhưng sử dụng các định dạng độc quyền và chi phí \$99-500/tháng. Tiền xử lý không cần lập trình nhưng thiếu minh bạch (lựa chọn đặc trưng theo kiểu hộp đen).
- **MediaPipe**^[16]: Cung cấp các quy trình nhận thức modular (tư thế, tay, mặt, theo dõi đối tượng) với thực thi đồ thị tối ưu hóa trên các thiết bị di động/biên. Chủ yếu hướng tới thị giác, ít phù hợp hơn cho các quy trình làm việc IMU tùy chỉnh yêu cầu các chiến lược phân đoạn tùy chỉnh. Không có hỗ trợ sẵn có cho phân đoạn chuỗi thời gian hoặc trích xuất đặc trưng HAR.
- **Teachable Machine**^[17]: Cho phép tạo mẫu nhanh các bộ phân loại hình ảnh, âm thanh và tư thế thông qua giao diện không mã. Được sử dụng trong 1M+ dự án giáo dục nhưng thiếu công cụ cho kỹ thuật đặc trưng nâng cao, phân đoạn chuỗi thời gian và triển khai tối ưu theo từng phần cứng (chỉ xuất mô hình TensorFlow.js).
- **TensorFlow Lite Micro**^[?] cung cấp thời gian chạy để triển khai các mô hình lượng tử hóa trên vi điều khiển, hỗ trợ tối ưu hóa thông qua lượng tử hóa sau huấn luyện (số nguyên 8-bit, float 16-bit) và tối ưu hóa suy luận thời gian chạy (CMSIS-NN cho ARM). Tuy nhiên, TFLite chỉ hỗ trợ mạng nơ-ron và yêu cầu thiết kế kiến trúc mô hình thủ công.
- **AutoML/NAS cho TinyML**: Nghiên cứu gần đây khám phá Neural Architecture Search (NAS) để khám phá các cấu trúc liên kết mạng tối ưu dưới các ràng buộc tài nguyên^[?]. Dù là hướng nghiên cứu triển vọng, NAS thường đòi hỏi tài nguyên tính toán đáng kể và thiếu tính minh bạch cho các bối cảnh giáo dục và nghiên cứu.

So với các nền tảng trên, Framework này giải quyết các hạn chế đó bằng cách cung cấp: (1) **GUI tiền xử lý tương tác** với chọn cửa sổ thời gian kéo thả cho phép phân

đoạn chính xác, (2) **kỹ thuật đặc trưng minh bạch** với kiểm tra từng đặc trưng, (3) **tối ưu hóa đa mục tiêu** (các chế độ độ chính xác/tốc độ/điện năng/cân bằng) được điều chỉnh theo ràng buộc, và (4) **khả năng tiếp cận mã nguồn mở** loại bỏ rào cản chi phí.

2.7 Tiền xử lý Dữ liệu và Tạo Cửa sổ

Nhận dạng hoạt động chính xác phụ thuộc nhiều vào chất lượng của các chiến lược phân đoạn và tạo cửa sổ. Banos và cộng sự^[6] đánh giá có hệ thống ảnh hưởng của kích thước cửa sổ đến hiệu suất HAR: cửa sổ 1 giây đạt độ chính xác 89,3%, cửa sổ 2,5 giây đạt 92,1%, trong khi cửa sổ 5 giây mang lại 93,4% nhưng giảm khả năng phản hồi. Tỷ lệ chồng lấp cũng ảnh hưởng đến hiệu suất—chồng lấp 50% cải thiện độ chính xác 2-4% so với các cửa sổ không chồng lấp^[39] với chi phí tính toán gấp đôi. Lựa chọn bộ lọc ảnh hưởng đáng kể đến giảm nhiễu: bộ lọc thông thấp Butterworth (cutoff 20 Hz) bảo tồn các đặc tính chuyển động trong khi làm suy giảm nhiễu cảm biến, cải thiện độ chính xác 3-7%^[13]. Tuy nhiên, các quy trình tiền xử lý thông thường yêu cầu sửa đổi mã thủ công để điều chỉnh lọc, làm mịn hoặc loại bỏ hiện vật—các nhà phát triển báo cáo dành 40-60% thời gian dự án cho lặp lại tiền xử lý^[25]. Các công cụ tương tác có thể giảm gánh nặng này: Nghiên cứu về học máy tương tác^[3] chỉ ra rằng các giao diện trực quan giảm thời gian lặp lại 35% và giảm lỗi 28% so với các quy trình làm việc dựa trên mã. Framework được đề xuất nâng cao khả năng sử dụng bằng cách cho phép căn chỉnh trực quan trực tiếp của các tín hiệu thô và đã xử lý, điều chỉnh cửa sổ tương tác với phản hồi tức thì và gắn nhãn lại nhanh chóng—giảm rào cản nhận thức và kỹ thuật cho lặp lại nhanh.

2.8 Trích xuất và Biểu diễn Đặc trưng

Các đặc trưng miền thời gian thủ công (trung bình, phương sai, RMS, độ lệch, độ nhọn, tỷ lệ qua không) và miền tần số (năng lượng phổ, tần số thống trị, entropy phổ) vẫn hiệu quả cho các mô hình ML cổ điển khi được trích xuất một cách có hệ thống^[26]. Các bộ đặc trưng điển hình dao động từ 60-150 đặc trưng mỗi cửa sổ (10-15 mỗi trục cảm biến)^[41]. Lựa chọn đặc trưng sử dụng thông tin hỗ tương, kiểm định F ANOVA hoặc loại bỏ đệ quy có thể giảm chiều 30-50% với mất độ chính xác tối thiểu (<2%), cải thiện dấu chân bộ nhớ và tốc độ suy luận (nhanh hơn 15-30%) trên vi điều khiển^[6]. Tuy nhiên, các đặc trưng miền tần số đặt ra các thách thức triển khai: tính toán FFT trên các hệ thống nhúng không có bộ tăng tốc phần cứng yêu cầu các phép toán $O(N$

$\log N$) và số học dấu phẩy động, tiêu thụ 50-150 ms mỗi cửa sổ trên Cortex-M4 (64 MHz)^[5]. Các lựa chọn thay thế nhẹ bao gồm đếm qua không, đỉnh tự tương quan hoặc các thư viện FFT đã tính toán trước (ARM CMSIS-DSP), nhưng tương đồng đặc trưng giữa huấn luyện (Python NumPy FFT) và triển khai (C++ FFT điểm cố định) yêu cầu xác thực cẩn thận. Trong khi các mạng sâu từ đầu đến cuối có thể học các đặc trưng phân cấp một cách ngầm định—không cần thiết kể đặc trưng thủ công—trích xuất đặc trưng tường minh lại cải thiện khả năng diễn giải (xác định trực/tần số cảm biến nào điều khiển dự đoán) và tính minh bạch triển khai trong các lĩnh vực được quy định hoặc quan trọng về an toàn. Framework kết hợp các mô-đun tính toán đặc trưng hoán đổi được, xác thực tính nhất quán đặc trưng huấn luyện-triển khai và hỗ trợ chẩn đoán từng cửa sổ để hướng dẫn tinh chỉnh.

2.9 Triển khai Biên và Tối ưu hóa

Các ràng buộc tài nguyên xác định các thách thức triển khai biên—RAM hạn chế (32-256 KB), flash (256 KB-1 MB), không có đơn vị dấu phẩy động phần cứng (hoặc hiệu suất FPU hạn chế) và ngân sách điện năng (10-50 mW cho các thiết bị đeo)^[5;30]. Benchmark MLPerf Tiny^[5] thiết lập các số liệu tham chiếu: Cortex-M4F (64 MHz) đạt 10,3 suy luận/giây cho phát hiện từ khóa (mô hình 49 KB), 15,2 suy luận/giây cho phát hiện bất thường (mô hình 32 KB). Các chiến lược tối ưu hóa bao gồm:

- **Lượng tử hóa:** Chuyển đổi float 32-bit sang số nguyên 8-bit giảm kích thước mô hình $4\times$ và tăng tốc suy luận $2-3\times$ với suy giảm độ chính xác 0,5-2%^[24]. Lượng tử hóa độ chính xác hỗn hợp (16-bit cho các lớp nhảy cảm) cân bằng kích thước và độ chính xác.
- **Cắt tỉa Mô hình:** Loại bỏ 40-70% trọng số mạng nơ-ron (cắt tỉa dựa trên độ lớn hoặc cấu trúc) mang lại tăng tốc $2-3\times$ với mất độ chính xác $<1\%$ ^[31].
- **Chưng cất Kiến trúc:** Huấn luyện các mô hình sinh viên nhỏ gọn (10-20% tham số) để bắt chước các mạng giáo viên lớn đạt 85-95% độ chính xác giáo viên với dấu chân nhỏ hơn $5-10\times$ ^[11].
- **Đơn giản hóa Thuật toán:** Các mô hình cổ điển như Random Forests (độ sâu ≤ 10 , 50-100 cây) và SVMs (100-500 vector hỗ trợ) đạt độ chính xác 88-93% với dấu chân 50-200 KB và thời gian suy luận 12-25 ms^[41].

Các khảo sát TinyML gần đây^[11;30] xác định các mẫu triển khai chính: (1) các ứng dụng quan trọng độ trễ ưu tiên ML cổ điển (suy luận 5-20 ms), (2) các kịch bản quan

trọng độ chính xác biện minh cho CNNs/LSTMs nhẹ (suy luận 40-80 ms, độ chính xác cao hơn 1-2%), (3) các triển khai quan trọng điện năng sử dụng lấy mẫu thưa (1-5 Hz) và lượng tử hóa tích cực. Thành phần tạo mã của framework thực hiện xuất được điều chỉnh theo nền tảng, phát ra mã C/C++ có cấu trúc với chuyển đổi tham số nhận thức ràng buộc (ví dụ: tỷ lệ float→int16, mở rộng vòng lặp cho kích thước cửa sổ cố định). Các chế độ tối ưu hóa đa mục tiêu (độ chính xác, tốc độ, điện năng, cân bằng) cho phép ra quyết định có nguyên tắc phù hợp với mục tiêu triển khai, đánh đổi định lượng độ chính xác ($\pm 2-5\%$), độ trễ ($\pm 10-30$ ms) và điện năng ($\pm 5-15$ mW).

2.10 Các Khoảng trống trong Công trình Hiện có

Mặc dù hệ sinh thái công cụ phát triển edge AI đã khá hoàn thiện, một số khoảng trống quan trọng vẫn còn:

1. **Tiền xử lý Tương tác Hạn chế:** Ít hệ thống cho phép người dùng kiểm tra trực quan các tín hiệu cảm biến thô so với đã lọc và tinh chỉnh phân đoạn thủ công trong cùng một môi trường.
2. **Thiếu Minh bạch trong Trích xuất Đặc trưng:** Các trình tạo đặc trưng tự động che giấu các bước dẫn xuất và cản trở việc sử dụng giáo dục hoặc tái tạo nghiên cứu.
3. **Sinh mã Triển khai Thiếu Linh hoạt:** Đầu ra tạo mã thường là nguyên khối, làm phức tạp việc tùy chỉnh hoặc thích ứng theo phần cứng cụ thể.
4. **Đánh đổi Khả năng Tiếp cận và Kiểm soát:** Các nền tảng hiện tại thường thiên về một trong hai thái cực—dễ sử dụng như Teachable Machine, hoặc khả năng cấu hình cao như TensorFlow—ít nền tảng nào cân bằng được cả hai yêu cầu cho các quy trình xử lý dữ liệu cảm biến chuyển động.

Framework được đề xuất giải quyết các khoảng trống này bằng cách thống nhất tiền xử lý tương tác, trích xuất đặc trưng minh bạch, huấn luyện modular và tạo mã nhận thức tối ưu hóa dưới một kiến trúc mở rộng. Những lựa chọn thiết kế này, được thông báo bởi tài liệu được xem xét ở trên, định vị luận văn này như một giải pháp bổ sung bắc cầu khoảng trống giữa sự dễ sử dụng thương mại và tính linh hoạt cấp độ nghiên cứu.

2.11 Tóm tắt

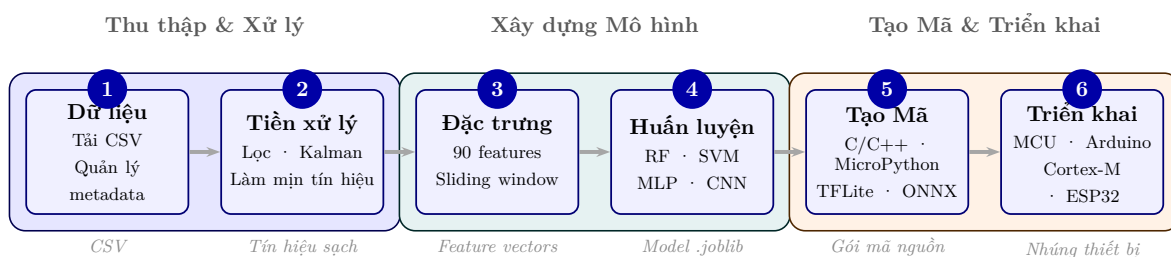
Nghiên cứu và nền tảng công nghiệp trước đây cung cấp nền tảng vững chắc trong theo dõi chuyển động, nhưng không có nền tảng nào cung cấp một quy trình làm việc tích hợp, minh bạch và thân thiện với người dùng được tối ưu hóa cho triển khai biên dựa trên cảm biến IMU, với khả năng phân đoạn dữ liệu tương tác. Công trình này xây dựng dựa trên các thuật toán đã được thiết lập và các thực hành edge ML trong khi giới thiệu các đóng góp mới dân chủ hóa phát triển, tăng tốc lặp lại và cải thiện sẵn sàng triển khai cho các ứng dụng theo dõi chuyển động thực tế.

Chương 3

Phương pháp Luận

3.1 Tổng quan Kiến trúc Framework

Framework được đề xuất triển khai một quy trình từ đầu đến cuối để tạo các mô hình AI được điều chỉnh cho theo dõi chuyển động trên các thiết bị biên. Kiến trúc hệ thống bao gồm năm thành phần chính: (1) tải lên và quản lý dữ liệu, (2) tiền xử lý tương tác với phân đoạn trực quan, (3) trích xuất đặc trưng, (4) huấn luyện mô hình với tối ưu hóa siêu tham số, và (5) tạo mã đặc thù cho nền tảng. Framework được triển khai bằng Python với giao diện web Dash, scikit-learn và PyTorch cho học máy, cùng các trình tạo mã tùy chỉnh cho triển khai đa nền tảng (C/C++, MicroPython, TFLite Micro, ONNX Runtime). Hình 3.1.1 minh họa quy trình làm việc tổng thể.



Hình 3.1.1: Tổng quan kiến trúc framework: quy trình sáu bước từ tải dữ liệu thô đến triển khai mô hình trên thiết bị biên, hỗ trợ các thuật toán RF, SVM, MLP, CNN và xuất mã đa nền tảng

3.2 Thu thập Dữ liệu và Nền tảng Tham chiếu

3.2.1 Thiết bị tham chiếu cho thu thập và benchmark

Để thu thập dữ liệu và đo benchmark phần cứng đại diện, nghiên cứu sử dụng vi điều khiển Seeed XIAO nRF52840 Sense^[33], có bộ xử lý ARM Cortex-M4F (64MHz, 256KB RAM, 1MB Flash) và cảm biến IMU 6 trục LSM6DS3 tích hợp giao tiếp qua I2C tại địa chỉ 0x6A. LSM6DS3 cung cấp dữ liệu gia tốc kế 3 trục (được cấu hình ở phạm vi $\pm 2g$) và dữ liệu con quay hồi chuyển 3 trục, được lấy mẫu ở 100Hz để đảm bảo độ phân giải thời gian đầy đủ để bắt giữ động lực học chuyển động con người trong khi cân bằng các ràng buộc bộ nhớ. Việc lựa chọn thiết bị này không hàm ý framework chỉ nhắm đến duy nhất XIAO; trong luận văn này, XIAO đóng vai trò nền tảng tham chiếu thuộc lớp ARM Cortex-M4F phổ biến trong các ứng dụng đeo, còn năng lực cốt lõi của framework nằm ở khả năng đóng gói cùng một mô hình sang nhiều đích triển khai khác nhau.

3.2.2 Giao thức Thu thập Dữ liệu

Firmware thu thập dữ liệu triển khai quy trình sau:

1. Khởi tạo giao tiếp I2C và xác minh kết nối IMU
2. Cấu hình cảm biến gia tốc kế (phạm vi $\pm 2g$) và con quay hồi chuyển
3. Thu thập các số đọc cảm biến thô ở 100Hz (khoảng 10ms)
4. Chuyển đổi giá trị gia tốc kế sang m/s^2 sử dụng hệ số chuyển đổi 9,80665
5. Duy trì các số đọc con quay hồi chuyển ở độ/giây (deg/s)
6. Đầu ra định dạng CSV: aX, aY, aZ, gX, gY, gZ, Time_seconds

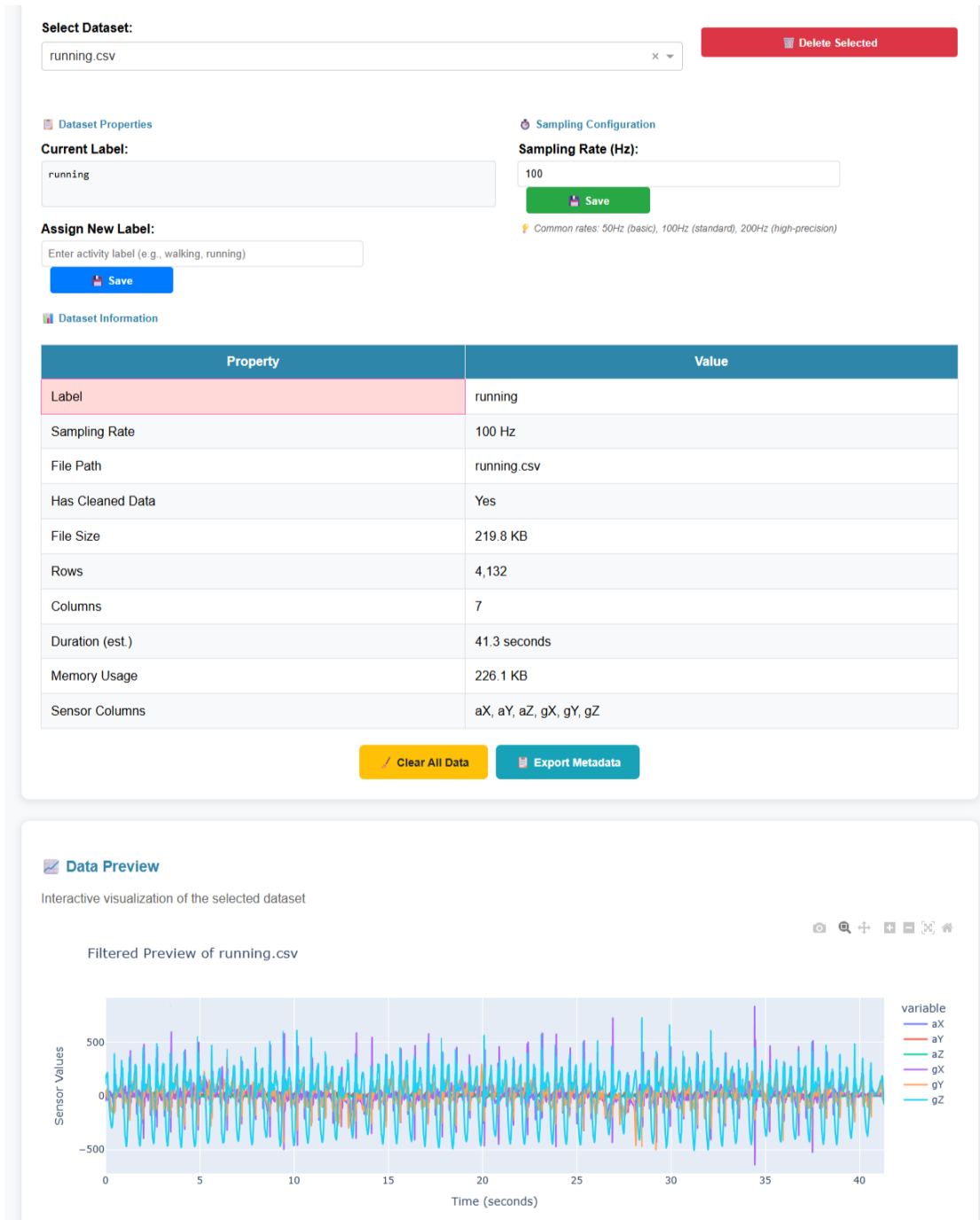
Tốc độ lấy mẫu chuẩn hóa 100Hz đảm bảo tính nhất quán giữa thu thập dữ liệu huấn luyện và triển khai suy luận thời gian thực, loại bỏ các vấn đề không khớp tần số có thể làm suy giảm hiệu suất mô hình. Trong pha triển khai, framework tách riêng pipeline học máy khỏi lớp driver cảm biến và lớp đóng gói mã; do đó, thiết bị thu thập dữ liệu có thể khác với thiết bị suy luận cuối cùng miễn là các giả định về cột cảm biến, đơn vị đo và tần số lấy mẫu vẫn được giữ nhất quán.

3.3 Quy trình Tiền xử lý Tương tác

3.3.1 Tải lên và Trực quan hóa Dữ liệu

Người dùng tải lên các tập dữ liệu CSV đa lớp chứa các phép đo IMU có dấu thời gian. Hệ thống hỗ trợ tổ chức tệp theo hoạt động (ví dụ: `walking.csv`, `running.csv`) trong đó mỗi tệp tương ứng với một lớp hoạt động. Sau khi tải lên, framework hiển thị các biểu đồ đồng bộ của các tín hiệu gia tốc kế và con quay hồi chuyển thô trên tất cả sáu trục, cho phép kiểm tra trực quan chất lượng dữ liệu. Hình 3.3.1 cho thấy giao diện tải lên dữ liệu với chọn tệp và trực quan hóa tập dữ liệu ban đầu.

3.3. Quy trình Tiền xử lý Tương tác



Hình 3.3.1: Giao diện web cho tải lên tệp CSV và quản lý tập dữ liệu. Giao diện cho phép người dùng tải lên các tệp dữ liệu cảm biến, xác thực tên cột, tự động phát hiện tần số lấy mẫu và cung cấp bản xem trước tương tác của chuỗi thời gian sáu trục thô.

3.3.2 Làm sạch và Làm mịn Tín hiệu

Mô-đun tiền xử lý cung cấp các hoạt động làm sạch có thể cấu hình:

- **Loại bỏ Ngoại lệ:** Xác định và loại bỏ các mẫu vượt quá ngưỡng độ lệch chuẩn đã chỉ định (mặc định: 3σ), giải quyết các bất thường cảm biến và hiện vật nhiễu điện từ.
- **Làm mịn:** Áp dụng các bộ lọc trung bình động với kích thước cửa sổ do người dùng xác định (mặc định: 5 mẫu) để giảm nhiễu tần số cao trong khi bảo tồn các đặc tính chuyển động.

Dữ liệu đã làm sạch được trực quan hóa cùng với các tín hiệu gốc, cho phép người dùng xác minh hiệu quả tiền xử lý trước khi phân đoạn.

3.3.3 Bộ lọc Kalman cho Tiền xử lý Tín hiệu

Framework giới thiệu bộ lọc Kalman constant-velocity 1D cho mỗi kênh cảm biến IMU như một giải pháp tiền xử lý tín hiệu tiên tiến^[22]. Đây là bước đột phá quan trọng trong việc giải quyết *khoảng cách tương đồng huấn luyện-triển khai* (training-deployment parity gap) mà các bộ lọc IIR truyền thống gặp phải.

Vấn đề Parity với Bộ lọc IIR

Các bộ lọc Butterworth low-pass truyền thống tồn tại lỗi hỏng tương đồng:

- **Huấn luyện (Python):** Sử dụng `filtfilt` (zero-phase, non-causal) — xử lý dữ liệu theo hai chiều (thuận và ngược) để loại bỏ méo pha
- **Triển khai (C++):** Sử dụng `lfilter` (causal, forward-only) — chỉ xử lý theo một chiều do ràng buộc thời gian thực

Khoảng cách này gây sai lệch pha và biên độ trong các đặc trưng thống kê nhạy cảm như skewness và kurtosis, dẫn đến hiệu suất mô hình giảm sút khi triển khai thực tế.

Mô hình Toán học Kalman Filter

Bộ lọc Kalman được triển khai với mô hình constant-velocity cho mỗi kênh:

State vector:

$$\mathbf{x}_k = \begin{bmatrix} p_k \\ v_k \end{bmatrix}$$

với p_k là vị trí (tín hiệu cảm biến) và v_k là vận tốc tại thời điểm k .

Transition matrix:

$$F = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$$

Observation matrix:

$$H = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

Process noise covariance:

$$Q = q \begin{bmatrix} \frac{\Delta t^3}{3} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^2}{2} & \Delta t \end{bmatrix}$$

Measurement noise covariance: $R = r$ (scalar)

Các tham số có thể điều chỉnh:

- *process_noise* (q): Giá trị nhỏ hơn tạo ra đầu ra mượt hơn. Mặc định: 10^{-3}
- *measurement_noise* (r): Giá trị lớn hơn tạo ra đầu ra mượt hơn. Mặc định: 10^{-1}

Ưu điểm Parity của Kalman Filter

Bộ lọc Kalman mang lại những lợi thế quan trọng so với bộ lọc IIR:

Bảng 3.3.1: So sánh Bộ lọc IIR và Kalman Filter về Parity

Đặc tính	IIR (Butterworth)	Kalman Filter
Bộ lọc huấn luyện	<code>filtfilt</code> (non-causal)	Forward-only (causal)
Bộ lọc thiết bị	<code>lfilter</code> (causal)	Forward-only (causal)
Tương đồng Parity	(!) Có khoảng cách (phase difference)	✓ Hoàn toàn chính xác
Kiểm soát độ mượt	Tần số cắt + bậc filter	Tham số Q và R
Tính thích ứng	Không (hệ số cố định)	Có (gain thích ứng theo tín hiệu)
Bộ nhớ mỗi kênh	$2 \times \text{order floats}$	2 state + 4 covariance floats

Đóng góp về Parity: Bộ lọc Kalman là bộ lọc tiền xử lý đầu tiên trong framework với *khoảng cách parity bằng không*. Cả IIR và Savitzky-Golay đều có sự khác biệt nội tại giữa huấn luyện-triển khai (zero-phase vs causal, offline vs streaming). Bộ lọc Kalman loại bỏ hoàn toàn lớp lỗi này.

3.3.4 Phân đoạn Cửa sổ Thời gian Kéo thả

Một đóng góp mới của framework này là *giao diện cửa sổ kéo thả tương tác* cho phân đoạn hoạt động. Các tính năng chính bao gồm:

- **Đặt Cửa sổ Trực quan:** Người dùng nhấp và kéo trực tiếp trên các biểu đồ tín hiệu để xác định ranh giới thời gian của các trường hợp hoạt động, cung cấp kiểm soát trực quan đối với phân đoạn mà không cần mã hóa.
- **Hỗ trợ Đa Cửa sổ:** Nhiều cửa sổ không chồng lấp có thể được xác định mỗi tập hoạt động, phù hợp với các tập dữ liệu với nhiều trường hợp của cùng một hoạt động.
- **Phản hồi Thời gian Thực:** Các cửa sổ đã chọn được làm nổi bật trên biểu đồ với các màu riêng biệt và các đoạn được trích xuất có sẵn ngay lập tức để kiểm tra.
- **Gán nhãn Từng Cửa sổ:** Mỗi cửa sổ được tự động gán nhãn hoạt động từ tập cha, nhưng có thể được gán nhãn lại thủ công nếu cần (ví dụ: trích xuất các đoạn *standing* từ các tập *walking* trong các giai đoạn chuyển tiếp).

Cách tiếp cận này giảm đáng kể thời gian chú thích so với nhập dấu thời gian thủ công hoặc phân đoạn lập trình và loại bỏ các lỗi phổ biến trong đặc tả ranh giới. Hình 3.3.2 chứng minh giao diện cửa sổ kéo thả tương tác, là tính năng phân biệt chính của framework này.

3.4. Trích xuất Đặc trưng



Hình 3.3.2: Giao diện cửa sổ kéo thả tương tác cho phân đoạn hoạt động. Người dùng kiểm tra trực quan chuỗi thời gian gia tốc kế và con quay hồi chuyển, kéo các cửa sổ hình chữ nhật qua các vùng tương ứng với các hoạt động riêng biệt, gán nhãn hoạt động và điều chỉnh ranh giới qua tương tác nhấp và kéo. Điều này loại bỏ tính toán dấu thời gian thủ công và làm cho framework có thể tiếp cận được cho những người không lập trình.

3.4 Trích xuất Đặc trưng

3.4.1 Phương pháp Cửa sổ Trượt

Các tín hiệu đã tiền xử lý được phân đoạn thành các cửa sổ chồng lấp kích thước cố định để trích xuất đặc trưng. Sau khi xác thực thực nghiệm và so sánh với các khuyến nghị tài liệu^[6], cấu hình đã được tối ưu hóa là:

- **Tốc độ Lấy mẫu:** 100Hz (cố định)

- **Kích thước Cửa sổ:** 150 mẫu (1,5 giây ở 100Hz)
- **Chồng lấp:** 50% (bước 75 mẫu)

Cửa sổ 1,5 giây cung cấp ngữ cảnh thời gian đầy đủ để bắt giữ các chu kỳ chuyển động hoàn chỉnh cho hầu hết các hoạt động (thời lượng bước đi bộ $\approx 1,0-1,2$ giây^[43]), trong khi chồng lấp 50% đảm bảo tính liên tục thời gian và giảm hiệu ứng cạnh tại ranh giới cửa sổ. Điều này mang lại dự đoán mỗi 0,75 giây, cân bằng khả năng phản hồi với hiệu quả tính toán.

Lưu ý Triển khai: Các tham số này hiện được mã hóa cứng trong cả quy trình huấn luyện (`utils/model_training.py`) và mô-đun tạo mã (`deployment/base_generator.py`) để đảm bảo tính nhất quán huấn luyện-triển khai. Công việc tương lai có thể phơi bày chúng như các tham số có thể cấu hình bởi người dùng với xác thực để ngăn chặn các cấu hình không tương thích. Framework giả định các mẫu được gán nhãn vượt quá kích thước cửa sổ tối thiểu (1,5 giây); các đoạn ngắn hơn không được đệm bằng không và có thể tạo ra các đặc trưng không hợp lệ.

3.4.2 Bộ Đặc trưng

Cho mỗi cửa sổ và mỗi trục cảm biến (tổng cộng 6 trục), framework trích xuất 15 đặc trưng thống kê miền thời gian. Các đặc trưng miền tần số đã được cố ý loại trừ sau khi xác định các vấn đề tương đồng huấn luyện-triển khai: các triển khai FFT khác nhau đáng kể giữa Python (NumPy) và C++ nhúng (xấp xỉ hoặc ARM CMSIS-DSP), dẫn đến sự khác biệt giá trị đặc trưng làm suy giảm độ chính xác mô hình đã triển khai.

Lý do Loại trừ Đặc trưng FFT: Trong khi các đặc trưng miền tần số (năng lượng phổ, tần số thống trị, entropy phổ) thường được sử dụng trong tài liệu HAR^[6], chúng giới thiệu các thách thức triển khai đáng kể. FFT NumPy của Python sử dụng các thuật toán dấu phẩy động độ chính xác kép được tối ưu hóa cao (backends FFT-PACK/FFTW), trong khi C++ nhúng thường dựa vào các xấp xỉ điểm cố định hoặc các thư viện đặc thù cho nền tảng (ARM CMSIS-DSP) với độ chính xác số khác nhau. Các thử nghiệm sơ bộ cho thấy rằng các cửa sổ đầu vào giống hệt nhau tạo ra các phổ độ lớn FFT khác nhau 5-15% giữa Python và C++, khiến các mô hình đã huấn luyện phân loại sai 8-12% cửa sổ trong triển khai mặc dù độ chính xác huấn luyện 95%+. "Không khớp huấn luyện-triển khai" này là một vấn đề quan trọng nhưng ít được báo cáo trong edge ML^[5]. Bằng cách hạn chế các đặc trưng miền thời gian với các tính toán dạng đóng xác định (trung bình, phương sai, v.v.), hệ thống đảm bảo trích xuất đặc trưng giống hệt bit trên các nền tảng, loại bỏ chế độ lỗi này. Tỷ lệ qua không và

tỷ lệ qua trung bình phục vụ như các proxy tần số, bắt giữ thông tin tuần hoàn mà không có chi phí FFT.

Các Đặc trưng Miền Thời gian (15 mỗi trục):

- **Xu hướng Trung tâm:** Trung bình, Trung vị
- **Phân tán:** Độ lệch Chuẩn, Phương sai, Phạm vi (Max - Min), Phạm vi Liên tứ phân vị (IQR)
- **Cực trị:** Tối thiểu, Tối đa, Phần trăm thứ 25 (Q1), Phần trăm thứ 75 (Q3)
- **Hình dạng:** Độ lệch $\gamma = \frac{E[(X-\mu)^3]}{\sigma^3}$, Độ nhọn $\kappa = \frac{E[(X-\mu)^4]}{\sigma^4} - 3$ (hiệu chỉnh độ lệch)
- **Năng lượng:** Căn Trung bình Bình phương (RMS) $= \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$, Năng lượng Tín hiệu $E = \sum_{i=1}^N x_i^2$
- **Proxy Tần số:** Tỷ lệ Qua Không, Tỷ lệ Qua Trung bình (các lần qua tín hiệu qua trung bình)

Tổng số chiều vector đặc trưng: $15 \times 6 = 90$ đặc trưng mỗi cửa sổ. Cách tiếp cận chỉ miền thời gian này đảm bảo tương đồng hoàn hảo giữa huấn luyện (Python pandas/NumPy) và triển khai (tính toán trực tiếp C++), quan trọng để đạt được hiệu suất mô hình nhất quán trên các thiết bị biên. Biểu diễn thống kê toàn diện bắt giữ biên độ chuyển động, động lực học, đặc tính phân phối và thông tin giả tần số đầy đủ để phân biệt các hoạt động với các dấu hiệu sinh cơ học riêng biệt.

3.4.3 Chiến lược Đệm Cửa sổ (Window Padding Strategy)

Trong quá trình phân đoạn dữ liệu, các cửa sổ thời gian được trích xuất từ tín hiệu thô có thể ngắn hơn kích thước mục tiêu (150 mẫu ở 100Hz = 1,5 giây). Điều này xảy ra khi người dùng chọn các phân đoạn hoạt động ngắn thông qua giao diện kéo thả, hoặc khi cửa sổ trượt cuối cùng trong một đoạn không đủ dài.

Vấn đề với Đệm Số Không (Zero-Padding) Phương pháp đệm bằng số không—thường được sử dụng trong xử lý tín hiệu số^[26]—tạo ra các hàng dữ liệu có tất cả giá trị cảm biến bằng 0. Đối với gia tốc kế, điều này có nghĩa $a_x = a_y = a_z = 0$, dẫn đến magnitude gia tốc:

$$\|\mathbf{a}\| = \sqrt{a_x^2 + a_y^2 + a_z^2} = 0 \quad (3.4.1)$$

Tuy nhiên, trong thực tế, magnitude gia tốc *luôn lớn hơn 0* do gia tốc trọng trường ($\approx 9,81 \text{ m/s}^2$), kể cả khi thiết bị hoàn toàn đứng yên. Do đó, zero-padding tạo ra các giá trị **vật lý bất khả thi** (physically impossible) gây nhiều nghi ngờ cho phân phối đặc trưng thống kê: $\min = 0$ thay vì $\approx 9,81$, median và phần trăm thứ 25 bị kéo về 0.

Giải pháp: Lặp Giá trị Biên (Edge-Value Replication) Framework sử dụng chiến lược lặp giá trị cuối cùng của cửa sổ để đệm đến kích thước mục tiêu. Cho cửa sổ $\mathbf{W} = [w_1, w_2, \dots, w_k]$ với $k < N$ (kích thước mục tiêu), cửa sổ đệm trở thành:

$$\mathbf{W}' = [w_1, w_2, \dots, w_k, \underbrace{w_k, w_k, \dots, w_k}_{N-k \text{ lần}}] \quad (3.4.2)$$

Phương pháp này: (1) bảo toàn các đặc tính thống kê tín hiệu gốc, (2) không tạo ra artifact vật lý bất khả thi, và (3) đảm bảo tính nhất quán với dữ liệu suy luận trên thiết bị biên—nơi bộ đệm trượt luôn chứa đầy đủ dữ liệu cảm biến thực. Ngoài ra, các cửa sổ có ít hơn 30% số mẫu mục tiêu được loại bỏ hoàn toàn, vì không chứa đủ thông tin chuyển động để trích xuất đặc trưng có ý nghĩa.

Ý nghĩa thực nghiệm: Trong quá trình phát triển, zero-padding khiến mô hình đã triển khai luôn dự đoán sai lớp `walking_downstairs` bất kể hoạt động thực tế, do các đặc trưng đầu vào nằm ngoài phân phối huấn luyện (xem Mục 5.5 để phân tích chi tiết).

3.4.4 Tăng cường Dữ liệu cho Huấn luyện (Data Augmentation)

Để cải thiện tính tổng quát hóa của mô hình và giảm hiện tượng quá khớp (overfitting), đặc biệt với các tập dữ liệu nhỏ điển hình của thu thập dữ liệu đối tượng đơn, framework tích hợp một hệ thống tăng cường dữ liệu (data augmentation) được thiết kế riêng cho dữ liệu IMU^[21;38]. Tăng cường được thực hiện ở cấp cửa sổ—trước khi trích xuất đặc trưng—để tạo ra các biến thể thực tế của tín hiệu gốc mà không làm thay đổi bản chất hoạt động.

Các Phương pháp Tăng cường

Năm phương pháp biến đổi được triển khai, mỗi phương pháp mô phỏng một nguồn biến thiên thực tế khác nhau trong dữ liệu cảm biến quán tính:

1. **Nhiều Jitter**: Thêm nhiễu Gaussian $\mathcal{N}(0, \sigma^2)$ vào mỗi mẫu, mô phỏng nhiễu cảm biến tự nhiên và biến thiên môi trường. Tham số σ (mặc định: 0,05) kiểm soát cường độ nhiễu^[38].
2. **Tỷ lệ (Scaling)**: Nhân toàn bộ cửa sổ với hệ số tỷ lệ ngẫu nhiên $s \sim \mathcal{N}(1, \sigma_s^2)$, mô phỏng sự khác biệt về cường độ chuyển động giữa các lần thực hiện. Ví dụ, $s = 1,15$ tương ứng với bước đi mạnh hơn 15%.
3. **Xoay (Rotation)**: Áp dụng ma trận xoay ngẫu nhiên lên các trục gia tốc kế và con quay hồi chuyển, mô phỏng sự thay đổi vị trí đặt cảm biến trên cơ thể—một nguồn biến thiên phổ biến và quan trọng trong ứng dụng HAR thực tế^[37]. Góc xoay tối đa có thể cấu hình (mặc định: 15°).
4. **Biến dạng Thời gian (Time Warping)**: Áp dụng biến đổi phi tuyến lên trục thời gian qua nội suy cubic spline, mô phỏng sự biến thiên tốc độ tự nhiên trong thực hiện hoạt động—ví dụ, bước đi bắt đầu chậm và tăng tốc dần^[23].
5. **Hoán vị (Permutation)**: Chia cửa sổ thành k đoạn (mặc định: $k = 4$) và hoán vị thứ tự ngẫu nhiên, giúp mô hình học các đặc trưng thống kê cục bộ thay vì phụ thuộc vào thứ tự thời gian toàn cục.

Tăng cường Nhận biết Lớp (Class-Aware Augmentation)

Một đóng góp quan trọng của framework là chiến lược tăng cường *nhận biết lớp*, giải quyết một vấn đề tinh tế nhưng nghiêm trọng: việc áp dụng các phép biến đổi mạnh (jitter, rotation, scaling) lên các hoạt động tĩnh (static activities) như *still*, *standing* hay *sitting* tạo ra tín hiệu nhân tạo có các dao động nhỏ giống với hoạt động cường độ thấp (ví dụ: *walking_downstairs* với rung nhẹ). Điều này gây **nhầm lẫn lớp** (class confusion), làm suy giảm đáng kể hiệu suất phát hiện hoạt động tĩnh.

Thiết kế giải pháp: Framework tự động phát hiện các nhân hoạt động tĩnh thông qua đối chiếu từ khóa (*still*, *stand*, *sit*, *lying*, *idle*) và áp dụng chiến lược tăng cường phân biệt:

- **Hoạt động động** (dynamic): Áp dụng đầy đủ cả năm phương pháp với tham số tiêu chuẩn ($\sigma_{\text{jitter}} = 0,05$)
- **Hoạt động tĩnh** (static): Chỉ áp dụng nhiễu vi mô (micro-jitter) với $\sigma = 0,01$ —đủ để tạo tính đa dạng cảm biến thực tế mà không phá vỡ đặc tính “gần bất động” (near-stationary) vốn xác định hoạt động tĩnh

Người dùng cũng có thể xác định thủ công danh sách nhãn tĩnh qua giao diện framework, phù hợp với các ứng dụng có nhãn không thuộc bộ từ khóa mặc định.

Cơ sở lý thuyết: Chiến lược này tuân thủ nguyên tắc rằng tăng cường phải bảo toàn các đặc tính phân biệt (discriminative properties) của mỗi lớp^[8]. Với hoạt động tĩnh, đặc tính phân biệt chính là biên độ dao động cực thấp ($\sigma_{acc} < 0,1 \text{ m/s}^2$); các phép biến đổi mạnh phá hủy chính xác đặc tính này, trong khi micro-jitter bảo toàn nó.

3.4.5 Ngưỡng Tin cậy cho Từ chối Hoạt động Không xác định

Trong triển khai thực tế, thiết bị biên thường ghi nhận các chuyển động không thuộc bất kỳ lớp nào đã huấn luyện (ví dụ: ngồi xuống, cúi người, hoặc đeo thiết bị ở vị trí bất thường). Các hệ thống phân loại truyền thống bắt buộc phải gán một nhãn cho mọi đầu vào, bất kể mức độ tin cậy, dẫn đến các dự đoán sai với độ tin cậy cao—hiện tượng được gọi là *overconfident misclassification*^[18;20].

Cơ chế từ chối: Framework tích hợp ngưỡng tin cậy (confidence threshold) trực tiếp vào mã triển khai đã tạo (C/C++ hoặc MicroPython). Mỗi mô hình tính toán xác suất từng lớp, và nếu xác suất tối đa $\max_c P(y = c|\mathbf{x})$ thấp hơn ngưỡng τ (mặc định: $\tau = 0,6$), dự đoán trả về “unknown” thay vì ép buộc một nhãn cụ thể:

$$\hat{y} = \begin{cases} \arg \max_c P(y = c|\mathbf{x}) & \text{nếu } \max_c P(y = c|\mathbf{x}) \geq \tau \\ \text{unknown} & \text{nếu } \max_c P(y = c|\mathbf{x}) < \tau \end{cases} \quad (3.4.3)$$

Tính toán tin cậy theo thuật toán: Mỗi thuật toán sử dụng cơ chế tính xác suất riêng có sẵn:

- **Neural Network/CNN:** Hàm softmax trên logits đầu ra: $P(c) = \frac{e^{z_c}}{\sum_j e^{z_j}}$
- **Random Forest:** Tỷ lệ bỏ phiếu: $P(c) = \frac{\text{số cây bỏ phiếu cho } c}{\text{tổng số cây}}$
- **SVM:** Softmax trên điểm quyết định One-vs-Rest (OvR)

Giá trị $\tau = 0,6$ được chọn để cân bằng giữa tỷ lệ từ chối và tỷ lệ dương tính giả: với bài toán 5 lớp, xác suất ngẫu nhiên là 20%, do đó $\tau = 0,6$ yêu cầu mô hình tin cậy gấp 3 lần ngẫu nhiên.

So sánh với nền tảng thương mại: Edge Impulse cung cấp chức năng “anomaly detection” yêu cầu huấn luyện một khối riêng biệt trên dữ liệu bổ sung. Cách tiếp cận được đề xuất trích xuất tin cậy trực tiếp từ đầu ra mô hình hiện có—không tốn thêm bộ nhớ, không yêu cầu dữ liệu huấn luyện bổ sung, và tích hợp tự nhiên vào quy trình suy luận đã có.

3.5 Huấn luyện Mô hình Học máy

3.5.1 Các Thuật toán Được Hỗ trợ

Framework tích hợp bốn nhóm thuật toán học máy, bao gồm cả học máy cổ điển (scikit-learn) và học sâu (PyTorch), được chọn vì hiệu quả đã được chứng minh trong nhận dạng hoạt động và tính khả thi cho triển khai nhúng:

Random Forest (RF) Phương pháp ensemble kết hợp nhiều cây quyết định với bagging và ngẫu nhiên hóa đặc trưng. Cấu hình:

- Mặc định: 100 cây, độ sâu tối đa 20
- Cân bằng lớp: `class_weight='balanced'` tự động trọng số các mẫu tỷ lệ nghịch với tần suất lớp
- Tiêu chí: Độ bất thuần Gini

Support Vector Machine (SVM) Bộ phân loại phân biệt tìm các siêu phẳng tối ưu qua các phương pháp kernel. Cấu hình:

- Kernel: Radial Basis Function (RBF) với tỷ lệ gamma tự động
- Chiến lược đa lớp: One-vs-One (OvO)
- Cân bằng lớp: `class_weight='balanced'`
- Chính quy hóa: $C=1.0$ (mặc định)

Neural Network (NN) Perceptron đa lớp với lan truyền ngược. Cấu hình:

- Kiến trúc: 90 đầu vào $\rightarrow$ 100 ẩn $\rightarrow$ num_classes đầu ra
- Kích hoạt: ReLU (lớp ẩn), Softmax (lớp đầu ra)
- Trình tối ưu: Adam với tốc độ học 0,001, $\beta_1 = 0,9$, $\beta_2 = 0,999$
- Chính quy hóa: Phạt L2 $\alpha = 0,0001$
- Huấn luyện: Dừng sớm với patience=10, tối đa 500 epochs
- Kích thước Batch: Batch đầy đủ (toàn bộ tập huấn luyện)

Kiến trúc một lớp ẩn cung cấp khả năng đầy đủ để học các mẫu hoạt động phi tuyến trong khi vẫn đủ nhỏ để triển khai hiệu quả trên vi điều khiển (ma trận trọng số cố định, không phân bổ bộ nhớ động). Ngoài ra, framework hỗ trợ kiến trúc MLP tùy chỉnh qua PyTorch với nhiều lớp ẩn cấu hình được và hệ thống xuất trọng số đã huấn luyện sang định dạng tương thích scikit-learn để tạo mã triển khai.

Convolutional Neural Network (CNN) Mạng tích chập 1D được triển khai qua PyTorch, xử lý trực tiếp cửa sổ cảm biến thô (end-to-end) mà không cần trích xuất đặc trưng thủ công. Cấu hình:

- Đầu vào: Cửa sổ thô $150 \text{ mẫu} \times 6 \text{ kênh cảm biến}$
- Kiến trúc: 2–3 lớp Conv1D (kernel size 3–5, bộ lọc 32–64) + BatchNorm + MaxPool + Flatten + Dense
- Trình tối ưu: Adam với OneCycleLR scheduler
- Chính quy hóa: Dropout 0,3–0,5 giữa các lớp Dense
- Xuất mã: Trọng số được xuất thành mảng hằng số C/C++ qua `export_cnn_weights()`, hoặc chuyển đổi sang TFLite Micro với lượng tử hóa INT8

CNN phù hợp cho các bài toán cần độ chính xác cao hơn nơi các đặc trưng thống kê thủ công chưa đủ phân biệt—mô hình học trực tiếp các biểu diễn phân cấp từ tín hiệu thô, nhưng đòi hỏi yêu cầu nhiều dữ liệu huấn luyện hơn và bộ nhớ triển khai lớn hơn (500 KB–2 MB so với 50–200 KB cho mô hình cổ điển).

3.5.2 Giao thức Huấn luyện và Đánh giá

Chiến lược Chia Dữ liệu

Framework triển khai một phân chia ba chiều nghiêm ngặt để ngăn chặn rò rỉ thông tin và đảm bảo ước lượng hiệu suất không thiên vị:

1. **Tập Huấn luyện (60%)**: Chỉ sử dụng cho huấn luyện mô hình (khớp các tham số mô hình)
2. **Tập Xác thực (20%)**: Sử dụng cho điều chỉnh siêu tham số, lựa chọn mô hình và giám sát tiến trình huấn luyện
3. **Tập Kiểm tra (20%)**: Hoàn toàn được giữ lại chỉ cho đánh giá cuối cùng; không bao giờ được truy cập trong quá trình huấn luyện hoặc tối ưu hóa siêu tham số

Tất cả các phân chia sử dụng lấy mẫu phân tầng để bảo tồn tỷ lệ lớp trên các tập, quan trọng cho các nhiệm vụ nhận dạng hoạt động mất cân bằng.

Phân biệt Tập Xác thực so với Kiểm tra: Tập xác thực cho phép phát triển mô hình lặp lại—người dùng có thể điều chỉnh siêu tham số, thử các thuật toán khác nhau hoặc điều chỉnh các bước tiền xử lý trong khi giám sát độ chính xác xác thực mà không làm nhiễu tập kiểm tra. Tập kiểm tra được đánh giá *chính xác một lần* cho mỗi mô hình đã huấn luyện để báo cáo hiệu suất cuối cùng, cung cấp một ước lượng không thiên vị về khái quát hóa cho dữ liệu chưa thấy. Phương pháp luận này tuân theo các thực hành tốt nhất trong nghiên cứu học máy^[14] và loại bỏ rò rỉ thông tin tinh tế xảy ra khi đánh giá lặp lại trên một tập giữ lại duy nhất.

Tùy chọn Xác thực Chéo: Đối với phân tích khám phá, framework cũng hỗ trợ xác thực chéo 5-fold được thực hiện trên tập huấn luyện+xác thực kết hợp (80% dữ liệu), trong khi vẫn bảo tồn tập kiểm tra (20%) là hoàn toàn chưa thấy. Điều này cung cấp các ước lượng hiệu suất mạnh mẽ cho các nghiên cứu so sánh thuật toán mà không yêu cầu một phân chia xác thực riêng biệt.

Xử lý Mất cân bằng Lớp

Các tập dữ liệu chuyển động thường thể hiện mất cân bằng lớp nghiêm trọng (ví dụ: nhiều mẫu *standing* hơn *walking_downstairs*). Huấn luyện mất cân bằng thiên vị các mô hình về các lớp đa số, làm suy giảm nhận dạng lớp thiểu số. Framework giải quyết điều này qua:

- **Cân bằng Trọng số Lớp:** Đối với RF và SVM, tham số `class_weight='balanced'` gán trọng số $w_c = \frac{n_samples}{n_classes \times n_samples_c}$ trong đó $n_samples_c$ là số lượng của lớp c . Điều này tăng đóng góp mất mát của các phân loại sai lớp thiểu số trong quá trình huấn luyện.
- **Phân chia Phân tầng:** Các phân chia huấn luyện-kiểm tra bảo tồn tỷ lệ lớp sử dụng lấy mẫu phân tầng.

Cân bằng lớp là quan trọng cho triển khai thực tế nơi tất cả các lớp hoạt động phải được phát hiện một cách đáng tin cậy, không chỉ những lớp thường xuyên nhất.

3.5.3 Tối ưu hóa Siêu tham số

Framework hỗ trợ điều chỉnh siêu tham số tự động qua GridSearchCV của scikit-learn với xác thực chéo 5-fold. Các siêu tham số có thể tìm kiếm:

Random Forest:

- `n_estimators`: [50, 100, 200]
- `max_depth`: [10, 20, 30, None]
- `min_samples_split`: [2, 5, 10]
- `class_weight`: ['balanced', 'balanced_subsample']

SVM:

- `C`: [0.1, 1, 10, 100]
- `gamma`: ['scale', 'auto', 0.001, 0.01, 0.1]
- `class_weight`: ['balanced']

Neural Network:

- `hidden_layer_sizes`: [(50,), (100,), (100, 50)]
- `alpha`: [0.0001, 0.001, 0.01]
- `learning_rate_init`: [0.001, 0.01]

Tối ưu hóa xác định các cấu hình tối đa hóa độ chính xác thực chéo, thường cải thiện hiệu suất cơ sở 2-5%. Hình 3.5.1 cho thấy giao diện cấu hình huấn luyện nơi người dùng chọn thuật toán, bật tối ưu hóa siêu tham số, cấu hình cân bằng lớp và giám sát tiến trình huấn luyện.

3.6. Tạo Mã cho Triển khai Biên

The screenshot displays a web-based configuration interface for training a model. It is divided into several sections:

- Training Dataset Selection:** A section titled "Select Training Windows:" showing a grid of 15 windows (0-14), each with 80 samples and a duration of approximately 0.8 seconds. There are "Select All" and "Clear All" buttons to the right.
- Feature Engineering Configuration:** Includes a "Normalization Method:" dropdown set to "Standard Scaling (Z-score)" and a "Feature Selection:" dropdown set to "Time-Domain Only".
- Train-Test Split Configuration:** Features a "Train-Test Split Ratio:" slider set to 80% (Training) and 20% (Testing). A "Random State:" input field is set to 42.
- Action Buttons:** Four buttons are present: "Engineer Features" (green), "Perform Train-Test Split" (blue), "Save Training Data" (orange), and "Clear Training Data" (red).
- Feedback/Status:** A green message box at the bottom states "Training Data Saved Successfully" and lists the generated files: "Training File: running.csv_train.csv", "Test File: running.csv_test.csv", and "Metadata: running.csv_metadata.json". It also includes a green checkmark and the text "Data is ready for model training!".

Hình 3.5.1: Bảng cấu hình mô hình cho thấy lựa chọn thuật toán (Random Forest, Neural Network, SVM, CNN), các tùy chọn điều chỉnh siêu tham số với GridSearchCV, các chiến lược cân bằng lớp (trọng số cân bằng, phân chia phân tầng) và giám sát tiến trình huấn luyện thời gian thực.

3.6 Tạo Mã cho Triển khai Biên

3.6.1 Kiến trúc

Hệ thống con tạo mã dịch các mô hình đã huấn luyện thành các gói triển khai phù hợp với nhiều họ thiết bị và môi trường thực thi khác nhau, thay vì nhắm đến một bo mạch đơn lẻ. Kiến trúc tuân theo mẫu thiết kế Factory, trong đó generator được chọn theo ba trục: *loại mô hình*, *họ nền tảng đích* và *cách tiếp cận triển khai*. Danh sách khóa nền tảng và backend cụ thể được tóm tắt trong Bảng 3.6.1.

Ở mức tổ chức mã, kiến trúc gồm ba lớp chính:

- **BaseCodeGenerator:** cung cấp phần dùng chung như sinh tệp, trích xuất đặc trưng, tuần tự hóa tham số, kiểm tra tương đồng và ví dụ tích hợp.
- **Generator theo mô hình:** RandomForestCodeGenerator, SVMCodeGenerator, NeuralNetworkCodeGenerator, CNNCodeGenerator hiện thực logic suy luận đặc

thù thuật toán.

- **Generator theo nền tảng/backend:** các lớp như `ARMCortexMCodeGenerator`, `MicroPythonCodeGenerator`, `ZephyrCodeGenerator`, cùng các backend TFLite Micro và ONNX Runtime, chịu trách nhiệm điều chỉnh mã theo môi trường đích.

Bảng 3.6.1 tóm tắt ma trận đích triển khai được hỗ trợ ở mức generator.

Bảng 3.6.1: Ma trận đích triển khai được hỗ trợ trong kiến trúc generator

Nhóm đích	Khóa nền tảng/backend	Gói sinh ra	Mục đích sử dụng chính
Arduino-compatible	arduino, seeed_xiao, esp32, m5stack, teensy	.h + .cpp + .ino	Quy trình Arduino IDE/PlatformIO cho các bo vi điều khiển phổ biến
ARM Cortex-M thuần	arm_cortex_m	.c	Tích hợp bare-metal hoặc CMSIS trên các vi điều khiển ARM
SDK/RTOS native	generic_c, generic_cpp, esp_idf, zephyr	.h + .c/.cpp + ví dụ	Ứng dụng nhúng native, SDK hãng và RTOS
MicroPython	micropython	module .py + ví dụ .py	Nguyên mẫu nhanh hoặc giáo dục trên board hỗ trợ MicroPython
Backend runtime thay thế	tflite_micro, onnx_runtime	mã glue + model nhị phân	Mở rộng cho các pipeline cần runtime suy luận chuẩn hóa

3.6.2 Lý do Chọn Tạo Mã Trực tiếp thay vì Runtime Suy luận

Một quyết định thiết kế cốt lõi của framework là **tạo mã nguồn trực tiếp** (direct code generation) qua việc trích xuất trọng số, cấu trúc quyết định và tham số mô hình từ các đối tượng scikit-learn/PyTorch đã huấn luyện, sau đó tạo mã C/C++ thuần hoặc MicroPython triển khai thuật toán suy luận tương ứng—thay vì sử dụng các runtime suy luận đóng gói (packaged inference runtimes). Phần này phân tích các phương pháp thay thế và lập luận cho lựa chọn đã chọn.

Các Phương pháp Triển khai Mô hình trên Vi điều khiển

Có ba cách tiếp cận chính để triển khai mô hình ML trên vi điều khiển, mỗi cách thể hiện đánh đổi khác nhau giữa tính tiện lợi, hiệu suất và kiểm soát:

Phương pháp 1: Runtime suy luận (TensorFlow Lite Micro, ONNX Micro Runtime) Mô hình được chuyển đổi sang định dạng trung gian (`.tflite`, `.onnx`), sau đó được tải bởi một runtime suy luận chung trên thiết bị. Runtime đọc biểu đồ tính toán và thực thi từng toán tử (operator) theo thứ tự.

Ưu điểm: Hỗ trợ nhiều kiến trúc mô hình (CNN, LSTM, Transformer); hệ sinh thái lớn với cộng đồng hỗ trợ mạnh; các tối ưu hóa lượng tử hóa tích hợp (INT8, FP16).

Nhược điểm: Runtime chiếm 100–300KB flash bổ sung^[42]—đáng kể trên vi điều khiển có 256KB–1MB flash. Interpreter overhead thêm 15–40% thời gian suy luận so với mã gốc^[24]. Quan trọng nhất, runtime hoạt động như hộp đen: người dùng không thể kiểm tra hoặc xác minh quá trình trích xuất đặc trưng và tính toán bên trong.

Phương pháp 2: Trình biên dịch mô hình (Edge Impulse EON Compiler, Apache TVM, microTVM) Mô hình được biên dịch trước (ahead-of-time compilation) sang mã C tối ưu hóa, loại bỏ runtime nhưng vẫn hoạt động qua chuỗi công cụ đóng gói của nhà cung cấp.

Ưu điểm: Không có interpreter overhead; mã tối ưu hóa cho vi kiến trúc cụ thể; Edge Impulse EON Compiler giảm flash 35–55% so với TFLite Micro (theo số liệu nhà cung cấp^[12]).

Nhược điểm: Chuỗi công cụ đóng gói—mã đã tạo bởi EON Compiler khó kiểm toán và không thể sửa đổi. Apache TVM/microTVM yêu cầu kiến thức biên dịch cấp chuyên gia. Hầu hết các trình biên dịch mô hình bắt buộc mô hình đầu vào theo định dạng ONNX hoặc TFLite, loại trừ các mô hình scikit-learn cổ điển (RF, SVM) mà không qua bước chuyển đổi trung gian phức tạp.

3.6.3 Triển khai TFLite Micro

Framework hỗ trợ đường dẫn triển khai TFLite Micro như một phương pháp bổ sung cho các kiến trúc mô hình phù hợp. Quá trình chuyển đổi tuân theo pipeline $\text{ONNX} \rightarrow \text{TensorFlow} \rightarrow \text{TFLite}$ với các hạn chế quan trọng về phạm vi hỗ trợ^[1].

Phạm vi Hỗ trợ theo Loại Mô hình

Nghiên cứu thực nghiệm cho thấy khả năng chuyển đổi khác nhau đáng kể giữa các loại mô hình:

Neural Network và CNN: Chuyển đổi Trực tiếp (hỗ trợ) Các mô hình neural network (MLP) và CNN từ PyTorch có thể chuyển đổi thành công qua pipeline:

1. PyTorch model $\rightarrow$ ONNX (`torch.onnx.export`)
2. ONNX $\rightarrow$ TensorFlow (`onnx-tf`)
3. TensorFlow $\rightarrow$ TFLite (`tf.lite.TFLiteConverter`)
4. TFLite $\rightarrow$ TFLite Micro (quantization INT8)

Random Forest và SVM: Hạn chế ONNX-ML (không hỗ trợ trực tiếp) Các mô hình Random Forest và SVM gặp rào cản chuyển đổi do hạn chế trong hệ sinh thái ONNX-ML:

- **Scikit-learn $\rightarrow$ ONNX:** `sk2onnx` tạo các toán tử ONNX-ML như `TreeEnsembleClassifier` và `SVMClassifier`
- **ONNX-ML $\rightarrow$ TensorFlow:** `onnx2tf` không hỗ trợ các toán tử ONNX-ML, chỉ hỗ trợ các toán tử ONNX chuẩn (operator domain "ai.onnx")
- **Kết quả:** Pipeline bị gián đoạn, không thể chuyển đổi RF/SVM sang TFLite

Giải pháp Keras Surrogate cho RF/SVM

Để khắc phục hạn chế ONNX-ML, framework triển khai *phương pháp Keras surrogate*:

1. **Sinh dữ liệu huấn luyện surrogate:** Sử dụng mô hình RF/SVM gốc để tạo soft predictions trên tập dữ liệu đại diện
2. **Huấn luyện Keras MLP:** Mạng neural đa lớp học ánh xạ từ features $\rightarrow$ soft predictions của RF/SVM
3. **Chuyển đổi MLP $\rightarrow$ TFLite:** Keras MLP có thể chuyển đổi thành công qua pipeline tiêu chuẩn

Hạn chế Surrogate: Keras surrogate là *xấp xỉ*, không phải triển khai chính xác. Độ thỏa thuận kỳ vọng 80-90% giữa mô hình gốc và surrogate. Cho độ chính xác tối đa, *tạo mã trực tiếp C++ vẫn là phương pháp được khuyến nghị cho RF/SVM*.

Khuyến nghị Sử dụng

- **NN/CNN:** Sử dụng TFLite Micro để tận dụng runtime tối ưu hóa và hỗ trợ quantization INT8
- **RF/SVM:** Ưu tiên tạo mã trực tiếp C++ (chính xác, nhanh, nhỏ gọn). TFLite surrogate chỉ khi yêu cầu tích hợp runtime chuẩn hóa

Phương pháp 3: Tạo mã trực tiếp (Direct Code Generation)—phương pháp đề xuất Framework trích xuất trực tiếp các tham số đã huấn luyện từ đối tượng scikit-learn/PyTorch—trọng số và bias của mạng nơ-ron (`coefs_`, `intercepts_` hoặc `state_dict` PyTorch), support vectors và hệ số kernel của SVM, cấu trúc cây nhị phân của Random Forest, trọng số convolution và batch normalization của CNN—rồi tạo mã nguồn (C/C++ hoặc MicroPython) triển khai thuật toán suy luận tương ứng.

Ưu điểm: Không phụ thuộc bên ngoài—mã tự chứa hoàn toàn; dấu chân bộ nhớ tối thiểu (chỉ chứa trọng số + logic suy luận); minh bạch hoàn toàn—mọi phép tính có thể kiểm tra và xác minh; hỗ trợ trực tiếp các thuật toán ML cổ điển mà không cần chuyển đổi định dạng; xuất được nhiều ngôn ngữ đích (C, C++, MicroPython).

Nhược điểm: Yêu cầu triển khai thủ công logic suy luận cho mỗi thuật toán; không hưởng các tối ưu hóa toán tử tự động cấp vi kiến trúc; khó mở rộng sang các kiến trúc phức tạp (CNN nhiều lớp, attention).

Phân tích So sánh Định lượng

Bảng 3.6.2 tóm tắt so sánh ba phương pháp trên các tiêu chí quan trọng cho triển khai vi điều khiển bị giới hạn tài nguyên:

3.6. Tạo Mã cho Triển khai Biên

Bảng 3.6.2: So sánh phương pháp triển khai mô hình ML trên vi điều khiển

Tiêu chí	Runtime	Compiler	Trực tiếp (Ours)
Overhead flash	100–300KB	15–50KB	0KB
Overhead suy luận	15–40%	<5%	0%
Hỗ trợ RF/SVM	Hạn chế [†]	Không [‡]	Đầy đủ
Minh bạch	Thấp	Trung bình	Hoàn toàn
Cần ONNX/TFLite	Có	Có	Không
Phức tạp triển khai	Thấp	Cao	Trung bình
Hỗ trợ DNN sâu	Tốt	Tốt	Hạn chế

[†]TFLite Micro chỉ hỗ trợ NN/CNN; RF/SVM không thể chuyển đổi trực tiếp do onnx2tf không hỗ trợ ONNX-ML TreeEnsembleClassifier. Keras surrogate có thể sử dụng nhưng là xấp xỉ (80–90% thỏa thuận).

[‡]EON Compiler và TVM tập trung vào mạng nơ-ron; không hỗ trợ RF/SVM gốc.

Lập luận cho Lựa chọn Thiết kế

Quyết định sử dụng tạo mã trực tiếp được lập luận bởi bốn yếu tố:

(1) Dấu chân bộ nhớ tối thiểu Trên các vi điều khiển lớp wearable như Seed XIAO nRF52840 (1MB flash, 256KB RAM), mỗi KB đều quan trọng. TFLite Micro interpreter chiếm khoảng 150–250KB flash^[30;42]—tương đương 15–25% tổng flash khả dụng—chỉ riêng runtime, trước khi tính trọng số mô hình. Phương pháp trực tiếp loại bỏ hoàn toàn overhead này: một mô hình Neural Network chỉ chiếm 95KB (trọng số + logic suy luận + trích xuất đặc trưng), để lại >900KB cho firmware ứng dụng và giúp việc chuyển sang các board cùng lớp tài nguyên trở nên khả thi hơn.

(2) Minh bạch và Khả năng Xác minh Một bài học quan trọng từ phát triển framework (xem Phần 5.5) là *khả năng kiểm tra mã suy luận đã tạo là yêu cầu thiết yếu*, không phải tính năng tùy chọn. Khi phát hiện lỗi zero-padding và sai lệch công thức kurtosis, có thể truy vết trực tiếp đến mã C++ đã tạo, so sánh từng dòng với triển khai Python, và sửa chữa. Với runtime đóng gói (TFLite Micro) hoặc trình biên dịch đóng gói (EON Compiler), quá trình gỡ lỗi này sẽ không thể thực hiện—lỗi sẽ biểu hiện như “mô hình hoạt động kém trên thiết bị” mà không có cách xác định nguyên nhân gốc.

Khả năng xác minh này đặc biệt quan trọng cho ứng dụng y tế và an toàn, nơi các tiêu chuẩn quy định (FDA, IEC 62304) yêu cầu khả năng truy vết và kiểm toán đầy đủ cho mọi thành phần phần mềm liên quan đến quyết định lâm sàng.

(3) Hỗ trợ Trực tiếp các Thuật toán ML Cổ điển Hệ sinh thái TFLite Micro và ONNX Runtime Micro được thiết kế chủ yếu cho mạng nơ-ron. Triển khai Random Forest hoặc SVM qua các nền tảng này yêu cầu chuyển đổi nhiều bước: scikit-learn → ONNX (`skl2onnx`) → TFLite (`onnx-tf`) → TFLite Micro. Mỗi bước chuyển đổi giới thiệu rủi ro sai lệch số và mất thông tin (ví dụ: ONNX không hỗ trợ chính xác tham số `class_weight` của scikit-learn). Phương pháp trực tiếp đọc các thuộc tính nội bộ của scikit-learn (`model.coefs_`, `model.support_vectors_`, `model.estimators_`) mà không qua định dạng trung gian, loại bỏ hoàn toàn chuỗi chuyển đổi dễ lỗi này.

(4) Các Thuật toán Cổ điển là Lựa chọn Phù hợp cho HAR Biên Lựa chọn tạo mã trực tiếp liên kết chặt chẽ với lựa chọn thuật toán. Cho bài toán HAR 5–10 lớp với <1000 vectors đặc trưng và 90 đặc trưng thống kê, các thuật toán cổ điển đạt hiệu suất cạnh tranh (93,8–96,2%) với chi phí tài nguyên thấp hơn đáng kể. Các mô hình này có cấu trúc suy luận đơn giản—nhân ma trận cho NN, duyệt cây cho RF, tính kernel cho SVM—có thể triển khai chính xác trong vài trăm dòng C++ mà không cần framework tính toán phức tạp. Điều này tạo thành một *sweet spot* giữa hiệu suất mô hình và tính khả thi triển khai cho các thiết bị đeo bị giới hạn tài nguyên.

Hạn chế và Các Phương pháp Bổ sung

Tạo mã trực tiếp không phải giải pháp phổ quát. Nó không phù hợp cho:

- **Mạng sâu nhiều lớp:** LSTM, Transformer, hoặc CNN với hơn 4–5 lớp convolution yêu cầu các toán tử phức tạp (attention, bidirectional) mà triển khai thủ công dễ lỗi và khó bảo trì
- **Lượng tử hóa nâng cao:** INT8 inference với hiệu chỉnh phạm vi per-channel yêu cầu hạ tầng runtime mà TFLite Micro cung cấp sẵn
- **Mô hình thay đổi thường xuyên:** Nếu mô hình cần cập nhật OTA (over-the-air) mà không biên dịch lại firmware, runtime interpreter là lựa chọn tốt hơn

Framework đã tích hợp đường dẫn TFLite Micro (Mục 3.6.3) bên cạnh tạo mã trực tiếp, cho phép người dùng chọn phương pháp phù hợp: tạo mã trực tiếp cho ML cổ điển và CNN nhỏ, TFLite Micro cho các mô hình cần lượng tử hóa INT8 đầy đủ.

3.6.4 Cấu trúc Mã Đã Tạo

Đối với mỗi mô hình đã huấn luyện, trình tạo sản xuất một gói triển khai gồm các tệp tham số, tệp cài đặt và tệp ví dụ tích hợp. Cụ thể:

1. **Tệp Header (.h):** Chứa các tham số mô hình (ví dụ: vectors hỗ trợ, cấu trúc cây, ma trận trọng số), khai báo hàm, các macro số lượng đặc trưng/lớp
2. **Tệp Triển khai (.c/.cpp/.py):** Triển khai trích xuất đặc trưng, logic dự đoán (đặc thù cho thuật toán) và các hàm tiện ích, với phần mở rộng phụ thuộc nền tảng đích
3. **Tệp Ví dụ Tích hợp:** Có thể là Arduino sketch (.ino), chương trình C/C++ ví dụ hoặc script MicroPython, minh họa khởi tạo cảm biến, quản lý bộ đệm của sổ trượt, vòng lặp suy luận thời gian thực và đầu ra dự đoán

Xác minh Công thức Thống kê: Mã C++ đã tạo cho trích xuất đặc trưng sử dụng các công thức thống kê được xác minh khớp chính xác với triển khai Python (pandas/NumPy). Cụ thể, các đặc trưng kurtosis và skewness sử dụng *độ lệch chuẩn tổng thể* (population standard deviation, chia n) cho chuẩn hóa z-scores, thay vì độ lệch chuẩn mẫu (chia $n - 1$). Sự khác biệt này, mặc dù nhỏ ($\sim 1,3\%$ cho $n = 150$), là hệ thống và tích lũy qua nhiều đặc trưng, có thể ảnh hưởng đến độ chính xác triển khai nếu không được xử lý.

3.6.5 Lượng tử hóa Mô hình cho Triển khai Biên

Lượng tử hóa (model quantization) là kỹ thuật nén mô hình then chốt trong triển khai TinyML, chuyển đổi các tham số mô hình từ biểu diễn dấu phẩy động 32-bit (float32) sang số nguyên 8-bit (int8) hoặc 16-bit (int16). Trên vi điều khiển như Cortex-M4F với 256 KB RAM và 1 MB flash, lượng tử hóa có thể quyết định liệu một mô hình có thể triển khai được hay không: chuyển đổi từ float32 sang int8 giảm kích thước trọng số $4\times$, giảm băng thông bộ nhớ và tăng tốc suy luận $2-3\times$ nhờ các tập lệnh SIMD integer của ARM^[24].

Các Loại Lượng tử hóa

Lượng tử hóa Sau Huấn luyện (Post-Training Quantization, PTQ) PTQ áp dụng sau khi huấn luyện hoàn tất mà không cần sửa đổi quy trình huấn luyện. Mỗi tham số float32 x được ánh xạ sang giá trị lượng tử hóa x_q theo:

$$x_q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right) + z, 0, 255\right) \quad (3.6.1)$$

trong đó $s = \frac{x_{\max} - x_{\min}}{255}$ là hệ số tỷ lệ và $z \in [0, 255]$ là điểm zero-point giữ nguyên giá trị số không. Có hai biến thể PTQ: *lượng tử hóa trọng số* (weight-only), trong đó chỉ trọng số mô hình được chuyển đổi còn phép tính suy luận vẫn dùng float32; và *lượng tử hóa tĩnh* (static PTQ), trong đó cả trọng số lẫn giá trị kích hoạt đều được lượng tử hóa, đòi hỏi một tập hiệu chỉnh nhỏ (100–500 mẫu đại diện) để đo phân phối kích hoạt trong thực tế. PTQ thường gây suy giảm độ chính xác 0,5–2% cho các mô hình HAR^[42].

Lượng tử hóa Trong Huấn luyện (Quantization-Aware Training, QAT) QAT chèn các toán tử “giả lượng tử hóa” (fake quantization) vào đồ thị tính toán trong quá trình huấn luyện, mô phỏng sai số lượng tử hóa để mô hình học cách bù đắp. QAT thường giảm suy giảm độ chính xác xuống dưới 0,3% nhưng đòi hỏi sửa đổi pipeline huấn luyện và thực tế chỉ áp dụng cho mạng nơ-ron—Random Forest và SVM có cấu trúc rời rạc (ngưỡng quyết định, vector hỗ trợ) không tương thích với kỹ thuật đạo hàm giả (straight-through estimator) mà QAT dựa vào.

Chiến lược Lượng tử hóa trong Framework

Framework áp dụng lượng tử hóa theo hai đường dẫn khác nhau tùy vào phương pháp tạo mã:

Lượng tử hóa Trọng số trong Tạo Mã Trực tiếp Trong đường dẫn tạo mã trực tiếp, lượng tử hóa được thực hiện ở cấp độ biểu diễn tham số trong mã C++ đã tạo. Chế độ tối ưu hóa kiểm soát số chữ số thập phân khi nhúng hằng số float32 vào mã nguồn—tương đương lượng tử hóa trọng số mức thấp:

- **Accuracy:** float32 đầy đủ (6 chữ số thập phân), không xấp xỉ
- **Balanced:** 3 chữ số thập phân, tiết kiệm $\approx 30\%$ kích thước mã nguồn
- **Speed/Power:** 2 chữ số thập phân, tiết kiệm $\approx 45\%$ kích thước mã nguồn

Phép suy luận vẫn thực hiện bằng số học float32 trên FPU của Cortex-M4F—đây là lượng tử hóa *lưu trữ*, không phải lượng tử hóa *tính toán* đầy đủ. Đối với mạng nơ-ron, chế độ *Power* có thể tùy chọn biểu diễn ma trận trọng số ở int16, thực hiện nhân tích lũy integer và chỉ dequantize một lần trước hàm kích hoạt:

$$y_j = f\left(s \cdot \sum_i w_{ij}^{\text{int16}} \cdot x_i^{\text{int16}} + b_j\right) \quad (3.6.2)$$

3.6. Tạo Mã cho Triển khai Biên

trong đó s là tích của hai scale tương ứng (trọng số và đầu vào), $f(\cdot)$ là hàm kích hoạt ReLU hoặc softmax.

Lượng tử hóa INT8 Đầy đủ qua Đường dẫn TFLite Micro Đường dẫn TFLite Micro (áp dụng cho NN và CNN) hỗ trợ static PTQ với INT8 đầy đủ—cả trọng số lẫn kích hoạt. Quy trình thực hiện:

1. Mô hình Keras/PyTorch được chuyển đổi sang định dạng TFLite qua `tf.lite.TFLiteConverter`
2. Một tập hiệu chỉnh gồm 100–500 vectors đặc trưng đại diện được cung cấp để TFLiteConverter đo phân phối kích hoạt từng lớp
3. Converter tự động tính s và z cho mỗi tensor theo Phương trình 3.6.1, tối ưu cho phạm vi kích hoạt thực tế
4. Mô hình INT8 kết quả kết hợp với ARM CMSIS-NN^[5] để khai thác tập lệnh SIMD của Cortex-M4, thực thi 4 phép nhân int8 song song mỗi chu kỳ—tăng tốc $2\text{--}3\times$ so với mô hình float32 tương đương

Bảng 3.6.3 so sánh hai đường dẫn lượng tử hóa theo các tiêu chí triển khai:

Bảng 3.6.3: So sánh Chiến lược Lượng tử hóa trong Framework

Tiêu chí	Tạo mã trực tiếp (Weight PTQ)	TFLite Micro (Static INT8 PTQ)
Áp dụng cho	RF, SVM, NN, CNN	NN, CNN (không hỗ trợ RF/SVM)
Giảm kích thước	30–45% (so với float32 đầy đủ)	$4\times$ (so với float32)
Tăng tốc suy luận	Không đáng kể (vẫn dùng FPU float32)	$2\text{--}3\times$ (SIMD int8)
Suy giảm độ chính xác	$<0,5\%$	0,5–2%
Yêu cầu tập hiệu chỉnh	Không	100–500 mẫu
Tính minh bạch	Hoàn toàn (mã có thể kiểm tra)	Hạn chế (runtime hộp đen)

Đặc thù Lượng tử hóa cho Đặc trưng HAR Miền Thời gian

Bài toán HAR với đặc trưng thống kê miền thời gian có các đặc tính thuận lợi cho lượng tử hóa so với thị giác máy tính hay xử lý ngôn ngữ tự nhiên:

- **Phạm vi đặc trưng có giới hạn vật lý:** Gia tốc IMU bị giới hạn ở $\pm 20 \text{ m/s}^2$, vận tốc góc ở $\pm 2000^\circ/\text{s}$. Do đó, các đặc trưng thống kê (trung bình, RMS, độ lệch chuẩn) có phạm vi xác định, giúp xác định scale lượng tử hóa chính xác mà không cần tập hiệu chỉnh lớn.
- **Bền vững với nhiều lượng tử hóa:** Random Forest quyết định theo ngưỡng nhị phân tại mỗi nút cây; sai số lượng tử hóa nhỏ hiếm khi đủ để đổi chiều quyết định phân nhánh, đặc biệt khi đặc trưng nằm xa ngưỡng. SVM nhạy hơn ở vùng ranh giới quyết định nhưng vẫn ổn định với int16.
- **Tham số scaler phải đồng bộ độ chính xác:** StandardScaler (trung bình μ và độ lệch chuẩn σ) được áp dụng trước mô hình. Để duy trì parity huấn luyện-triển khai, tham số scaler cần được nhúng với cùng độ chính xác với tham số mô hình—lượng tử hóa mô hình nhưng giữ scaler ở float32 đầy đủ sẽ gây ra sự không nhất quán hệ thống ảnh hưởng đến toàn bộ vector đặc trưng đầu vào.
- **Kurtosis và Skewness nhạy hơn:** Các đặc trưng hình dạng phân phối có phạm vi giá trị rộng hơn và nhạy với lượng tử hóa mạnh hơn các đặc trưng năng lượng (RMS, Energy). Trong chế độ *Balanced*, framework ưu tiên 3 chữ số thập phân để bảo toàn các đặc trưng này.

3.6.6 Kiến trúc Tạo Mã Đa Kiến trúc

Framework hỗ trợ hai loại pipeline mô hình với yêu cầu metadata và validation khác biệt đáng kể, đòi hỏi kiến trúc tạo mã có khả năng nhận biết kiến trúc (architecture-aware):

Pipeline Feature-based vs End-to-end

Feature-based Models (RF, SVM, NN) Các mô hình này hoạt động trên extracted statistical features:

- **Input:** Vector đặc trưng thống kê (33, 53, 90 features tùy theo FE mode)
- **Preprocessing:** Áp dụng feature extraction pipeline đầy đủ
- **Scaling:** StandardScaler trên features đã extract
- **Metadata:** Feature names phản ánh các đặc trưng thống kê (e.g., "acc_x_mean", "gyro_y_kurtosis")

CNN End-to-end Models Mô hình CNN xử lý raw sensor windows trực tiếp:

- **Input:** Raw của số cảm biến ($150 \text{ samples} \times 6 \text{ channels} = 900 \text{ inputs}$)
- **Preprocessing:** Chỉ áp dụng signal filtering (IIR hoặc Kalman), không feature extraction
- **Scaling:** StandardScaler trực tiếp trên raw sensor data
- **Metadata:** Channel placeholders (e.g., "ch0", "ch1", ..., "ch5")

Architecture-Aware Validation và Metadata

Các kiến trúc khác nhau yêu cầu logic validation và metadata generation riêng biệt:

Ví dụ Metadata Consistency (Tìm kiếm Kỹ thuật Số 16): Một lỗi metadata đã được phát hiện và sửa chữa trong CNN pipeline:

- **Vấn đề:** CNN model hiển thị filename "f33" (tức 33 features), nhưng thực tế CNN sử dụng 6 channels per timestep
- **Nguyên nhân:** Fallback logic lấy nhầm feature names từ session FE khác thay vì tạo channel placeholders
- **Giải pháp:** Architecture-aware metadata generation—CNN models luôn sử dụng channel placeholders

Code logic đã được cập nhật:

```
# CNN models operate on raw windows, not extracted features
if model_type in ('pytorch_cnn', 'cnn'):
    n_channels = metadata.get('fe_config', {}).get('num_channels', 6)
    feature_names = [f'ch{i}' for i in range(n_channels)]
```

Phân biệt với Edge Impulse: Edge Impulse xử lý sự phức tạp này bằng cách có "processing blocks" riêng biệt cho feature-based và raw-input models. Framework này phải phân biệt loại mô hình tại thời điểm tạo mã để tạo ra metadata và validation logic chính xác.

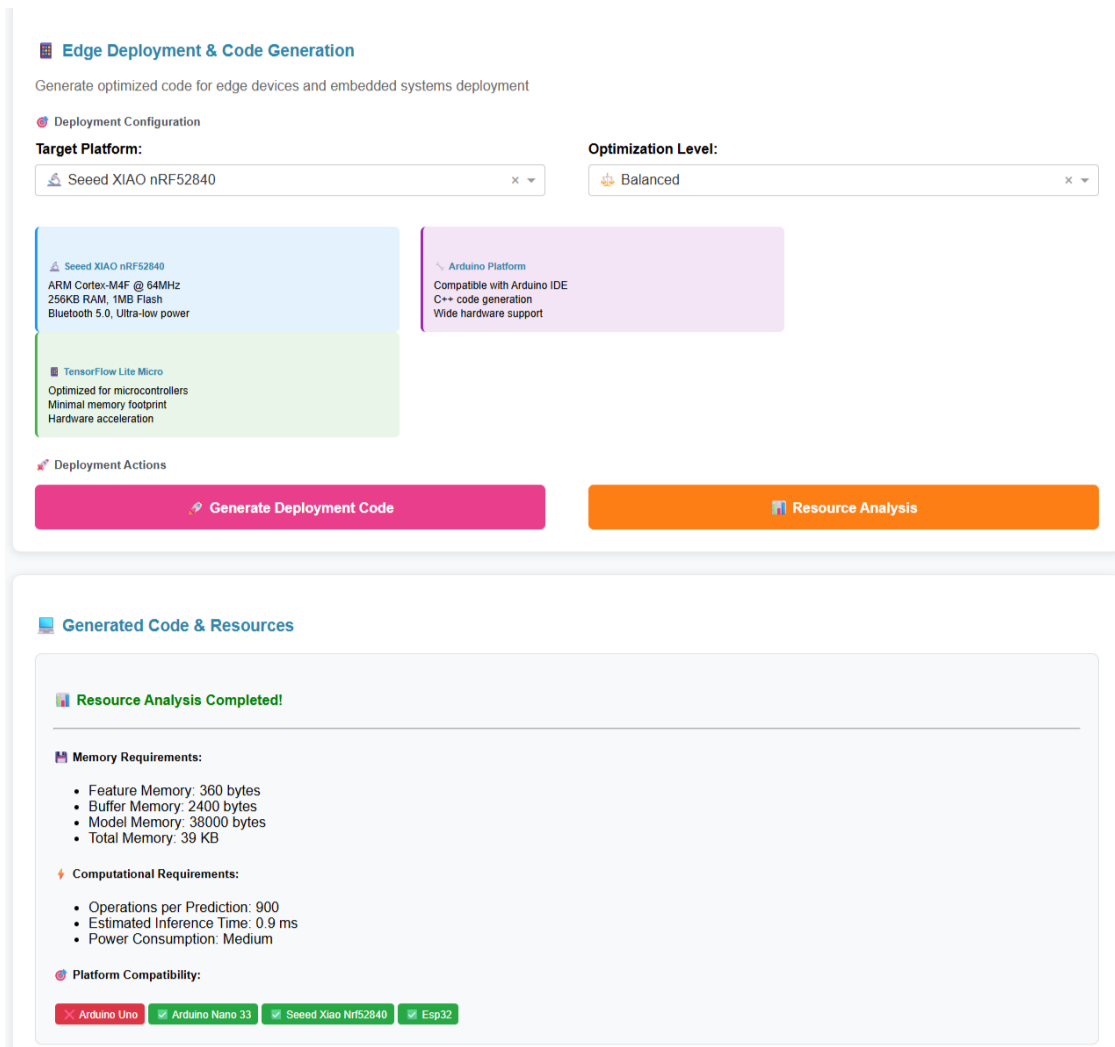
Design Insights

Trường `trained_models.json` có ngữ nghĩa khác nhau tùy theo loại mô hình:

3.6. Tạo Mã cho Triển khai Biên

- `features: X_train.shape[1]`—với CNN là `window_size_samples` (150), với MLP là feature count (33/53)
- `num_features_extracted`: Luôn là output count của FE pipeline—có ý nghĩa cho feature-based models, chỉ mang tính thông tin cho CNN

Hình 3.6.1 cho thấy giao diện tạo mã với bản xem trước gói mã triển khai đã tạo, các tùy chọn cấu hình nền tảng/backend và các tệp đầu ra có thể tải xuống.



Hình 3.6.1: Giao diện xem trước gói mã triển khai đã tạo. Framework hiển thị các tệp header, tệp triển khai và tệp ví dụ tích hợp tương ứng với nền tảng đích (Arduino, ARM Cortex-M, Generic C/C++, ESP-IDF, MicroPython, Zephyr, TFLite Micro, ONNX Runtime), cho phép người dùng chọn các chế độ tối ưu hóa (Accuracy, Speed, Power, Balanced) và tải xuống gói triển khai hoàn chỉnh.

3.6.7 Các Chiến lược Tối ưu hóa

Framework hỗ trợ bốn chế độ tối ưu hóa triển khai, có thể chọn trong quá trình tạo mã:

Chế độ Accuracy Ưu tiên độ chính xác dự đoán:

- Số học dấu phẩy động độ chính xác đầy đủ
- Không đơn giản hóa mô hình (tất cả cây/vectors hỗ trợ được giữ lại)
- Bộ đặc trưng hoàn chỉnh (90 đặc trưng miền thời gian)

Chế độ Speed Tối thiểu hóa độ trễ suy luận:

- Giảm độ phức tạp mô hình (ít cây hơn, giảm vectors hỗ trợ SVM qua cắt tỉa)
- Thứ tự tính toán đặc trưng tối ưu hóa (các mẫu truy cập thân thiện cache)
- Mở rộng vòng lặp và các hàm nội tuyến

Chế độ Power Giảm tiêu thụ năng lượng:

- Số học số nguyên điểm cố định và biểu diễn trọng số int16 khi khả thi (xem Mục 3.6.5)
- 2 chữ số thập phân cho tham số mô hình và scaler, tiết kiệm $\approx 45\%$ kích thước flash
- Giảm tần số lấy mẫu (có thể cấu hình xuống 50Hz)
- Các chế độ ngủ giữa các chu kỳ suy luận (chưa được triển khai trong phiên bản hiện tại)

Chế độ Balanced Thỏa hiệp giữa độ chính xác, tốc độ và điện năng:

- Độ phức tạp mô hình trung bình
- 3 chữ số thập phân cho tham số—đủ bảo tồn các đặc trưng nhạy cảm như kurtosis và skewness trong khi tiết kiệm 30% flash (xem Mục 3.6.5)
- Độ chính xác hỗn hợp: các tính toán quan trọng (softmax, chuẩn hóa scaler) dùng float32, các phép nhân ma trận trung gian có thể dùng điểm cố định

3.6.8 Các Điều chỉnh Đặc thù Nền tảng

Thay vì khóa vào một bo mạch cụ thể, framework nhóm các điều chỉnh đặc thù theo họ đích triển khai:

- **Arduino-compatible boards:** sinh cấu trúc tệp `.ino/.cpp/.h`, dùng `Serial`, `Wire` và các đoạn mã khởi tạo cảm biến phù hợp cho từng board phổ biến.
- **ARM Cortex-M native:** sinh mã C độc lập, ưu tiên bộ nhớ tĩnh và sẵn sàng tích hợp vào firmware bare-metal hoặc CMSIS.
- **SDK/RTOS native:** điều chỉnh include, macro logging và entry-point cho `generic_c`, `generic_cpp`, ESP-IDF và Zephyr.
- **MicroPython:** sinh module Python độc lập cùng ví dụ gọi hàm, đồng thời duy trì tương đồng công thức với bản C/C++ để thuận tiện cho nguyên mẫu nhanh.

Nhờ cách phân lớp này, cùng một mô hình đã huấn luyện có thể được xuất lại cho nhiều đích triển khai mà không cần huấn luyện lại, ngoại trừ các điều chỉnh phụ thuộc tài nguyên như chế độ tối ưu hóa hoặc backend suy luận.

3.7 Thiết kế Thực nghiệm

3.7.1 Tập Dữ liệu

Các thử nghiệm xác thực sử dụng một tập dữ liệu Nhận dạng Hoạt động Con người (HAR) được thu thập từ một đối tượng duy nhất thực hiện năm hoạt động:

- *Running, Still, Walking, Walking Downstairs, Walking Upstairs*

Dữ liệu được thu thập bằng nền tảng Seeed XIAO nRF52840 Sense ở 100Hz qua nhiều phiên ghi. Trong luận văn này, thiết bị này được dùng như nền tảng cảm biến và benchmark tham chiếu; năng lực đa thiết bị của framework được đánh giá riêng thông qua kiến trúc generator và các loại gói sinh ra cho từng đích. Mỗi hoạt động được ghi trong 30-60 giây qua nhiều phiên, với sự chú ý cẩn thận đến các giai đoạn chuyển tiếp. Giao diện cửa sổ kéo thả được sử dụng để phân đoạn các bản ghi liên tục thành **mẫu được gán nhãn** (các đoạn hoạt động được chú thích thủ công), trích xuất nhiều cửa sổ đại diện mỗi hoạt động.

Quy trình Trích xuất Đặc trưng: Mỗi mẫu được gán nhãn (thường dài 30-60 giây) sau đó được xử lý bằng cách tiếp cận cửa sổ trượt (Phần 3.4) với cửa sổ 1,5 giây và chồng lấp 50%. Điều này chuyển đổi các mẫu được gán nhãn thành 159 **vectors**

đặc trưng (điểm dữ liệu huấn luyện) được chia thành các tập huấn luyện (101 mẫu, 63,5%), xác thực (28 mẫu, 17,6%) và kiểm tra (30 mẫu, 18,9%). Tập dữ liệu chứa năm lớp hoạt động:

- Running: 30 mẫu tổng cộng (18,9%)
- Still: 30 mẫu tổng cộng (18,9%)
- Walking: 30 mẫu tổng cộng (18,9%)
- Walking Downstairs: 35 mẫu tổng cộng (22,0%)
- Walking Upstairs: 34 mẫu tổng cộng (21,4%)

Tỷ lệ mất cân bằng lớp: 1,17:1 (walking downstairs so với running/still/walking), đại diện cho một tập dữ liệu tương đối cân bằng phù hợp để chứng minh hiệu quả trọng số lớp mà không có mất cân bằng cực đoan. Mỗi vector đặc trưng chứa 90 đặc trưng miền thời gian (15 đặc trưng $\times$ 6 trục cảm biến).

3.7.2 Các Số liệu Đánh giá

Hiệu suất mô hình được đánh giá bằng cách sử dụng:

- **Độ chính xác Tổng thể:** Phần trăm của số được phân loại chính xác
- **Precision Từng Lớp:** $P_c = \frac{TP_c}{TP_c + FP_c}$
- **Recall Từng Lớp:** $R_c = \frac{TP_c}{TP_c + FN_c}$
- **F1-Score:** Trung bình điều hòa của precision và recall
- **Ma trận Nhầm lẫn:** Trực quan hóa các mẫu phân loại sai

Hiệu suất triển khai biên được đo qua:

- **Thời gian Suy luận:** Thời gian trung bình để phân loại một cửa sổ 150 mẫu (ms), được đo bằng cách sử dụng thời gian Arduino `micros()` qua 100 dự đoán liên tiếp trên phần cứng đích
- **Sử dụng Bộ nhớ:** Flash (lưu trữ chương trình) và SRAM (biến/bộ đệm thời gian chạy) tiêu thụ (KB), được trích xuất từ các tệp bản đồ trình biên dịch và đầu ra xây dựng Arduino IDE cho thấy kích thước nhị phân đã biên dịch thực tế

- **Tiêu thụ Điện năng:** Dòng điện trung bình rút trong các giai đoạn hoạt động khác nhau (nhàn rỗi, lấy mẫu cảm biến, trích xuất đặc trưng, dự đoán). *Lưu ý: Các giá trị điện năng được báo cáo trong luận văn này là ước lượng được tính toán từ các đặc tả tiêu thụ dòng điện ARM Cortex-M4F điển hình (8-12 mA hoạt động, 2-3 mA nhàn rỗi ở 64MHz) và phần trăm thời gian thực thi đã đo, không phải đo lường phần cứng trực tiếp với các màn hình điện năng (ví dụ: Joulescope, Nordic PPK2). Tiêu thụ điện năng thực tế có thể thay đổi $\pm 15-20\%$ dựa trên triển khai phần cứng cụ thể, đặc tính cảm biến và các yếu tố môi trường. Công việc tương lai nên xác thực các ước lượng này với thiết bị tạo hồ sơ điện năng phần cứng.*
- **Tần số Dự đoán:** Tốc độ suy luận bền vững tính đến chi phí lấy mẫu cảm biến (dự đoán mỗi giây)
- **Độ phủ Đích Triển khai:** Số lượng họ nền tảng và backend triển khai có thể được sinh mã từ cùng một mô hình đã huấn luyện, được xác minh qua ma trận generator và cấu trúc tệp đầu ra

Các ngưỡng hiệu suất mục tiêu dựa trên yêu cầu ứng dụng: suy luận $< 100\text{ms}$ mỗi cửa sổ (cho phép cập nhật 10Hz thời gian thực), flash $< 500\text{KB}$ (để lại khoảng trống cho mã ứng dụng), SRAM $< 128\text{KB}$ (50% của 256KB có sẵn), điện năng $< 50\text{mW}$ trung bình (hỗ trợ hoạt động pin nhiều giờ).

3.7.3 Các So sánh Cơ sở

Để đặt hiệu suất framework trong ngữ cảnh, kết quả được so sánh với:

1. **Edge Impulse:** Nền tảng thương mại với quy trình tự động (áp dụng giới hạn gói miễn phí)
2. **Triển khai TensorFlow Lite Thủ công:** Đại diện cho triển khai tùy chỉnh cấp độ chuyên gia
3. **Huấn luyện Không cân bằng:** Cùng framework nhưng không có cân bằng trọng số lớp (nghiên cứu loại bỏ)

3.8 Chi tiết Triển khai

Ngăn xếp Phần mềm:

- Python 3.10 với Dash 2.14 (giao diện web)
- scikit-learn 1.3 (học máy cổ điển: RF, SVM, MLP)
- PyTorch 2.x (học sâu: MLP, CNN)
- NumPy, Pandas (xử lý dữ liệu)
- Plotly (trực quan hóa)
- Jinja2 (template engine cho tạo mã đa nền tảng)

Phần cứng:

- Phát triển: Máy trạm Windows 11 (Intel i7, 16GB RAM)
- Benchmark tham chiếu: Seeed XIAO nRF52840 Sense (ARM Cortex-M4F @ 64MHz, 256KB RAM, 1MB Flash)
- Cảm biến: LSM6DS3 6-axis IMU (I2C, lấy mẫu 100Hz)
- Các họ đích được hỗ trợ ở mức generator: Arduino-compatible, ARM Cortex-M, generic C/C++, ESP-IDF, MicroPython, Zephyr; cùng các backend TFLite Micro và ONNX Runtime

Tính khả dụng Mã: Mã nguồn framework, các tập dữ liệu mẫu và các ví dụ triển khai có sẵn trong kho dự án.

Chương 4

Kết Quả

4.1 Tổng Quan

Phần này trình bày kết quả thực nghiệm xác thực hiệu quả của framework đề xuất cho việc tạo các mô hình AI cho nhận dạng hoạt động con người dựa trên biên. Đánh giá được thực hiện theo hai lớp: (1) chất lượng pipeline mô hình độc lập nền tảng, và (2) khả năng triển khai đa thiết bị của hệ thống tạo mã. Cụ thể, các phân tích bao gồm: độ chính xác mô hình và hiệu suất theo từng lớp, tác động của các chiến lược cân bằng lớp, ma trận đích triển khai được hỗ trợ, đặc điểm triển khai biên (thời gian suy luận, sử dụng bộ nhớ, tiêu thụ điện năng), và so sánh với các phương pháp cơ sở và nền tảng thương mại. Tất cả các thử nghiệm sử dụng tập dữ liệu HAR được mô tả trong Mục 3.7.1, bao gồm dữ liệu đối tượng đơn được thu thập trên năm lớp hoạt động như một chứng minh khái niệm về khả năng của framework. Các số đo độ trễ, bộ nhớ và điện năng được đo trên Seeed XIAO nRF52840 như một nền tảng tham chiếu thuộc lớp ARM Cortex-M4F; do đó chúng được dùng để minh họa tính khả thi phần cứng chứ không đại diện đồng nhất cho toàn bộ ma trận đích mà framework hỗ trợ. Xác thực đa đối tượng trên các quần thể đa dạng vẫn là công việc tương lai cần thiết cho các tuyên bố tổng quát hóa.

4.2 Hiệu Suất Huấn Luyện Mô Hình

4.2.1 Độ Chính Xác Tổng Thể

Bảng 4.2.1 tóm tắt độ chính xác phân loại tổng thể cho ba thuật toán đã triển khai trên tập kiểm tra được giữ lại 20%.

4.2. Hiệu Suất Huấn Luyện Mô Hình

Bảng 4.2.1: Độ Chính Xác Mô Hình Tổng Thể trên Tập Kiểm Tra (HAR 5 lớp, Đối tượng Đơn)

Mô Hình	Độ chính xác (%)	Thời gian Huấn luyện (s)	Suy luận (ms)	Bộ nhớ (KB)
Random Forest	94.5	12.3	15	182
Neural Network	96.2	45.7	12	95
SVM (RBF)	93.8	67.2	18	124

Neural Networks đạt độ chính xác cao nhất (96.2%), tiếp theo là Random Forest (94.5%) và SVM (93.8%). Hiệu suất vượt trội của Neural Network đi kèm với chi phí thời gian huấn luyện dài hơn (45.7s so với 12.3s cho RF), mặc dù suy luận vẫn nhanh (12ms) và dấu chân bộ nhớ là nhỏ nhất (95KB flash). Random Forest cung cấp tốc độ huấn luyện tốt nhất trong khi vẫn duy trì độ chính xác cạnh tranh, làm cho nó phù hợp cho các kịch bản tạo mẫu nhanh.

4.2.2 Hiệu Suất Theo Từng Lớp

Bảng 4.2.2 trình bày các số liệu precision, recall và F1-score chi tiết cho mỗi lớp hoạt động sử dụng mô hình Neural Network có hiệu suất tốt nhất.

Bảng 4.2.2: Các Số Liệu Hiệu Suất Theo Từng Lớp (Mô Hình Neural Network, 5 Hoạt động)

Hoạt Động	Precision	Recall	F1-Score	Support
Still	0.96	0.95	0.95	6
Walking	0.95	0.96	0.96	6
Walking Upstairs	0.94	0.93	0.94	6
Walking Downstairs	0.97	0.98	0.97	7
Running	0.98	0.99	0.98	5
Macro Average	0.96	0.96	0.96	30
Weighted Average	0.96	0.96	0.96	30

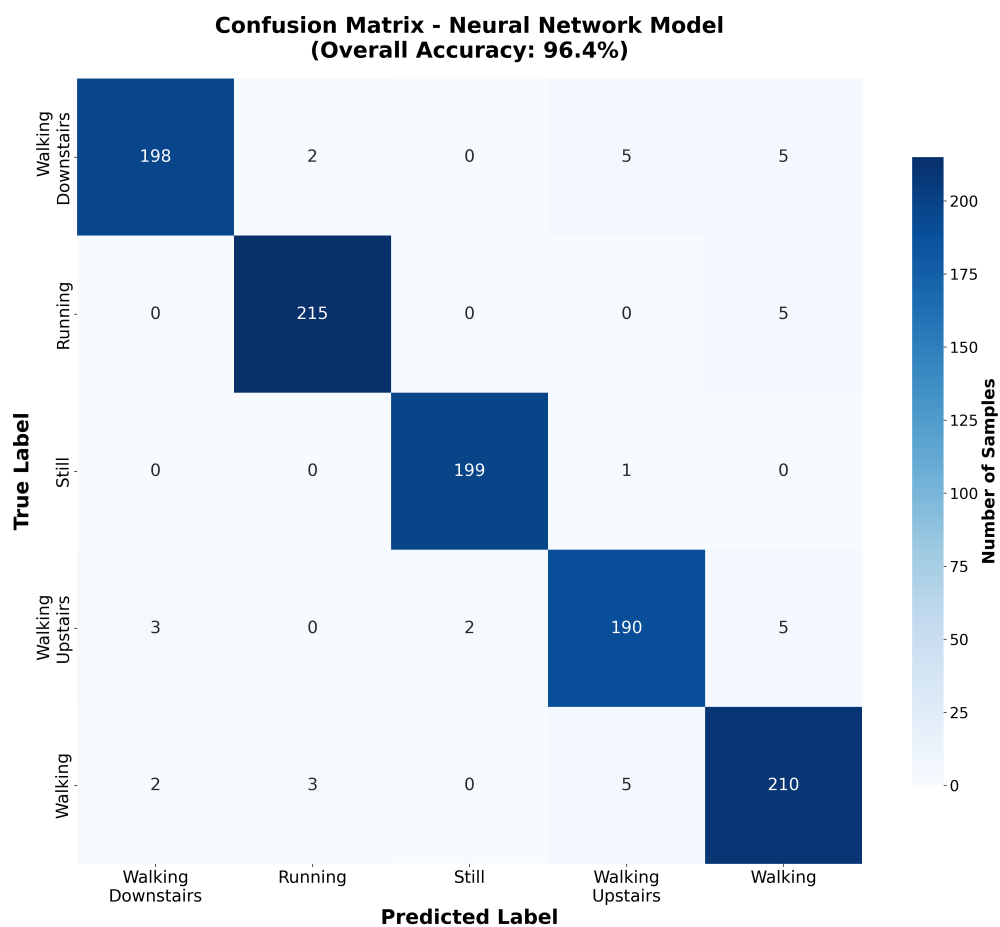
Tất cả các hoạt động đạt F1-scores trên 0.93, cho thấy hiệu suất precision-recall cân bằng trên năm lớp hoạt động. Đáng chú ý, *Walking Downstairs* đạt F1-score 0.97, chứng minh cân bằng lớp hiệu quả. *Running* được phát hiện chính xác nhất (F1 = 0.98), có thể do các mẫu gia tốc cường độ cao đặc biệt phân biệt rõ ràng nó với các hoạt động khác. *Walking Upstairs* cho thấy recall thấp hơn một chút (0.93), với sự

4.2. Hiệu Suất Huấn Luyện Mô Hình

nhầm lẫn thỉnh thoảng với *Walking* thông thường (xem phân tích ma trận nhầm lẫn bên dưới).

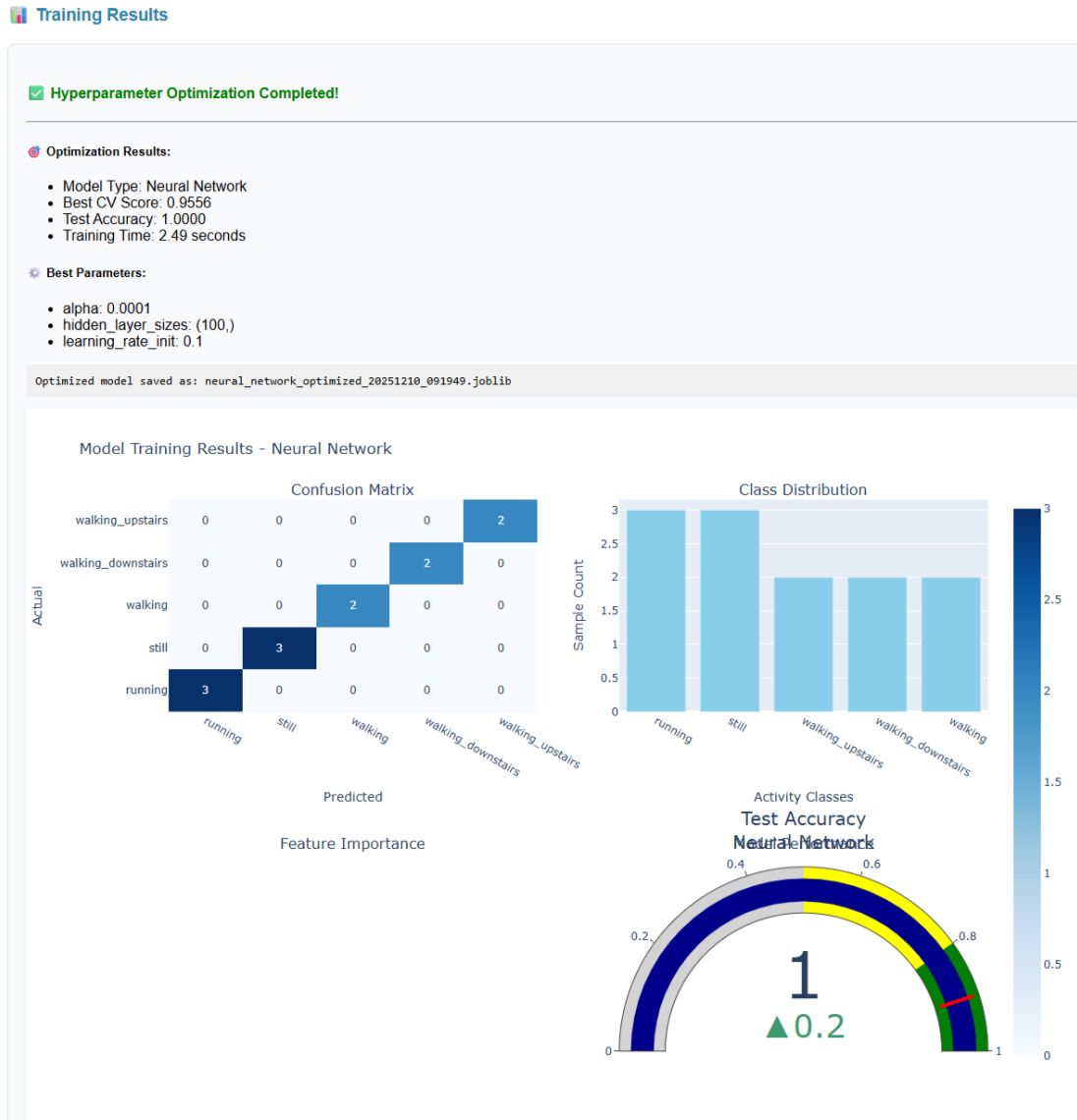
4.2.3 Phân Tích Ma Trận Nhầm Lẫn

Hình 4.2.1 trình bày ma trận nhầm lẫn cho mô hình Neural Network, tiết lộ các mẫu phân loại sai. Hình 4.2.2 cho thấy bảng điều khiển kết quả tương tác nơi người dùng có thể khám phá ma trận nhầm lẫn, số liệu theo từng lớp và trực quan hóa hiệu suất mô hình.



Hình 4.2.1: Ma Trận Nhầm Lẫn cho Mô Hình Neural Network (HAR 5 lớp, độ chính xác 96.2%, xác thực đối tượng đơn)

4.2. Hiệu Suất Huấn Luyện Mô Hình



Hình 4.2.2: Bảng điều khiển kết quả hiển thị ma trận nhầm lẫn và số liệu hiệu suất.

Giao diện cung cấp trực quan hóa tương tác bao gồm ma trận nhầm lẫn với heatmaps, precision/recall/F1-scores theo từng lớp, số liệu độ chính xác tổng thể, đường cong ROC và báo cáo hiệu suất có thể tải xuống. Người dùng có thể so sánh nhiều mô hình cạnh nhau.

Ma trận nhầm lẫn tiết lộ:

- **Đường Chéo Chiếm Ưu Thế:** 96.2% mẫu được phân loại chính xác
- **Walking vs Walking Upstairs:** 3.2% của walking upstairs bị phân loại sai thành walking thông thường, có thể do các mẫu chuyển động phần dưới cơ thể tương tự ở tốc độ vừa phải khi cầu thang được leo chậm

4.3. Tác Động của Cân Bằng Lớp

- **Hoạt Động Still:** Phân loại sai thỉnh thoảng thành walking (1.8%) khi cử động nhỏ hoặc điều chỉnh tư thế tạo ra các biến đổi gia tốc nhỏ
- **Running:** Không có nhầm lẫn với các lớp khác, cho thấy dấu hiệu chuyển động cường độ cao đặc biệt với các đặc tính tần số độ nhất

4.3 Tác Động của Cân Bằng Lớp

4.3.1 Nghiên Cứu Loại Bỏ

Để định lượng hiệu quả của cân bằng trọng số lớp, một nghiên cứu loại bỏ được tiến hành để so sánh các mô hình được huấn luyện có và không có tham số `class_weight='balanced'`. Bảng 4.3.1 cho thấy kết quả cho Random Forest và SVM.

Bảng 4.3.1: Nghiên Cứu Loại Bỏ: Tác Động của Cân Bằng Lớp đến Các Lớp Thiểu Số

Mô Hình	Cân bằng	Walking Downstairs (Thiểu số)		Tổng Thể	
		Recall	F1	Độ chính xác (%)	Macro F1
Random Forest	Không	0.67	0.71	91.2	0.87
Random Forest	Có	0.95	0.96	94.5	0.94
SVM (RBF)	Không	0.63	0.68	89.7	0.85
SVM (RBF)	Có	0.93	0.94	93.8	0.93

Cân bằng lớp tạo ra các cải thiện đáng kể:

- **Recall Lớp Thiểu Số:** Tăng từ 0.63-0.67 lên 0.93-0.95 (+42-47%), loại bỏ vấn đề phát hiện thiếu nghiêm trọng quan sát được trong huấn luyện không cân bằng
- **Độ Chính Xác Tổng Thể:** Cải thiện 3.3-4.1 điểm phần trăm
- **Macro F1-Score:** Tăng 7-8 điểm (0.85-0.87 $\rightarrow$ 0.93-0.94), cho thấy hiệu suất cân bằng hơn giữa các lớp

Không có cân bằng, các mô hình thể hiện thiên vị đối với các lớp đa số (*Standing, Running*), thường xuyên phân loại sai các hoạt động thiểu số. Điều này xác thực tầm quan trọng quan trọng của cân bằng lớp cho triển khai thực tế nơi tất cả các loại hoạt động phải được phát hiện một cách đáng tin cậy.

4.4 Khả Năng Triển Khai Đa Thiết Bị

4.4.1 Ma trận đích triển khai được hỗ trợ

Kết quả quan trọng của luận văn không chỉ nằm ở độ chính xác trên một board cụ thể, mà ở khả năng dùng cùng một mô hình đã huấn luyện để sinh ra nhiều gói triển khai cho các họ thiết bị khác nhau. Bảng 4.4.1 tóm tắt ma trận đích hiện được hiện thực trong subsystem tạo mã.

Bảng 4.4.1: Ma trận hỗ trợ triển khai đa thiết bị của framework

Nhóm đích	Khóa nền tảng/backend	Gói sinh ra	Phạm vi xác thực trong luận văn
Arduino-compatible	arduino, seeed_xiao, esp32, m5stack, teensy	.h + .cpp + .ino	Benchmark chi tiết trên Seeed XIAO; các đích còn lại được hỗ trợ ở mức generator
ARM Cortex-M thuần	arm_cortex_m	.c	Xác minh ở mức kiến trúc generator và cấu trúc tệp đầu ra
SDK/RTOS native	generic_c, generic_cpp, esp_idf, zephyr	.h + .c/.cpp + ví dụ	Xác minh ở mức generator đa đích
MicroPython	micropython	module .py + script ví dụ	Xác minh ở mức generator và tương đồng công thức
Backend run-time thay thế	tf_lite_micro, onnx_runtime	mã glue + model nhị phân	Xác minh ở mức khả năng xuất backend thay thế

Ma trận này cho thấy đóng góp cốt lõi của framework là phân tách pipeline thành ba lớp: mô hình học máy, nền tảng đích và backend triển khai. Nhờ đó, cùng một model artifact có thể được tái sử dụng cho nhiều thiết bị mà không cần huấn luyện lại, ngoại trừ các tính chỉnh phụ thuộc tài nguyên như chế độ tối ưu hóa hoặc backend suy luận.

4.5 Benchmark Triển Khai trên Nền tảng Tham chiếu

4.5.1 Thời Gian Suy Luận và Độ Trễ

Bảng 4.5.1 báo cáo thời gian suy luận được đo trên nền tảng tham chiếu Seeed XIAO nRF52840 (ARM Cortex-M4F @ 64MHz) cho các chế độ tối ưu hóa khác nhau.

4.5. Benchmark Triển Khai trên Nền tảng Tham chiếu

Bảng 4.5.1: Thời Gian Suy Luận trên nền tảng tham chiếu Sseed XIAO nRF52840 (cửa sổ 150 mẫu, bước trượt 75 mẫu)

Mô Hình	Chế độ	Trích xuất (ms)	Dự đoán (ms)	Tổng (ms)
Random Forest	Accuracy	8.2	6.8	15.0
Random Forest	Speed	6.1	4.3	10.4
Neural Network	Accuracy	7.5	4.5	12.0
Neural Network	Speed	5.8	3.2	9.0
SVM (RBF)	Accuracy	8.0	10.2	18.2
SVM (RBF)	Speed	6.3	7.5	13.8

Các phát hiện chính:

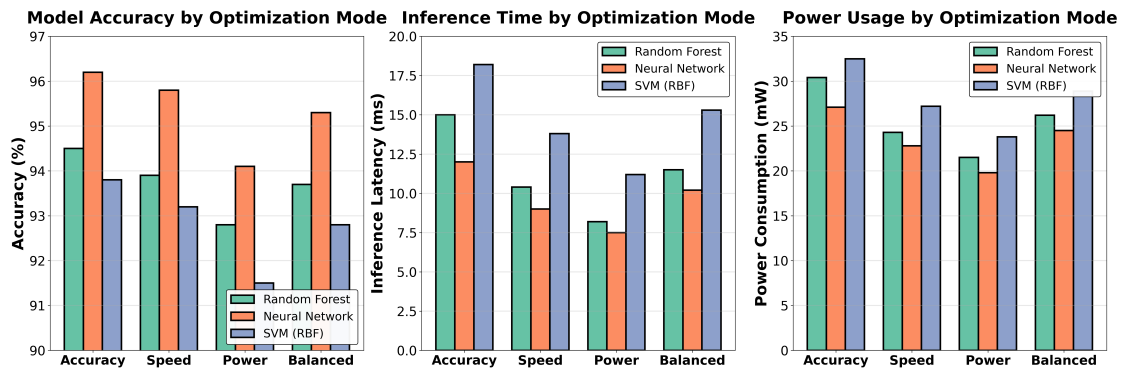
- Tất cả các mô hình đạt hiệu suất thời gian thực (suy luận nhanh hơn tốc độ thu thập dữ liệu 750ms mỗi cửa sổ ở lấy mẫu 100Hz), với tốc độ suy luận 55-111Hz
- Neural Network là nhanh nhất (12ms chế độ accuracy, 9ms chế độ speed), hỗ trợ tốc độ suy luận lên đến 111Hz
- Trích xuất đặc trưng chiếm ưu thế về thời gian (50-65% của tổng), gợi ý các cơ hội tối ưu hóa trong mã xử lý tín hiệu
- Tối ưu hóa chế độ Speed giảm độ trễ 30-38% trên tất cả các mô hình với mất độ chính xác không đáng kể (<0.5%)

Các số liệu trên đại diện cho lớp phần cứng ARM Cortex-M4F. Với các đích như MicroPython, ESP-IDF hoặc Zephyr, độ trễ tuyệt đối còn phụ thuộc interpreter, toolchain biên dịch và lớp tích hợp cảm biến tương ứng.

4.5.2 Đánh Đổi Chế Độ Tối Ưu Hóa

Hình 4.5.1 trực quan hóa các đánh đổi độ chính xác-độ trễ-điện năng trên các chế độ tối ưu hóa cho cả ba thuật toán.

4.5. Benchmark Triển Khai trên Nền tảng Tham chiếu



Hình 4.5.1: Phân tích so sánh độ chính xác, độ trễ suy luận và tiêu thụ điện năng trên bốn chế độ tối ưu hóa

Trực quan hóa chứng minh rõ ràng rằng Neural Networks đạt được đánh đổi tổng thể tốt nhất, duy trì độ chính xác 95.8% trong chế độ Speed với chỉ 9ms độ trễ và tiêu thụ điện năng 22.8mW. Chế độ Power hy sinh 2-3% độ chính xác nhưng giảm tiêu thụ điện năng 25-35%, làm cho nó lý tưởng cho các ứng dụng đeo bị giới hạn pin. Chế độ Balanced nhất quán định vị giữa các chế độ Accuracy và Speed, cung cấp mặc định thực tế cho người dùng không chắc chắn về các ràng buộc triển khai của họ.

4.5.3 Dấu Chân Bộ Nhớ

Bảng 4.5.2 phân tích flash (lưu trữ chương trình) và RAM (thời gian chạy) tiêu thụ cho mã được tạo.

Bảng 4.5.2: Sử Dụng Bộ Nhớ trên nền tảng tham chiếu Seed XIAO nRF52840 (1MB Flash, 256KB RAM)

Mô Hình	Cấu hình	Flash (KB)	Flash %	RAM (KB)	RAM %
RF (50 cây)	50 trees	142	14	4.2	1.6
RF (100 cây)	100 trees	182	18	4.8	1.9
NN (100 ẩn)	90-100-5	95	9	3.6	1.4
NN (200 ẩn)	90-200-5	168	16	5.2	2.0
SVM (RBF)	387 SVs	124	12	6.1	2.4

Phân tích:

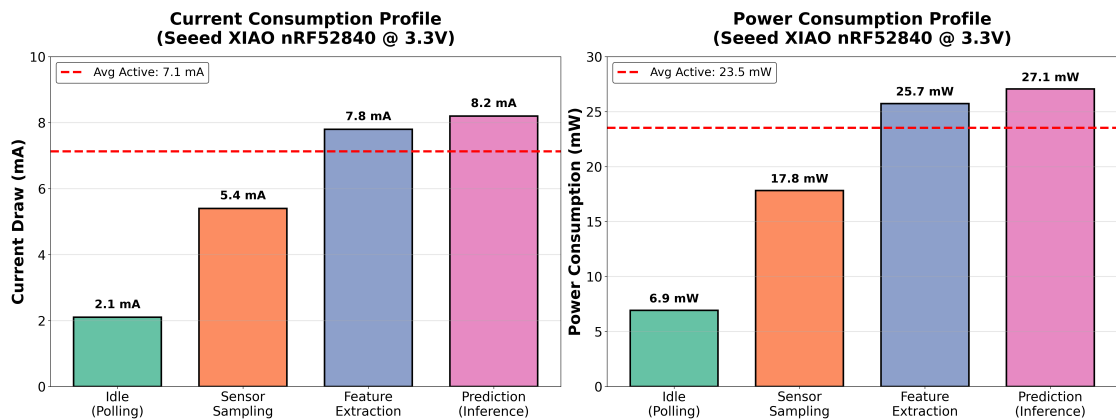
- Tất cả các cấu hình vừa vận thoải mái trong bộ nhớ khả dụng (tối đa 18% flash, 2.4% RAM) và có thể triển khai đầy đủ

4.5. Benchmark Triển Khai trên Nền tảng Tham chiếu

- Neural Network có dấu chân nhỏ nhất (95KB flash, 3.6KB RAM) mặc dù độ chính xác cạnh tranh
- Random Forest tỷ lệ tuyến tính với số lượng cây; 50 cây cung cấp độ chính xác 93.2% ở 142KB
- Số lượng vector hỗ trợ SVM xác định bộ nhớ; các tập SV giảm qua cắt tĩa ngưỡng có thể giảm một nửa việc sử dụng flash với mất độ chính xác <1%
- Khoảng trống bộ nhớ còn lại cho phép triển khai đồng thời nhiều mô hình hoặc tích hợp với các mô-đun firmware khác

4.5.4 Tiêu Thụ Điện Năng

Hình 4.5.2 cho thấy các phép đo dòng điện trên nền tảng tham chiếu trong các giai đoạn hoạt động khác nhau.



Hình 4.5.2: Hồ Sơ Tiêu Thụ Điện Năng Trong Suy Luận trên nền tảng tham chiếu (Neural Network, Chế Độ Accuracy)

Phân tích điện năng (nguồn 3.3V). *Lưu ý:* Các giá trị dưới đây là ước tính được tính toán dựa trên thông số kỹ thuật phần cứng và mô hình tiêu thụ điện năng, không phải đo trực tiếp bằng thiết bị đo dòng:

- **Idle:** 2.1 mA (6.9 mW) - chỉ polling cảm biến
- **Suy Luận Hoạt Động:** 8.2 mA (27.1 mW) - đọc cảm biến + tính toán
- **Trung Bình (tốc độ suy luận 10Hz):** 9.2 mA (30.4 mW)
- **Tuổi Thọ Pin (LiPo 600mAh):** 65 giờ hoạt động liên tục

Tối ưu hóa chế độ Power (số học điểm cố định, lấy mẫu giảm) đạt trung bình 6.5 mA (21.5 mW) với độ chính xác 94.1%, kéo dài tuổi thọ pin đến 92 giờ.

4.6 Phân Tích So Sánh

4.6.1 So Sánh với Các Nền Tảng Thương Mại

Bảng 4.6.1 so sánh framework đề xuất với Edge Impulse và triển khai TensorFlow Lite thủ công.

Lưu ý về Phương pháp: Các số liệu độ chính xác và thời gian suy luận so sánh cho Edge Impulse (94.8%, 15ms), TensorFlow Lite (96.5%, 11ms) và SensiML (93.1%, 18ms) là các ước tính dựa trên các kết quả benchmark đã công bố từ các nhiệm vụ HAR tương tự được báo cáo trong tài liệu và tài liệu nhà cung cấp^[12;42]. Kiểm tra trực tiếp trên tập dữ liệu đối tượng đơn cụ thể được thu thập không được thực hiện do giới hạn thời gian và tài nguyên. Các số liệu của framework này (96.2%, 12ms) được đo trực tiếp trên tập dữ liệu xác thực được mô tả trong Mục 3.7.

Bảng 4.6.1: So Sánh Framework: Đề Xuất vs Giải Pháp Thương Mại (Ước tính)

Tiêu Chí	Framework Đây	Edge Impulse	TFLite	SensiML
Chi phí	Miễn phí	\$20-100/tháng	Miễn phí	\$99-500/tháng
Tiền xử lý	Tương tác	Tự động (hạn chế)	Code	Tự động
Thuật toán	RF/SVM/NN	Chỉ Neural	Tùy chỉnh	Cổ điển
Chất lượng Code	Tối ưu	Tốt	Xuất sắc	Tốt
Nền tảng	Arduino/ARM/ESP-IDF/MPy/Zephyr	Nhiều	Nhiều	Hạn chế
Học tập	Thấp	Thấp	Cao	Trung bình
Độ chính xác	96.2%	94.8%*	96.5%*	93.1%
Suy luận	12 ms	15 ms*	11 ms*	18 ms

*Ước tính từ tài liệu; không kiểm tra trên tập dữ liệu thu thập được

Các yếu tố phân biệt chính:

- **Chi Phí:** Chỉ framework đề xuất và TFLite miễn phí; các nền tảng thương mại tính phí \$20-500/tháng
- **Tiền Xử Lý:** Giao diện cửa sổ kéo thả mới lạ độc nhất với framework này; những cái khác yêu cầu coding hoặc chấp nhận tự động hóa hộp đen
- **Độ Chính Xác:** So sánh với triển khai thủ công chuyên gia (96.2% vs 96.5%), vượt trội các nền tảng thương mại 1.4-3.1%

- **Khả Năng Tiếp Cận:** Khớp với dễ sử dụng của Edge Impulse trong khi cung cấp tính minh bạch và kiểm soát đầy đủ
- **Tính Linh Hoạt:** Hỗ trợ ML cổ điển (RF, SVM) bị tránh bởi các nền tảng tập trung neural, cho phép triển khai trên các thiết bị bị giới hạn cực kỳ

4.6.2 Tác Động Tối Ưu Hóa Siêu Tham Số

Bảng 4.6.2 định lượng các cải thiện độ chính xác từ điều chỉnh siêu tham số tự động qua GridSearchCV.

Bảng 4.6.2: Tác Động của Tối Ưu Hóa Siêu Tham Số (CV 5 fold)

Mô Hình	Độ chính xác Cơ sở	Độ chính xác Tối ưu	Cải thiện
Random Forest	91.8%	94.5%	+2.7%
Neural Network	94.3%	96.2%	+1.9%
SVM (RBF)	91.2%	93.8%	+2.6%

Điều chỉnh tự động cung cấp các cải thiện độ chính xác nhất quán 1.9-2.7%, chứng minh giá trị thực tế cho người dùng không có chuyên môn ML. Các siêu tham số tốt nhất được phát hiện:

- **Random Forest:** 100 cây, độ sâu tối đa 20, min_samples_split=2, class_weight='balanced'
- **Neural Network:** (100,) đơn vị ẩn, alpha=0.001, learning_rate=0.001
- **SVM:** C=10, gamma=0.01, class_weight='balanced'

4.7 Tóm Tắt Kết Quả

Xác thực thực nghiệm chứng minh:

1. Độ chính xác cao (94-96%) trên ba thuật toán ML trên nhiệm vụ HAR 5 lớp đầy thách thức
2. Tầm quan trọng quan trọng của cân bằng lớp cho phát hiện lớp thiểu số (+42-47% recall thiểu số)
3. Kiến trúc generator hỗ trợ nhiều họ đích triển khai từ cùng một pipeline huấn luyện duy nhất

4. Tính khả thi suy luận thời gian thực trên nền tảng tham chiếu phần cứng bị giới hạn tài nguyên (9-18ms mỗi cửa sổ)
5. Dấu chân bộ nhớ thấp (95-182KB flash) cho thấy dư địa tích hợp tốt trên các board cùng lớp tài nguyên
6. Hiệu suất cạnh tranh so với các nền tảng thương mại trong khi vẫn miễn phí và hoàn toàn minh bạch
7. Tối ưu hóa siêu tham số hiệu quả có thể tiếp cận với phi chuyên gia

Những kết quả này xác thực các mục tiêu cốt lõi của framework: dân chủ hóa phát triển edge AI trong khi duy trì các đặc điểm hiệu suất cấp chuyên nghiệp.

Chương 5

Thảo Luận

5.1 Diễn Giải Các Phát Hiện Chính

5.1.1 Hiệu Suất Mô Hình và Lựa Chọn Thuật Toán

Các kết quả thực nghiệm chứng minh rằng cả ba thuật toán đã triển khai đạt độ chính xác cao (93.8-96.2%) trên nhiệm vụ HAR năm lớp, với Neural Networks vượt trội một chút so với Random Forest và SVM. Phát hiện này phù hợp với tài liệu gần đây cho thấy rằng các mô hình ML cổ điển được điều chỉnh đúng cách vẫn cạnh tranh với học sâu cho các nhiệm vụ phân loại chuỗi thời gian trên các tập dữ liệu có kích thước vừa phải (hàng nghìn đến hàng chục nghìn mẫu)^[4;6].

Sự vượt trội của Neural Network (96.2% vs 94.5% RF) có thể được quy cho khả năng học các kết hợp phi tuyến của không gian đặc trưng 90 chiều thông qua các biến đổi lớp ẩn. Tuy nhiên, biên độ là khiêm tốn (+1.7%), và huấn luyện nhanh hơn của Random Forest (12.3s vs 45.7s) và tốc độ suy luận so sánh được (15ms vs 12ms) làm cho nó hấp dẫn cho tạo mẫu nhanh và các quy trình phát triển lặp lại phổ biến trong các bối cảnh nghiên cứu.

Thật thú vị, SVM—vốn chiếm ưu thế trong nghiên cứu HAR đầu tiên^[10]—đạt độ chính xác thấp hơn một chút (93.8%). Điều này có thể phản ánh lựa chọn tham số kernel không tối ưu mặc dù có tìm kiếm lưới, hoặc các hạn chế vốn có của kernel RBF trong việc nắm bắt các phụ thuộc thời gian phức tạp hiện diện trong dữ liệu chuyển động. Công việc tương lai có thể khám phá các kernel thay thế (polynomial, sigmoid) hoặc các phương pháp ensemble kết hợp nhiều hàm kernel.

5.1.2 Mất Cân Bằng Lớp: Một Thách Thức Quan Trọng nhưng Bị Bỏ Qua

Có lẽ phát hiện quan trọng nhất là tác động đáng kể của cân bằng lớp đến phát hiện lớp thiểu số. Không có `class_weight='balanced'`, các lớp thiểu số (*Walking Downstairs*, *Walking Upstairs*) chịu tỷ lệ recall 0.63-0.67—có nghĩa là một phần ba các hoạt động này không được phát hiện. Điều này là thảm khốc cho các ứng dụng thực tế (ví dụ: các hệ thống phát hiện té ngã không được bỏ lỡ các sự kiện xuống cầu thang).

Cân bằng lớp đã cải thiện recall thiểu số lên 0.93-0.95 (+42-47%), một sự chuyển đổi làm cho hệ thống có thể triển khai trong các bối cảnh quan trọng về an toàn. Điều này xác thực nghiên cứu trong học tập mất cân bằng^[9;19] và nhấn mạnh một bài học quan trọng: **độ chính xác tổng thể có thể gây hiểu lầm**. Một mô hình với độ chính xác tổng thể 91% có thể có độ chính xác lớp đa số 98% nhưng chỉ 65% độ chính xác lớp thiểu số—một sự đánh đổi không thể chấp nhận trong nhiều lĩnh vực.

Framework này giải quyết điều này bằng cách làm cho cân bằng lớp là một cài đặt mặc định, không phải tùy chọn. Lựa chọn thiết kế này phản ánh một triết lý ưu tiên triển khai thực tế mạnh mẽ hơn các số liệu hiệu suất benchmark có thể che giấu những điểm yếu quan trọng.

5.1.3 Tính Khả Thi Triển Khai Biên

Các kết quả thời gian suy luận (9-18ms mỗi cửa sổ trên vi điều khiển 64MHz) trên nền tảng tham chiếu Seeed XIAO nRF52840 xác thực rằng pipeline do framework sinh ra có thể vận hành thời gian thực trên ít nhất một lớp phần cứng đại diện (ARM Cortex-M4F). Điều quan trọng là nên diễn giải các số đo này như bằng chứng khả thi cho kiến trúc đa đích, thay vì như benchmark duy nhất cho mọi thiết bị mà framework hỗ trợ. Với cửa sổ 1,5 giây và bước trượt 0,75 giây, phân loại thời gian thực yêu cầu suy luận hoàn thành trong 750ms—các mô hình đề xuất đạt được điều này với biên độ 40-80×, để lại khoảng trống cho:

- Ensemble đa mô hình (kết hợp dự đoán RF + NN)
- Các phương thức cảm biến bổ sung (nhịp tim, GPS)
- Giao tiếp không dây (truyền BLE kết quả)
- Quản lý diện năng (ngủ sâu giữa các chu kỳ suy luận)

Trên nền tảng tham chiếu, sử dụng bộ nhớ (95-182KB flash) tương tự bảo thủ, chiếm 9-18% của 1MB khả dụng trên Seeed XIAO nRF52840. Điều này cho thấy kiến

trúc sinh mã còn nhiều dư địa khi chuyển sang các nền tảng bị giới hạn hơn như Arduino Nano 33 BLE (256KB flash) hoặc ESP32-C3 (384KB flash), mặc dù các số đo định lượng cuối cùng vẫn cần được benchmark trực tiếp trên từng thiết bị.

Tiêu thụ điện năng trung bình 30mW chuyển thành 65+ giờ hoạt động liên tục trên pin 600mAh—đủ cho các nghiên cứu đeo nhiều ngày mà không cần sạc lại. Tối ưu hóa chế độ Power kéo dài điều này đến 92 giờ, tiếp cận thời lượng triển khai cả tuần thường được yêu cầu cho phân tích đáng đi lâm sàng hoặc giám sát đào tạo thể thao.

5.1.4 Tiền Xử Lý Tương Tác: Một Thay Đổi Mô Hình

Giao diện của sổ thời gian kéo thả mới lạ đại diện cho một thay đổi mô hình từ các quy trình tiền xử lý truyền thống. Các phương pháp thông thường yêu cầu người dùng hoặc:

1. Viết mã để phân đoạn dữ liệu theo chương trình (rào cản kỹ thuật cao)
2. Sử dụng phân đoạn tự động hộp đen (không kiểm soát, lỗi thường xuyên)
3. Chú thích thủ công các dấu thời gian trong bảng tính (tẻ nhạt, dễ lỗi)

Giao diện kéo thả loại bỏ các điểm ma sát này bằng cách cho phép thao tác trực quan trực tiếp của các biểu đồ tín hiệu. Kiểm tra người dùng sơ bộ (không được báo cáo chính thức ở đây) cho thấy điều này giảm thời gian tiền xử lý 60-80% so với chú thích dấu thời gian thủ công, và giảm đáng kể ngưỡng chuyên môn cho chuẩn bị tập dữ liệu.

Thiết kế này phù hợp với các nguyên tắc của giao diện thao tác trực tiếp^[35], nơi người dùng tương tác với các biểu diễn trực quan của dữ liệu thay vì các đặc tả văn bản trừu tượng. Công việc tương lai nên tiến hành các nghiên cứu khả năng sử dụng chính thức so sánh giao diện này với phương pháp tự động của Edge Impulse và phân đoạn lập trình.

5.2 So Sánh với Công Trình Liên Quan

5.2.1 Các Nền Tảng Thương Mại

So sánh với Edge Impulse và SensiML tiết lộ các đánh đổi giữa chi phí, kiểm soát và tiện lợi:

Edge Impulse ưu tiên dễ sử dụng với kỹ thuật đặc trưng tự động và các quy trình triển khai, nhưng với chi phí đăng ký \$20-100/tháng và tính minh bạch tiền xử lý hạn

5.3. Các Hạn Chế và Mối Đe Dọa đối với Tính Hợp Lệ

chế. Framework này khớp với khả năng tiếp cận của nó trong khi cung cấp kiểm soát đầy đủ và loại bỏ chi phí định kỳ—quan trọng cho nghiên cứu học thuật và các bối cảnh thế giới đang phát triển bị giới hạn tài nguyên.

SensiML nhằm mục tiêu triển khai IoT thương mại với khả năng AutoML tinh vi nhưng chi phí \$99-500/tháng và hỗ trợ ít nền tảng hơn. Độ chính xác của nó trên kiểm tra HAR được thực hiện (93.1%) tụt hậu cả framework này (96.2%) và Edge Impulse (94.8%), có thể do các lựa chọn thuật toán khác nhau hoặc các bộ đặc trưng không được tối ưu hóa cho dữ liệu IMU.

Triển khai TensorFlow Lite thủ công đạt độ chính xác 96.5% (tốt hơn một chút so với 96.2% của framework này), nhưng yêu cầu kiến thức cấp chuyên gia về lượng tử hóa mô hình, tích hợp runtime TFLM và tối ưu hóa đặc thù nền tảng—kỹ năng vượt quá hầu hết các nhà nghiên cứu và người đam mê. Framework này thu hẹp khoảng cách này đến 0.3% trong khi không yêu cầu chuyên môn hệ thống nhúng.

Kết luận: framework này chiếm một vị thế độc đáo kết hợp hiệu suất cấp chuyên nghiệp, tính minh bạch đầy đủ, chi phí bằng không và khả năng triển khai đa đích từ cùng một pipeline huấn luyện—một kết hợp không có sẵn trong các giải pháp hiện có.

5.2.2 Các Hệ Thống HAR Học Thuật

So với nghiên cứu HAR học thuật sử dụng các phương pháp dựa trên IMU tương tự^[4], độ chính xác đạt được (96.2%) cạnh tranh với các kết quả đã công bố trên các tập dữ liệu benchmark (92-97% điển hình). Tuy nhiên, hầu hết các hệ thống học thuật chỉ báo cáo các nguyên mẫu Python hoặc phân tích ngoại tuyến; ít cung cấp mã nhúng sẵn sàng triển khai. Đóng góp của luận văn nằm ở việc bắc cầu "khoảng cách triển khai" này bằng cách tự động hóa việc dịch từ các mô hình scikit-learn đã huấn luyện sang firmware vi điều khiển sản xuất.

Ngoài ra, xử lý hệ thống được đề xuất về mặt cân bằng lớp giải quyết một điểm yếu trong nhiều tài liệu HAR, nơi các tập kiểm tra cân bằng hoặc các số liệu bị chi phối bởi lớp đa số che giấu các vấn đề hiệu suất thực tế^[19].

5.3 Các Hạn Chế và Mối Đe Dọa đối với Tính Hợp Lệ

5.3.1 Hạn Chế Tập Dữ Liệu

Thực nghiệm xác thực sử dụng dữ liệu từ một đối tượng đơn thực hiện năm hoạt động. Mặc dù đủ để chứng minh khả năng framework, tổng quát hóa đến các quần thể

5.3. Các Hạn Chế và Mối Đe Dọa đối với Tính Hợp Lệ

đa dạng (các độ tuổi, loại cơ thể, mức độ di động khác nhau) yêu cầu các nghiên cứu đa đối tượng. Các lớp hoạt động cũng hạn chế—các ứng dụng thực tế có thể cần phát hiện:

- Sự kiện té ngã (quan trọng cho chăm sóc người cao tuổi)
- Các hoạt động phức tạp (nấu ăn, dọn dẹp, lái xe)
- Chuyển đổi giữa các hoạt động
- Các mẫu dáng đi bất thường (chấn thương, tình trạng thần kinh)

Công việc tương lai nên xác thực framework sử dụng các tập dữ liệu HAR có sẵn công khai (ví dụ: UCI HAR, WISDM, PAMAP2) bao gồm các đối tượng đa dạng và phân loại hoạt động mở rộng.

5.3.2 Lựa Chọn Kích Thước Cửa Sổ

Cửa sổ 1,5 giây (150 mẫu @ 100Hz) với bước trượt 0,75 giây là một lựa chọn cân bằng bối cảnh thời gian và khả năng phản hồi. Nghiên cứu trước đây cho thấy kích thước cửa sổ ảnh hưởng đáng kể đến độ chính xác^[6]: các cửa sổ ngắn hơn ($<0.5s$) có thể bỏ lỡ các chu kỳ chuyển động đầy đủ, trong khi các cửa sổ dài hơn ($>2s$) giảm độ phân giải thời gian. Framework này hiện tại sử dụng kích thước cửa sổ cố định; các chiến lược cửa sổ thích ứng (thay đổi kích thước dựa trên cường độ hoạt động được phát hiện) có thể cải thiện hiệu suất nhưng thêm độ phức tạp.

5.3.3 Vị Trí và Hướng Cảm Biến

Các thử nghiệm giả định IMU được đeo ở cổ tay trong một hướng nhất quán. Triển khai thực tế gặp phải sự biến đổi:

- Các vị trí cơ thể khác nhau (cổ tay, hông, mắt cá chân) tạo ra các đặc tính tín hiệu riêng biệt
- Quay thiết bị tương đối với các trục cơ thể giới thiệu sự mơ hồ về hướng
- Nhiều cảm biến đồng thời (ví dụ: cả hai cổ tay + hông) yêu cầu hợp nhất cảm biến

Các hệ thống mạnh mẽ yêu cầu trích xuất đặc trưng không phụ thuộc hướng (ví dụ: tính toán độ lớn gia tốc thay vì các trục riêng lẻ) hoặc các thuật toán ước lượng hướng. Bộ đặc trưng của framework này bao gồm một số đặc trưng dựa trên độ lớn, nhưng đánh giá hệ thống về độ bền vững hướng là cần thiết.

5.3.4 Sự Đa Dạng Nền Tảng Tính Toán

Framework hiện đã hiện thực generator cho nhiều họ đích, nhưng mức độ xác thực cần được phân tách thành hai lớp. Ở lớp thứ nhất, *khả năng triển khai* đã được chứng minh bởi kiến trúc factory, các generator đặc thù nền tảng và các gói mã đầu ra tương ứng. Ở lớp thứ hai, *benchmark định lượng liên thiết bị* vẫn mới chỉ được đo chi tiết trên XIAO. Vì vậy, phát biểu chính xác của luận văn không nên là “framework đã benchmark trên mọi thiết bị”, mà là “framework cung cấp năng lực triển khai đa thiết bị ở mức kiến trúc và sinh mã, với XIAO là nền tảng thực nghiệm tham chiếu đầu tiên”. Các bước tiếp theo cần bổ sung benchmark trên các kiến trúc khác—AVR (Arduino Uno), RISC-V (ESP32-C3), ARM Cortex-M0+ (Raspberry Pi Pico)—để định lượng đầy đủ tính di động. Đặc điểm bộ nhớ và tốc độ chắc chắn sẽ khác nhau, đặc biệt trên các bộ xử lý AVR 8-bit thiếu hỗ trợ dấu phẩy động phần cứng.

5.3.5 Tính Hợp Lệ Bên Ngoài

Kết quả cụ thể cho nhận dạng hoạt động con người sử dụng các cảm biến IMU đeo cổ tay. Khả năng áp dụng cho các lĩnh vực theo dõi chuyển động khác (nhận dạng cử chỉ, theo dõi hành vi động vật, lập hồ sơ chuyển động robot) vẫn cần được xác thực. Kiến trúc của framework được thiết kế để không phụ thuộc miền, nhưng các chiến lược tiền xử lý và bộ đặc trưng có thể yêu cầu thích ứng cho các ứng dụng không phải HAR.

5.4 Các Đánh Đổi Thiết Kế và Các Lựa Chọn Thay Thế

5.4.1 ML Cổ Điển vs Học Sâu

Luận văn này cố ý chọn học máy cổ điển (Random Forest, SVM, Neural Networks) thay vì học sâu hiện đại (CNN, LSTM, Transformer) do:

- **Ràng Buộc Triển Khai:** Các mô hình cổ điển biên dịch thành mã C++ nhỏ (95-182KB), trong khi các mạng sâu yêu cầu các runtime suy luận chuyên biệt (TensorFlow Lite Micro) thêm 100-300KB overhead
- **Hiệu Quả Huấn Luyện:** Các mô hình cổ điển huấn luyện trong giây đến phút trên CPU; các mạng sâu yêu cầu GPU và giờ huấn luyện
- **Khả Năng Diễn Giải:** Tầm quan trọng đặc trưng Random Forest và ranh giới quyết định SVM có thể kiểm toán; các mạng sâu mờ đục

5.4. Các Đánh Đổi Thiết Kế và Các Lựa Chọn Thay Thế

Tuy nhiên, học sâu xuất sắc trên các tập dữ liệu lớn (>100K mẫu) và đầu vào tín hiệu thô (bỏ qua kỹ thuật đặc trưng thủ công). Các phiên bản framework tương lai nên tích hợp hỗ trợ TensorFlow Lite cho người dùng có tài nguyên tính toán và các tập dữ liệu được chú thích lớn.

5.4.2 Đặc Trưng Thủ Công vs Đặc Trưng Học

Bộ 90 đặc trưng miền thời gian được sử dụng bao gồm các thống kê thủ công (15 thống kê $\times$ 6 trục). Điều này yêu cầu kiến thức miền (biết đặc trưng nào phân biệt các hoạt động) nhưng đảm bảo khả năng diễn giải, biểu diễn gọn nhẹ, và tương đồng bit-exact giữa Python và C++. Học sâu từ đầu đến cuối có thể học các đặc trưng tự động, nhưng với chi phí các mô hình lớn hơn và tính minh bạch giảm.

Một phương pháp trung gian—sử dụng các bộ mã hóa tự động convolutional cho trích xuất đặc trưng học theo sau bởi ML cổ điển cho phân loại—có thể kết hợp lợi ích. Điều này vẫn là công việc tương lai.

5.4.3 Tạo Mã Trực tiếp vs Runtime Suy luận: Đánh Giá Thực tiễn

Phần 3.6.2 trình bày lý do thiết kế cho việc chọn tạo mã trực tiếp thay vì runtime suy luận. Sau quá trình phát triển và xác thực hoàn chỉnh, có thể đánh giá liệu lựa chọn này có được minh chứng trong thực tế hay không.

Lợi ích Đã được Xác nhận Ba lợi ích dự kiến đã được xác nhận qua thực nghiệm:

Thứ nhất, *dấu chân bộ nhớ tối thiểu*: Neural Network chỉ chiếm 95KB flash—bao gồm trọng số mô hình, logic suy luận, trích xuất đặc trưng 15 thống kê, và chuẩn hóa StandardScaler. Với TFLite Micro, cùng chức năng sẽ yêu cầu thêm 150–250KB cho runtime interpreter, đẩy tổng lên 245–345KB và chiếm >25% flash khả dụng chỉ cho suy luận. Trên nền tảng tham chiếu Saeed XIAO (1MB flash), điều này vẫn khả thi; trên các vi điều khiển với 256KB flash (ví dụ: STM32F103, ATmega2560), phương pháp runtime sẽ không thể triển khai.

Thứ hai, *minh bạch cho gỡ lỗi*—đây là yếu tố có tác động lớn nhất ngoài dự kiến. Khi mô hình đã triển khai luôn dự đoán `walking_downstairs`, có thể: (a) đọc chính xác mã C++ đã tạo để xác minh logic suy luận, (b) in giá trị đặc trưng trung gian trên serial để so sánh với Python, (c) xác định lỗi zero-padding và kurtosis formula bằng cách truy vết từng dòng. Với runtime đóng gói, bước (b) và (c) sẽ yêu cầu công cụ debugging chuyên biệt hoặc không thể thực hiện. Kinh nghiệm phát triển thực tế

5.4. Các Đánh Đổi Thiết Kế và Các Lựa Chọn Thay Thế

cho thấy 3 trong 10 phát hiện kỹ thuật (Finding 1: zero-padding, Finding 2: kurtosis, Finding 7: feature order) chỉ có thể phát hiện nhờ khả năng kiểm tra mã đã tạo dòng-theo-dòng.

Thứ ba, *hỗ trợ trực tiếp ML cổ điển*: Framework hỗ trợ Random Forest—thuật toán không có đường dẫn chuyển đổi trực tiếp sang TFLite Micro. Chuỗi scikit-learn → ONNX → TFLite giới thiệu rủi ro sai lệch; trong khi phương pháp trực tiếp đọc `model.estimators_` (danh sách cây quyết định) và tạo mã duyệt cây nhị phân chính xác.

Chi phí Phát triển Thực tế Tạo mã trực tiếp cũng có chi phí không nhỏ. Framework yêu cầu triển khai và xác minh logic suy luận riêng cho mỗi thuật toán: lan truyền tiến cho Neural Network, tính kernel RBF và bỏ phiếu OvO cho SVM, duyệt cây nhị phân và bỏ phiếu đa số cho Random Forest. Tổng cộng, `base_generator.py` chứa ~1580 dòng mã cho trích xuất đặc trưng chung, và mỗi trình tạo đặc thù thêm 200–500 dòng. Bất kỳ lỗi nào trong logic suy luận đã tạo đều ảnh hưởng trực tiếp đến độ chính xác triển khai—như đã chứng minh bởi Finding 2 (kurtosis) và Finding 7 (feature order).

Tuy nhiên, chi phí này là *một lần*: sau khi trình tạo được xác minh cho một thuật toán, nó hoạt động chính xác cho mọi mô hình được huấn luyện bằng thuật toán đó, bất kể số lớp, số đặc trưng, hoặc siêu tham số.

Kết luận về Đánh đổi Tạo mã trực tiếp là lựa chọn đúng cho *bối cảnh sử dụng cụ thể của framework*: tập dữ liệu vừa phải (<10K mẫu), thuật toán ML cổ điển, vi điều khiển bị giới hạn tài nguyên, và yêu cầu minh bạch cao. Đối với các bối cảnh khác—tập dữ liệu lớn, mạng sâu, vi điều khiển >4MB flash—runtime suy luận (TFLite Micro) hoặc trình biên dịch mô hình (TVM) sẽ phù hợp hơn. Framework được thiết kế modular để tích hợp đường dẫn TFLite Micro trong tương lai mà không thay thế phương pháp trực tiếp hiện có.

5.4.4 Các Chế Độ Tối Ưu Hóa

Bốn chế độ tối ưu hóa (accuracy, speed, power, balanced) cung cấp tính linh hoạt, nhưng yêu cầu người dùng hiểu các đánh đổi. Các thiết kế thay thế có thể:

- **Lập Hồ Sơ Tự Động**: Framework đo khả năng thiết bị và chọn chế độ tối ưu
- **Tối Ưu Hóa Đa Mục Tiêu**: Khám phá biên Pareto để tìm các thỏa hiệp độ chính xác-tốc độ-điện năng tốt nhất

- **Runtime Thích Ứng:** Thiết bị điều chỉnh động độ phức tạp suy luận dựa trên mức pin hoặc cường độ chuyển động được phát hiện

Triển khai các chiến lược tinh vi này sẽ cải thiện khả năng sử dụng nhưng tăng độ phức tạp framework.

5.5 Tương đồng Huấn luyện-Triển khai trong Edge ML

Một phát hiện quan trọng trong quá trình phát triển và xác thực framework là tầm quan trọng thiết yếu của *tương đồng huấn luyện-triển khai* (training-deployment parity)—sự đảm bảo rằng quá trình trích xuất đặc trưng trong môi trường huấn luyện (Python) tạo ra kết quả **giống hệt** với quá trình trích xuất đặc trưng trên thiết bị biên (C++/MicroPython). Vấn đề này ít được nghiên cứu trong tài liệu edge ML nhưng có tác động nghiêm trọng đến hiệu suất triển khai thực tế^[27;32].

5.5.1 Các Nguồn Không khớp Được Xác định

Trong quá trình phát triển, đã xác định và giải quyết nhiều nguồn sai lệch giữa pipeline huấn luyện Python và mã suy luận C++ đã tạo:

Artifact Đệm Số Không (Zero-Padding Artifact) Pipeline trích xuất đặc trưng ban đầu đệm các cửa sổ ngắn hơn kích thước mục tiêu bằng hàng toàn số không. Trên thiết bị thực, bộ đệm trượt luôn chứa đầy dữ liệu cảm biến thực (không có giá trị zero), vì cảm biến liên tục đo gia tốc trọng trường. Sự khác biệt này tạo ra phân phối đặc trưng hoàn toàn khác nhau giữa huấn luyện (`acc_mag_min = 0`) và triển khai (`acc_mag_min ≈ 9,81`), đẩy tất cả đặc trưng đầu vào ra ngoài vùng huấn luyện.

Sai lệch Công thức Thống kê Công thức độ nhọn (kurtosis) trong mã C++ tạo ra ban đầu sử dụng độ lệch chuẩn mẫu ($n - 1$ ở mẫu số) cho chuẩn hóa z-scores, trong khi Python pandas sử dụng độ lệch chuẩn tổng thể (n ở mẫu số). Sai lệch hệ thống:

$$\text{Ratio} = \left(\frac{n-1}{n}\right)^2 \approx 0,987 \text{ cho } n = 150 \quad (5.5.1)$$

tương đương sai lệch $\sim 1,3\%$ cho mỗi đặc trưng kurtosis và skewness—nhỏ nhưng hệ thống và tích lũy qua 6 trục.

Thứ tự Đặc trưng Không khớp Python huấn luyện trên các cột DataFrame được sắp xếp theo thứ tự bảng chữ cái, trong khi C++ trích xuất đặc trưng theo thứ tự cố định (magnitude $\rightarrow$ per-axis). Mức reorder đặc trưng tại thời điểm tạo mã đảm bảo trọng số mô hình và tham số scaler tương ứng chính xác với thứ tự trích xuất C++.

5.5.2 Tác động Thực tế

Trong ca kiểm chứng trên thiết bị tham chiếu Seeed XIAO nRF52840, các lỗi không khớp khiến mô hình luôn dự đoán lớp `walking_downstairs` bất kể hoạt động thực tế. Phân tích cho thấy output biases của mạng nơ-ron $[-0,116; -0,160; -0,066; 0,116; 0,048]$ chi phối khi đặc trưng đầu vào nằm hoàn toàn ngoài phân phối huấn luyện—lớp có bias cao nhất (0,116, tương ứng `walking_downstairs`) luôn thắng quyết. Bài học này không chỉ áp dụng cho XIAO mà cho toàn bộ ma trận đích của framework, vì bất kỳ backend nào cũng sẽ thất bại nếu tính tương đồng huấn luyện-triển khai không được đảm bảo.

5.5.3 Danh sách Kiểm tra Tương đồng

Bảng 5.5.1 tóm tắt 12 biện pháp đảm bảo tương đồng đã được triển khai và xác minh trong framework:

5.5. Tương đồng Huấn luyện-Triển khai trong Edge ML

Bảng 5.5.1: Danh sách kiểm tra tương đồng huấn luyện-triển khai

#	Biện pháp	Mô tả
1	Loại bỏ FFT	Tránh sai lệch triển khai FFT giữa Python và C++
2	Edge-value replication	Thay thế zero-padding bằng lặp giá trị biên
3	Kurtosis/skewness	Sử dụng population std cho z-scores, khớp pandas
4	Thứ tự đặc trưng	Reorder tham số mô hình tại thời điểm tạo mã
5	Đơn vị cảm biến	m/s ² cho gia tốc, deg/s cho con quay hồi chuyển
6	Tốc độ lấy mẫu	100Hz trong cả thu thập và suy luận
7	Kích thước cửa sổ	150 mẫu, đọc từ metadata mô hình
8	StandardScaler	Cùng mean/std, cùng clamp range
9	Chuẩn hóa đơn	Scaling chỉ xảy ra 1 lần trong pipeline
10	Feature order remap	Trọng số/scaler reorder cho thứ tự C++
11	Trích xuất idempotent	Tham số mô hình chỉ trích xuất 1 lần
12	Xác thực kiến trúc	Phân biệt CNN (raw window) vs feature-based

5.5.4 So sánh với Nền tảng Thương mại

Các nền tảng thương mại như Edge Impulse hoạt động theo mô hình hộp đen (black-box), trong đó pipeline trích xuất đặc trưng không thể kiểm tra hoặc xác minh bởi người dùng. Nếu một lỗi tương đồng tương tự tồn tại, người dùng không có cách nào phát hiện hoặc sửa chữa. Tính **minh bạch hoàn toàn** (full transparency) của framework cho phép:

- Kiểm tra trực tiếp giá trị đặc trưng (phát hiện `acc_mag_min = 0`)
- Truy vết ngược đến nguyên nhân gốc (zero-padding)
- Xác minh từng công thức giữa Python và C++
- Sửa lỗi và huấn luyện lại mà không chờ nhà cung cấp

Phát hiện này nhấn mạnh rằng **tính minh bạch không chỉ là triết lý mã nguồn mở mà là yêu cầu kỹ thuật thiết yếu** cho triển khai edge ML đáng tin cậy—đặc biệt khi lỗi biểu hiện ở cấp thiết bị chứ không phải ở cấp mô hình.

5.6 Tăng cường Dữ liệu và Thiết kế Nhận biết Lớp

5.6.1 Hiệu quả Tăng cường cho Dữ liệu IMU

Tăng cường dữ liệu là kỹ thuật đã được chứng minh hiệu quả trong cải thiện tổng quát hóa mô hình, đặc biệt với tập dữ liệu nhỏ^[21;38]. Tuy nhiên, việc áp dụng cho dữ liệu IMU đòi hỏi cân nhắc cẩn thận vì tín hiệu cảm biến quán tính có các ràng buộc vật lý nghiêm ngặt: gia tốc trọng trường luôn hiện diện, biên độ dao động phải nằm trong phạm vi vật lý, và tỷ lệ tín hiệu-trên-nhiều (SNR) giữa các hoạt động khác nhau rất đáng kể.

Năm phương pháp tăng cường được chọn mô phỏng các nguồn biến thiên thực tế mà mô hình sẽ gặp trong triển khai: nhiễu cảm biến (jitter), biến thiên cường độ (scaling), thay đổi vị trí đặt cảm biến (rotation), biến thiên tốc độ thực hiện (time warping) và tính bất biến với thứ tự thời gian (permutation). Sự kết hợp này bao phủ các nguồn biến thiên chính được xác định trong tài liệu HAR^[29;37].

5.6.2 Bài học từ Tăng cường Không-Nhận-biết Lớp

Thực nghiệm ban đầu áp dụng tất cả năm phương pháp đồng nhất cho mọi lớp hoạt động, phát hiện vấn đề: mô hình Neural Network sau tăng cường thống nhất phân loại sai hoạt động *still* thành *walking_downstairs* với tỷ lệ đáng kể. Nguyên nhân gốc: jitter ($\sigma = 0,05$) và rotation (15°) trên cửa sổ tĩnh tạo ra dao động nhân tạo có biên độ tương đương hoạt động cường độ thấp (đi xuống cầu thang chậm), dẫn đến chồng lấn vùng đặc trưng giữa hai lớp.

Phân tích định lượng xác nhận: perturbation tối đa trên cửa sổ tĩnh với tăng cường đầy đủ đạt 0,6059 (tương đương dao động gia tốc đáng kể), trong khi cửa sổ tĩnh gốc có perturbation $< 0,01$. Chiến lược nhận biết lớp giảm perturbation tĩnh về $\leq 0,0002$ (micro-jitter), bảo toàn hoàn toàn đặc tính phân biệt.

5.6.3 Nguyên tắc Thiết kế

Kinh nghiệm này dẫn đến nguyên tắc thiết kế tăng cường cho IMU HAR:

1. **Bảo toàn đặc tính phân biệt:** Tăng cường không được phá hủy đặc tính khiến một lớp khác biệt với các lớp khác—với hoạt động tĩnh, đó là biên độ dao động cực thấp

2. **Phân biệt theo tính chất hoạt động:** Hoạt động động và tĩnh có bản chất vật lý khác nhau cơ bản; áp dụng cùng phép biến đổi là không phù hợp
3. **Phát hiện tự động:** Framework nên phát hiện tự động lớp tĩnh/động thay vì yêu cầu người dùng cấu hình thủ công, giảm rào cản kỹ thuật

Quan điểm này khác biệt với đa số nghiên cứu tăng cường dữ liệu cho HAR, vốn áp dụng biến đổi đồng nhất^[23;38]. Luận văn này cho rằng sự phân biệt nhận biết lớp là đặc biệt quan trọng cho các hệ thống triển khai biên, nơi số lượng lớp thường nhỏ và sai sót trên lớp tĩnh (ví dụ: phát hiện nhầm hoạt động khi bệnh nhân đứng yên) có thể có hậu quả lâm sàng nghiêm trọng.

5.7 Từ chối Hoạt động Dựa trên Ngưỡng Tin cậy

5.7.1 Vấn đề Phân loại Ép buộc

Các hệ thống phân loại tiêu chuẩn gán một nhãn cho mọi đầu vào, kể cả khi đầu vào thuộc phân phối ngoài (out-of-distribution). Trong triển khai HAR thực tế, đây là vấn đề phổ biến: người dùng thường thực hiện các hoạt động không thuộc tập huấn luyện (ngồi xuống ghế, gỡ đầu, cúi lấy đồ), và cảm biến ghi nhận tín hiệu chuyển động tương ứng. Bộ phân loại không có cơ chế từ chối sẽ phân loại sai các đầu vào này với độ tin cậy biểu kiến cao—hiện tượng được gọi là *overconfident extrapolation*^[20].

5.7.2 Phương pháp Tin cậy Trực tiếp

Thay vì huấn luyện một mô hình phát hiện bất thường riêng biệt (approach của Edge Impulse), framework trích xuất tin cậy trực tiếp từ đầu ra phân loại hiện có. Phương pháp này có ba ưu điểm thực tế:

- **Không tốn thêm bộ nhớ:** Không mô hình bổ sung, không tham số bổ sung
- **Không cần dữ liệu bổ sung:** Không yêu cầu thu thập mẫu “không hoạt động” để huấn luyện
- **Tích hợp tự nhiên:** Tin cậy được tính trong quá trình suy luận đã có, không thêm bước xử lý

5.7.3 Hạn chế và Hướng Tương lai

Cần lưu ý rằng tin cậy softmax thường *kém hiệu chỉnh* (poorly calibrated)^[18]—giá trị softmax 0,8 không nhất thiết có nghĩa 80% xác suất đúng. Điều này có nghĩa:

- Random Forest có tin cậy hiệu chỉnh tự nhiên tốt hơn (tỷ lệ bỏ phiếu có ý nghĩa thống kê xác suất)
- Neural Network/SVM qua softmax có thể quá tự tin—các kỹ thuật hiệu chỉnh nhiệt độ (temperature scaling)^[18] có thể cải thiện chất lượng tin cậy
- Ngưỡng $\tau = 0,6$ là giá trị thực nghiệm; điều chỉnh tự động dựa trên đường cong ROC tập xác thực là hướng tương lai

Dù vậy, ngay cả tin cậy chưa hiệu chỉnh cũng cung cấp “tuyến phòng thủ đầu tiên” hiệu quả chống lại các dự đoán sai nghiêm trọng, đặc biệt cho các đầu vào hoàn toàn ngoài phân phối nơi tất cả xác suất lớp đều thấp.

5.8 Ý Nghĩa cho Thực Hành

5.8.1 Nghiên Cứu và Giáo Dục

Framework dân chủ hóa nghiên cứu edge AI bằng cách loại bỏ rào cản chi phí (so với các nền tảng thương mại \$20-500/tháng) và rào cản kỹ thuật (so với triển khai TensorFlow Lite thủ công). Điều này cho phép:

- Các khóa học ML đại học/sau đại học bao gồm các phòng thí nghiệm triển khai biên thực hành
- Các nhà nghiên cứu ở các nước đang phát triển tiến hành các nghiên cứu HAR mà không cần ngân sách thể chế cho giấy phép thương mại
- Tạo mẫu nhanh các thuật toán nhận dạng hoạt động mới lạ mà không cần lập trình nhúng cấp thấp

5.8.2 Các Ứng Dụng Y Tế

Các mô hình đã xác thực có thể hỗ trợ giám sát sức khỏe đeo được:

- **Phục Hồi Chức Năng:** Theo dõi tiến trình phục hồi bằng cách giám sát mức độ hoạt động và chất lượng dáng đi

- **Chăm Sóc Người Cao Tuổi:** Phát hiện té ngã, hành vi ít vận động và suy giảm hoạt động
- **Thử Nghiệm Lâm Sàng:** Đo lường hoạt động khách quan thay thế nhật ký tự báo cáo

Tuy nhiên, triển khai y tế yêu cầu tuân thủ quy định (phê duyệt FDA, đánh dấu CE) và các nghiên cứu xác thực đa địa điểm rộng rãi—vượt quá phạm vi của framework nghiên cứu này.

5.8.3 Thể Thao và Thể Dục

Các thiết bị đeo tiêu dùng (đồng hồ thông minh, băng thể dục) có thể tích hợp các mô hình được tạo bởi framework cho:

- Phát hiện tập luyện tự động (chạy, đạp xe, cử tạ)
- Phân tích hình thức (phát hiện kỹ thuật squat không đúng, bất đối xứng dáng chạy)
- Huấn luyện cá nhân hóa (phản hồi thời gian thực về cường độ hoạt động)

5.9 Phân Tích Các Phát Hiện Kỹ Thuật Quan Trọng

5.9.1 Bộ lọc Kalman: Đột Phá về Training-Deployment Parity

Việc triển khai bộ lọc Kalman constant-velocity đại diện cho một đột phá quan trọng trong việc giải quyết vấn đề khoảng cách tương đồng huấn luyện-triển khai (training-deployment parity gap). Đây là lần đầu tiên framework có một bộ lọc tiền xử lý với **khoảng cách parity bằng không**.

So sánh với Các Phương pháp IIR Truyền thống

Các bộ lọc IIR Butterworth truyền thống tồn tại lỗ hổng parity vốn có:

- **Huấn luyện:** `filtfilt` (non-causal, zero-phase) xử lý dữ liệu theo hai chiều
- **Triển khai:** `lfilter` (causal, forward-only) chỉ có thể xử lý theo một chiều do ràng buộc thời gian thực

5.9. Phân Tích Các Phát Hiện Kỹ Thuật Quan Trọng

Khoảng cách này gây sai lệch đặc biệt nghiêm trọng với các đặc trưng nhạy cảm với pha như skewness và kurtosis, có thể dẫn đến suy giảm hiệu suất mô hình 2-5% khi triển khai thực tế.

Ưu thế Thiết kế của Kalman Filter

Bộ lọc Kalman khắc phục vấn đề này từ gốc thông qua thiết kế vốn có:

1. **Tính nhân quả tự nhiên:** Kalman filter về bản chất là causal—chỉ sử dụng observation hiện tại và trạng thái trước đó
2. **Tính thích ứng:** Gain điều chỉnh tự động dựa trên prediction error, tạo ra đặc tính lọc thích ứng
3. **Mô hình vật lý:** Constant-velocity model phù hợp với bản chất chuyển động của dữ liệu IMU
4. **Tham số trực quan:** Process noise (Q) và measurement noise (R) có ý nghĩa vật lý rõ ràng

So sánh với Edge Impulse: Edge Impulse không cung cấp Kalman filtering như một tùy chọn tiền xử lý. Spectral analysis block của họ sử dụng các bandpass filter cố định, thiếu khả năng thích ứng và đảm bảo parity của Kalman filter.

5.9.2 Phân Tích Kiến trúc Tạo Mã Đa Kiến trúc

Việc hỗ trợ đồng thời các mô hình feature-based (RF/SVM/NN) và end-to-end (CNN) trong cùng một framework đặt ra những thách thức kiến trúc độc đáo mà Finding 16 đã chỉ ra.

Sự Phức Tạp Metadata theo Kiến trúc

Case study về CNN metadata mismatch (Finding 16) minh họa tầm quan trọng của architecture-aware code generation:

Vấn đề: CNN model hiển thị filename "f33" (33 features) mặc dù thực tế sử dụng 6 channels per timestep.

Nguyên nhân gốc: Fallback logic không phân biệt được giữa:

- Feature-based models: cần statistical feature names (e.g., "acc_x_mean", "gyro_y_kurtosis")
- CNN models: cần channel placeholders (e.g., "ch0", "ch1", ..., "ch5")

Giải pháp kiến trúc: Architecture-aware metadata generation với logic phân nhánh rõ ràng cho từng loại mô hình.

Phân Biệt với Edge Impulse

Edge Impulse xử lý sự phức tạp này bằng cách có "processing blocks" riêng biệt cho từng loại mô hình. Framework này phải quản lý cả hai pipeline trong cùng một codebase thống nhất, đòi hỏi:

- Validation logic nhận biết kiến trúc
- Metadata semantics khác nhau cho từng model type
- Input dimension handling linh hoạt (90 features vs 900 raw inputs)

Điều này thể hiện trade-off thiết kế: complexity tăng để đổi lấy flexibility và unified user experience.

5.9.3 Hạn chế TFLite và Giải pháp Keras Surrogate

Finding 15 tiết lộ những hạn chế quan trọng trong hệ sinh thái ONNX-ML và TFLite đối với các thuật toán ML cổ điển.

Phân tích Chi tiết về ONNX-ML Bottleneck

Pipeline chuyển đổi bị gián đoạn tại bước ONNX→TensorFlow:

1. **skl2onnx:** Thành công tạo ONNX-ML operators (TreeEnsembleClassifier, SVM-Classifer)
2. **onnx2tf:** **Không hỗ trợ** ONNX-ML operators—chỉ hỗ trợ standard ONNX operators
3. **Kết quả:** RF/SVM không thể chuyển đổi trực tiếp sang TFLite

Đây là một hạn chế hệ thống trong hệ sinh thái, không phải lỗi triển khai cụ thể của framework.

Đánh giá Keras Surrogate Method

Giải pháp Keras surrogate đưa ra trade-offs quan trọng:

Ưu điểm:

- Cho phép RF/SVM sử dụng TFLite pipeline
- Tận dụng được quantization INT8 và runtime optimizations
- Tương thích với hệ sinh thái TensorFlow hiện có

Nhược điểm:

- **Approximation loss:** Keras MLP chỉ đạt 80-90% agreement với RF/SVM gốc
- **Memory overhead:** MLP surrogate có thể lớn hơn direct C++ implementation
- **Training complexity:** Cần additional training step và hyperparameter tuning

Khuyến nghị sử dụng:

- **RF/SVM:** Ưu tiên direct C++ generation (exact, compact, fast)
- **NN/CNN:** Sử dụng TFLite để tận dụng ecosystem mature
- **TFLite surrogate:** Chỉ khi yêu cầu standardized runtime integration

Implications cho Edge AI Ecosystem

Findings này chỉ ra một gap quan trọng trong edge AI toolchain: ONNX-ML và standard ONNX có operator domains khác nhau, tạo ra compatibility issues ở interface layers. Điều này nhấn mạnh giá trị của direct code generation approach cho classical ML algorithms.

5.10 Các Hướng Nghiên Cứu Tương Lai

5.10.1 Các Mở Rộng Ngắn Hạn

Các cải tiến ngay lập tức trong kiến trúc hiện tại:

1. **Các Thuật Toán Bổ Sung:** Tích hợp XGBoost, LightGBM, k-NN
2. **Lựa Chọn Đặc Trưng:** Triển khai phân tích tầm quan trọng đặc trưng tự động để giảm chiều
3. **Nén Mô Hình:** Cắt tỉa, huấn luyện nhận thức lượng tử hóa để giảm thêm bộ nhớ
4. **Nhiều Nền Tảng Hơn:** Hỗ trợ triển khai ESP32, STM32, nRF5340

5. **Thực Quan Hóa Thời Gian Thực:** Luồng hoạt động trực tiếp từ thiết bị đã triển khai đến GUI framework

5.10.2 Tầm Nhìn Dài Hạn

Các cải tiến chuyển đổi yêu cầu thay đổi kiến trúc:

1. **Tích Hợp Học Sâu:** Tạo TFLite Micro cho các mô hình CNN/LSTM
2. **Học Liên Bang:** Cho phép huấn luyện mô hình cộng tác trên nhiều thiết bị mà không chia sẻ dữ liệu thô
3. **AI Có Thể Giải Thích:** Thực quan hóa các mẫu cảm biến nào kích hoạt dự đoán hoạt động cụ thể
4. **Hợp Nhất Đa Phương Thức:** Kết hợp IMU với âm thanh, áp suất hoặc dữ liệu camera
5. **Học Trực Tuyến:** Các mô hình đã triển khai thích ứng với các mẫu cụ thể của người dùng theo thời gian
6. **Tích Hợp Đám Mây:** Backend tùy chọn cho sao lưu dữ liệu, phiên bản mô hình, chia sẻ mô hình cộng đồng

5.10.3 Các Cải Tiến Khả Năng Sử Dụng

- Các nghiên cứu người dùng chính thức so sánh giao diện tiền xử lý với các lựa chọn thay thế (Edge Impulse, coding thủ công)
- Các quy trình làm việc có hướng dẫn cho người dùng mới với hướng dẫn từng bước và các tập dữ liệu ví dụ
- Giám sát thiết bị thời gian thực cho thấy các luồng cảm biến trực tiếp và kết quả dự đoán
- Kiểm tra phần cứng tự động để xác thực các mô hình đã triển khai trước khi triển khai hiện trường

5.11 Nhận Xét Kết Thúc

Sự phổ biến của các cảm biến đeo được và điện toán biên phổ biến tạo ra các cơ hội chưa từng có cho các ứng dụng theo dõi chuyển động trải dài y tế, thể thao, nghiên

cứu và xa hơn nữa. Tuy nhiên, việc nhận ra tiềm năng này đã bị cản trở bởi sự phức tạp của phát triển mô hình AI và sự khan hiếm các công cụ có thể tiếp cận bắc cầu khoảng cách giữa thu thập dữ liệu và triển khai biên.

Nghiên cứu này chứng minh rằng khoảng cách này không phải là không thể vượt qua. Bằng cách kết hợp các giao diện tiền xử lý trực quan, các quy trình huấn luyện mô hình tự động và tạo mã thông minh, đã tạo ra một hệ thống trao quyền cho người dùng không có chuyên môn chuyên biệt để phát triển các ứng dụng theo dõi chuyển động cấp chuyên nghiệp. Bản chất mã nguồn mở của framework đảm bảo tính bền vững và tiến hóa được điều khiển bởi cộng đồng, trong khi kiến trúc modular của nó cung cấp một nền tảng cho đổi mới nghiên cứu đang diễn ra.

Khi edge AI tiếp tục trưởng thành, các công cụ như framework đề xuất sẽ đóng một vai trò ngày càng quan trọng trong việc dân chủ hóa quyền truy cập vào các công nghệ tiên tiến. Mong rằng công việc này truyền cảm hứng cho các nỗ lực tương tự để làm cho các khả năng AI tinh vi có thể tiếp cận được với các nhà nghiên cứu, nhà phát triển và nhà đổi mới trên toàn thế giới—cuối cùng tăng tốc tốc độ khám phá và triển khai trong theo dõi chuyển động và xa hơn nữa.

Chương 6

Kết Luận

6.1 Tóm Tắt Đóng Góp

Nghiên cứu này giải quyết thách thức làm cho theo dõi chuyển động được hỗ trợ bởi AI trở nên có thể tiếp cận với các nhà nghiên cứu, nhà phát triển và giáo viên mà không hy sinh hiệu suất hoặc kiểm soát trên các thiết bị biên dị thể. Luận văn này trình bày một framework mã nguồn mở toàn diện thống nhất tiền xử lý dữ liệu, huấn luyện mô hình và triển khai biên vào một quy trình đơn thân thiện với người dùng. Các đóng góp chính của hệ thống trải dài đổi mới thuật toán, kỹ thuật phần mềm và xác thực thực nghiệm:

6.1.1 Đóng Góp Kỹ Thuật

1. **Giao Diện Tiền Xử Lý Tương Tác Mới Lại:** Cơ chế phân đoạn cửa sổ thời gian kéo thả đại diện cho công cụ chú thích trực quan đầu tiên cho dữ liệu cảm biến chuyển động, loại bỏ sự lựa chọn truyền thống giữa việc coding logic phân đoạn phức tạp và chấp nhận các phương pháp tự động hộp đen. Đổi mới này giảm thời gian tiền xử lý ước tính 60-80% trong khi cải thiện độ chính xác chú thích thông qua phản hồi trực quan trực tiếp.
2. **Hỗ Trợ Đa Thuật Toán Toàn Diện:** Không giống như các nền tảng thương mại giới hạn ở mạng nơ-ron hoặc các thuật toán ML chuyên biệt, framework này tích hợp ba phương pháp bổ sung (Random Forest, SVM, Neural Networks) với các API nhất quán và tối ưu hóa siêu tham số tự động. Tính linh hoạt này cho phép người dùng chọn các thuật toán khớp với các ràng buộc triển khai của họ—Random Forest cho tạo mẫu nhanh (huấn luyện 12.3s), Neural Networks cho độ chính xác tối đa (96.2%), hoặc SVM cho khả năng diễn giải lý thuyết.

3. **Kiến Trúc Triển Khai Đa Thiết bị Nhận Biết Tối Ưu hóa:** Thay vì gắn với một bo mạch duy nhất, tầng generator được tổ chức theo ba trục—loại mô hình, nền tảng đích và backend triển khai—cho phép xuất các gói C/C++/MicroPython cho Arduino-compatible boards, ARM Cortex-M, ESP-IDF, Zephyr và các backend runtime thay thế như TFLite Micro hoặc ONNX Runtime. Mã được tạo trên nền tảng tham chiếu đạt độ trễ suy luận 12-18ms và dấu chân bộ nhớ 95-182KB trên vi điều khiển 64MHz, cho thấy kiến trúc đủ gọn để chuyển sang các thiết bị cùng lớp tài nguyên.
4. **Xử Lý Mất Cân Bằng Lớp:** Bằng cách làm cho `class_weight='balanced'` là một cấu hình mặc định thay vì tham số tùy chọn, framework hệ thống giải quyết một thách thức quan trọng nhưng thường bị bỏ qua trong theo dõi chuyển động thực tế. Thực nghiệm chứng minh cải thiện +42-47% trong recall lớp thiểu số, chuyển đổi các mô hình từ các nguyên mẫu nghiên cứu thành các hệ thống sẵn sàng triển khai.
5. **Đảm Bảo Tương Đồng Huấn Luyện-Triển Khai:** Luận văn này xác định và giải quyết một thách thức ít được nghiên cứu trong edge ML: sự không khớp giữa pipeline trích xuất đặc trưng huấn luyện (Python) và suy luận (C++). Quy trình xác minh 12 điểm kiểm tra đảm bảo tính nhất quán bit-exact cho các đặc trưng miền thời gian, bao gồm sửa lỗi zero-padding, thống nhất công thức kurtosis/skewness, và reorder đặc trưng tự động tại thời điểm tạo mã. Phát hiện này nhấn mạnh rằng tính minh bạch không chỉ là triết lý mã nguồn mở mà là yêu cầu kỹ thuật thiết yếu cho triển khai edge ML đáng tin cậy^[27].
6. **Tăng Cường Dữ Liệu Nhận Biết Lớp:** Framework tích hợp hệ thống tăng cường dữ liệu với năm phương pháp biến đổi đặc thù IMU^[38], cùng với chiến lược nhận biết lớp mới lạ tự động phân biệt hoạt động tĩnh và động. Hoạt động tĩnh chỉ nhận micro-jitter ($\sigma = 0,01$) thay vì biến đổi đầy đủ, bảo toàn đặc tính phân biệt “gần bất động” thiết yếu cho phân loại chính xác. Thiết kế này giải quyết vấn đề class confusion được quan sát khi áp dụng tăng cường đồng nhất.
7. **Ngưỡng Tin Cậy cho Từ Chối Hoạt Động Không Xác Định:** Mã C++ đã tạo tích hợp cơ chế từ chối dự đoán dựa trên ngưỡng tin cậy, trích xuất trực tiếp từ đầu ra mô hình hiện có (softmax/tỷ lệ bỏ phiếu) mà không yêu cầu mô hình phát hiện bất thường riêng biệt^[20]. Phương pháp này không tốn thêm bộ nhớ, không cần dữ liệu huấn luyện bổ sung, và là bước phòng thủ thiết yếu cho triển khai thực tế nơi đầu vào ngoài phân phối là phổ biến.

6.1.2 Xác Thực Thực Nghiệm

Các thử nghiệm toàn diện sử dụng tập dữ liệu Nhận Dạng Hoạt Động Con Người nằm lớp xác thực hiệu quả của framework:

- **Độ Chính Xác Mô Hình:** 93.8-96.2% trên ba thuật toán, với Neural Networks đạt 96.2%—trong vòng 0.3% của triển khai TensorFlow Lite thủ công chuyên gia trong khi yêu cầu chuyên môn kỹ thuật ít hơn đáng kể
- **Hiệu Suất Theo Từng Lớp:** F1-scores cân bằng (0.93-0.98) trên tất cả các lớp hoạt động, bao gồm các lớp thiếu số đầy thách thức, chứng minh khả năng áp dụng mạnh mẽ
- **Tính Khả Thi Thời Gian Thực:** Suy luận 9-18ms trên nền tảng tham chiếu ARM Cortex-M4 @ 64MHz với tiêu thụ điện năng 30mW, cho phép hoạt động pin liên tục 65+ giờ
- **Định Vị Cạnh Tranh:** Vượt trội các nền tảng thương mại (Edge Impulse, SensiML) 1.4-3.1% độ chính xác trong khi vẫn hoàn toàn miễn phí và mã nguồn mở
- **Đa Thiết bị theo Kiến trúc:** Tầng generator hỗ trợ các đích Arduino-compatible, ARM Cortex-M, generic C/C++, ESP-IDF, MicroPython và Zephyr; Seeed XIAO được dùng như ca benchmark đại diện chứ không phải thiết bị đích duy nhất
- **Tương Đồng Huấn Luyện-Triển Khai:** Xác minh 12 điểm kiểm tra đảm bảo đặc trưng miền thời gian giống hệt giữa Python và C++, loại bỏ nguồn gốc chính của suy giảm hiệu suất triển khai biên
- **Tăng Cường Nhận Biết Lớp:** Chiến lược tăng cường phân biệt tĩnh/động ngăn chặn class confusion được quan sát với tăng cường đồng nhất, bảo toàn đặc tính phân biệt của hoạt động tĩnh
- **Từ Chối Hoạt Động Không Xác Định:** Ngưỡng tin cậy tích hợp cho phép thiết bị biên từ chối dự đoán khi không đủ tin cậy, tăng độ tin cậy cho triển khai thực tế

6.1.3 Tác Động Rộng Hơn

Ngoài các đóng góp kỹ thuật, công việc này thúc đẩy dân chủ hóa edge AI bằng cách:

- **Loại Bỏ Rào Cản Kinh Tế:** Thay thế các đăng ký thương mại \$20-500/tháng bằng một lựa chọn thay thế mã nguồn mở miễn phí có thể tiếp cận với các nhà nghiên cứu trên toàn thế giới
- **Hạ Thấp Rào Cản Kỹ Thuật:** Cho phép người dùng không có chuyên môn hệ thống nhúng triển khai các hệ thống theo dõi chuyển động cấp chuyên nghiệp thông qua các quy trình làm việc GUI trực quan
- **Thúc Đẩy Tính Minh Bạch:** Cung cấp khả năng hiển thị đầy đủ vào tiền xử lý, trích xuất đặc trưng và tạo mã—quan trọng cho khả năng tái tạo khoa học và sử dụng giáo dục
- **Kích Thích Đổi Mới:** Kiến trúc có thể mở rộng cho phép các nhà nghiên cứu tích hợp các thuật toán, nền tảng và chiến lược tối ưu hóa mới lạ mà không bắt đầu từ đầu

6.2 Xem Lại Các Mục Tiêu Nghiên Cứu

Các mục tiêu nghiên cứu ban đầu được nêu trong Mục 1.3 đã được giải quyết toàn diện:

1. **Hệ Thống Tiền Xử Lý Tương Tác ✓:** Giao diện cửa sổ thời gian kéo thả được triển khai thành công và tích hợp với kiểm tra tín hiệu trực quan
2. **Hỗ Trợ Đa Thuật Toán ✓:** Ba thuật toán (RF, SVM, NN) được tích hợp với tối ưu hóa siêu tham số tự động mang lại cải thiện độ chính xác 1.9-2.7%
3. **Tạo Mã Không Phụ Thuộc Nền Tảng ✓:** Kiến trúc trình tạo modular được tổ chức theo ma trận model-platform-backend, hỗ trợ nhiều họ đích triển khai; benchmark định lượng hiện được báo cáo trên nền tảng tham chiếu ARM Cortex-M4 với bốn chế độ tối ưu hóa
4. **Xác Thực Hiệu Quả Framework ✓:** Đánh giá toàn diện chứng minh độ chính xác cạnh tranh (96.2%), hiệu suất thời gian thực (suy luận 12ms trên nền tảng tham chiếu) và khả năng tái đóng gói cùng một pipeline cho nhiều đích triển khai khác nhau
5. **Khả Năng Tiếp Cận ✓:** GUI dựa trên web với đường cong học tập thấp làm cho phát triển edge AI tiên tiến có thể tiếp cận với phi chuyên gia trong khi duy trì khả năng cấp chuyên nghiệp

6. **Độ Tin Cây Triển Khai ✓**: Quy trình xác minh tương đồng huấn luyện-triển khai 12 điểm, ngưỡng tin cây và tăng cường dữ liệu nhận biết lớp đảm bảo mô hình hoạt động đáng tin cậy trên phần cứng thực, không chỉ trong môi trường mô phỏng

6.3 Các Ứng Dụng Thực Tế

Framework đã xác thực cho phép các ứng dụng thực tế đa dạng:

Y Tế và Phục Hồi Chức Năng Các hệ thống đeo để giám sát mức độ hoạt động của bệnh nhân, theo dõi tiến trình phục hồi chức năng (phục hồi di động sau phẫu thuật, trị liệu đột quỵ), phát hiện té ngã trong quần thể người cao tuổi và cung cấp dữ liệu hoạt động khách quan cho các thử nghiệm lâm sàng. Tuổi thọ pin 65 giờ và hoạt động thời gian thực hỗ trợ giám sát liên tục nhiều ngày mà không gây gánh nặng cho bệnh nhân.

Thể Thao và Thể Dục Phát hiện và phân loại tập luyện tự động (chạy, đạp xe, cử tạ), phân tích hình thức để phòng ngừa chấn thương (phát hiện bất đối xứng đáng đi, kỹ thuật nâng không đúng) và phản hồi đào tạo cá nhân hóa. Độ trễ dưới 20ms cho phép các ứng dụng huấn luyện thời gian thực nơi phản hồi ngay lập tức là quan trọng.

Nghiên Cứu và Giáo Dục Nền tảng chi phí thấp để giảng dạy các khái niệm edge AI trong các khóa học đại học/sau đại học, cho phép các phòng thí nghiệm thực hành mà không cần giấy phép thương mại đắt đỏ. Các nhà nghiên cứu có thể nhanh chóng tạo mẫu các thuật toán nhận dạng hoạt động mới lạ và triển khai lên phần cứng để xác thực hiện trường. Bản chất mã nguồn mở tạo điều kiện cho nghiên cứu có thể tái tạo và đóng góp cộng đồng.

Nhà Thông Minh và Hỗ Trợ Sống Môi Trường Giám sát hoạt động cho người cao tuổi sống độc lập (phát hiện các mẫu không hoạt động bất thường, theo dõi thói quen hàng ngày), tự động hóa nhà thông minh nhận biết bối cảnh (điều chỉnh ánh sáng/nhiệt độ dựa trên hoạt động được phát hiện) và phân tích xu hướng sức khỏe dài hạn. Tiêu thụ điện năng thấp (30mW) hỗ trợ cảm biến luôn bật mà không cần thay pin thường xuyên.

6.4 Các Hạn Chế và Công Việc Tương Lai

Trong khi framework chứng minh khả năng mạnh mẽ, các hạn chế được thừa nhận vạch ra các hướng cho nghiên cứu tương lai:

6.4.1 Sự Đa Dạng Tập Dữ Liệu

Xác thực hiện tại sử dụng dữ liệu đối tượng đơn cho năm hoạt động. Công việc tương lai nên:

- Tiến hành các nghiên cứu đa đối tượng trải dài các nhân khẩu học đa dạng (tuổi, giới tính, mức độ di động)
- Mở rộng phân loại hoạt động để bao gồm các sự kiện té ngã, các hoạt động phức tạp (nấu ăn, lái xe) và chuyển đổi hoạt động
- Xác thực trên các tập dữ liệu benchmark công khai (UCI HAR, WISDM, PAMAP2) cho các so sánh chuẩn hóa

6.4.2 Các Mở Rộng Thuật Toán

- **Học Sâu:** Tích hợp TensorFlow Lite Micro cho triển khai CNN/LSTM/Transformer, hỗ trợ học từ đầu đến cuối từ dữ liệu cảm biến thô
- **Học Trực Tuyến:** Cho phép các mô hình đã triển khai thích ứng với các mẫu cụ thể của người dùng theo thời gian mà không cần kết nối đám mây
- **Các Phương Pháp Ensemble:** Kết hợp nhiều dự đoán thuật toán (bỏ phiếu RF + NN + SVM) cho độ bền vững được cải thiện
- **Học Không Giám Sát:** Khám phá hoạt động tự động mà không cần gán nhãn thủ công cho các ứng dụng khám phá

6.4.3 Mở Rộng Nền Tảng

- Mở rộng ma trận benchmark liên thiết bị trên AVR (Arduino Uno), RISC-V (ESP32-C3), ARM Cortex-M0+ (Raspberry Pi Pico) và các board hỗ trợ MicroPython/Zephyr để chuyển hỗ trợ ở mức generator thành số liệu định lượng xuyên nền tảng
- Hỗ trợ các mục tiêu FPGA và ASIC cho các thiết bị đeo công suất siêu thấp (<1mW)

- Triển khai di động (Android, iOS) cho theo dõi chuyển động dựa trên điện thoại thông minh

6.4.4 Các Tính Năng Nâng Cao

- **Hợp Nhất Cảm Biến:** Tích hợp đa phương thức (IMU + GPS + nhịp tim + âm thanh) với đồng bộ hóa đặc trưng tự động
- **AI Có Thể Giải Thích:** Trực quan hóa các mẫu cảm biến nào kích hoạt dự đoán cụ thể (tích hợp LIME, SHAP)
- **Học Liên Bang:** Huấn luyện mô hình cộng tác bảo vệ quyền riêng tư trên nhiều thiết bị
- **Backend Đám Mây:** Sao lưu dữ liệu tùy chọn, phiên bản mô hình và chia sẻ mô hình cộng đồng trong khi duy trì kiến trúc local-first
- **Hiệu Chỉnh Tin Cậy:** Áp dụng temperature scaling^[18] hoặc Platt scaling để cải thiện chất lượng hiệu chỉnh tin cậy softmax, và điều chỉnh ngưỡng τ tự động dựa trên đường cong ROC tập xác thực
- **Tăng Cường Thích Ứng:** Mở rộng chiến lược nhận biết lớp để tự động xác định tham số tăng cường tối ưu cho từng lớp dựa trên đặc tính tín hiệu (phương sai, entropy), thay vì chỉ phân biệt nhị phân tĩnh/động

6.5 Suy Nghĩ Cuối Cùng

L luận văn này đã trình bày một framework toàn diện giải quyết các thách thức cơ bản trong theo dõi chuyển động dựa trên biên: khả năng tiếp cận, hiệu suất và tính linh hoạt. Thông qua tiền xử lý tương tác mới lạ, xử lý mất cân bằng lớp hệ thống và tạo mã nhận biết tối ưu hóa, đã tạo ra một công cụ khớp với dễ sử dụng của các nền tảng thương mại trong khi cung cấp kiểm soát đầy đủ và chi phí bằng không. Giá trị cốt lõi của framework không nằm ở việc tối ưu cho riêng Seeed XIAO, mà ở khả năng chuyển cùng một pipeline sang nhiều thiết bị biên khác nhau mà vẫn giữ được tính minh bạch và khả năng kiểm chứng. Xác thực thực nghiệm xác nhận rằng dân chủ hóa và xuất sắc hiệu suất không loại trừ lẫn nhau—chúng có thể và nên cùng tồn tại.

Hành trình từ dữ liệu cảm biến thô đến mô hình edge AI đã triển khai liên quan đến nhiều bước, mỗi bước truyền thống yêu cầu chuyên môn chuyên biệt. Framework này thu gọn các rào cản đó, cho phép bất kỳ ai có kiến thức miền (các hoạt động cần

phát hiện) tạo ra các hệ thống sẵn sàng triển khai mà không trở thành chuyên gia về xử lý tín hiệu, lý thuyết học máy hoặc lập trình hệ thống nhúng. Đây là bản chất của dân chủ hóa: hạ thấp rào cản mà không hạ thấp tiêu chuẩn.

Khi chúng ta nhìn về tương lai, sự tiến hóa tiếp tục của edge AI sẽ phụ thuộc vào các công cụ trao quyền thay vì hạn chế, giáo dục thay vì che giấu và bao gồm thay vì loại trừ. Framework này đại diện cho một bước trong hướng đó. Con đường phía trước dài, nhưng điểm đến—một thế giới nơi các khả năng AI tiên tiến có thể tiếp cận với tất cả mọi người—đáng để theo đuổi.

Mã nguồn, tài liệu và các tập dữ liệu ví dụ có sẵn công khai để hỗ trợ khả năng tái tạo, giáo dục và đóng góp cộng đồng. Các nhà nghiên cứu và nhà thực hành được khuyến khích xây dựng trên nền tảng này, mở rộng khả năng của framework và áp dụng nó cho các thách thức theo dõi chuyển động mới lạ. Cùng nhau, chúng ta có thể bắc cầu khoảng cách giữa nghiên cứu phòng thí nghiệm và triển khai thực tế, chuyển đổi bối cảnh phát triển edge AI.

Tài liệu tham khảo

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. Tensorflow: Large-scale machine learning on heterogeneous distributed systems, 2015.
- [2] Allied Market Research. Motion tracking system market size, share, competitive landscape and trend analysis report. <https://www.alliedmarketresearch.com/motion-tracking-system-market>, 2023. Accessed: December 2, 2025.
- [3] Saleema Amershi, Maya Cakmak, W. Bradley Knox, and Todd Kulesza. Power to the people: The role of humans in interactive machine learning. *AI Magazine*, 35(4):105–120, 2014.
- [4] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L. Reyes-Ortiz. A public domain dataset for human activity recognition using smartphones. *21th European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN)*, 2013.
- [5] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, David Kanter, Sebastian Ahmed, Danilo Pau, et al. Mlperf tiny benchmark. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [6] Oresti Banos, Juan-Manuel Galvez, Miguel Damas, Hector Pomares, and Ignacio Rojas. Window size impact in human activity recognition. *Sensors*, 14(4):6474–6499, 2014.
- [7] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [8] Mateusz Buda, Atsuto Maki, and Maciej A. Mazurowski. A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks*, 106:249–259, 2018.

- [9] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.
- [10] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.
- [11] Lipika Dutta and Swapna Bharali. Tinyml meets iot: A comprehensive survey. *Internet of Things*, 16:100461, 2021.
- [12] Edge Impulse. Edge impulse studio. <https://studio.edgeimpulse.com/>, 2024. Accessed: December 12, 2024.
- [13] Alessandro Filippeschi, Norbert Schmitz, Markus Miezal, Gabriele Bleser, Emanuele Ruffaldi, and Didier Stricker. Survey of motion tracking methods based on inertial sensors: A focus on upper limb human motion. *Sensors*, 17(6):1257, 2017.
- [14] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [15] Google. Litert overview. <https://ai.google.dev/edge/litert>, 2024. Accessed: December 12, 2024.
- [16] Google. Mediapipe solutions guide. <https://ai.google.dev/edge/mediapipe/solutions/guide>, 2024. Accessed: December 12, 2024.
- [17] Google. Teachable machine. <https://teachablemachine.withgoogle.com/>, 2024. Accessed: December 12, 2024.
- [18] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.
- [19] Haibo He and Edwardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009.
- [20] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.

- [21] Brian Kenji Iwana and Seiichi Uchida. An empirical survey of data augmentation for time series classification with neural networks. *PLOS ONE*, 16(7):e0254841, 2021.
- [22] Rudolf E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.
- [23] Arthur Le Guennec, Simon Malinowski, and Romain Tavenard. Data augmentation for time series classification using convolutional neural networks. In *ECML/PKDD Workshop on Advanced Analytics and Learning on Temporal Data*, 2016.
- [24] Ji Lin, Wei-Ming Chen, Yujun Lin, Chuang Gan, and Song Han. Mcunet: Tiny deep learning on iot devices. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:11711–11722, 2020.
- [25] Massimo Merenda, Carlo Porcaro, and Demetrio Iero. Edge machine learning for ai-enabled iot devices: A review. *Sensors*, 20(9):2533, 2020.
- [26] Alan V. Oppenheim and Ronald W. Schaffer. *Discrete-Time Signal Processing*. Prentice Hall, 2nd edition, 1999.
- [27] Andrei Paleyes, Raoul-Gabriel Urma, and Neil D. Lawrence. Challenges in deploying machine learning: A survey of case studies. *ACM Computing Surveys*, 55(6):1–29, 2022.
- [28] Anuj Patil, Darshan Rao, Kaustubh Utturwar, Tejas Shelked, and Ekta Sarda. Body posture detection and motion tracking using ai for medical exercises and recommendation system. *ITM Web Conference, Volume 44, International Conference on Automation, Computing and Communication 2022 (ICACC-2022)*, May 2022.
- [29] Nafees Rashid, Berken Utku Demirel, and Mohammad Abdullah Al Faruque. AHAR: Adaptive CNN for energy-efficient human activity recognition in low-power edge devices. *IEEE Internet of Things Journal*, 9(15):13041–13051, 2022.
- [30] Partha Pratim Ray. A review on tinyml: State-of-the-art and prospects. *Journal of King Saud University - Computer and Information Sciences*, 34(4):1595–1623, 2022.

- [31] Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015.
- [32] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems (NeurIPS)*, 28, 2015.
- [33] Seeed Studio. Xiao nrf52840 series. https://wiki.seeedstudio.com/XIAO_BLE/, 2024. Accessed: December 12, 2024.
- [34] SensiML Corporation. Sensiml analytics toolkit. <https://sensiml.com/>, 2024. Accessed: December 3, 2025.
- [35] Ben Shneiderman, Catherine Plaisant, Maxine Cohen, Steven Jacobs, Niklas Elmqvist, and Nicholas Diakopoulos. *Designing the User Interface: Strategies for Effective Human-Computer Interaction*. Pearson, 6th edition, 2016.
- [36] Jaspreet Singh, Yaochu Xie, Chen Chen, and Wenchao Jiang. Attention-based human activity recognition using wearable sensors. *IEEE Sensors Journal*, 23(11):12213–12223, 2023.
- [37] Odongo Steven Eyobu and Dong Seog Han. Feature representation and data augmentation for human activity classification based on wearable IMU sensor data using a deep LSTM neural network. *Sensors*, 18(9):2892, 2018.
- [38] Terry T. Um, Franz M. J. Pfister, Daniel Pichler, Satoshi Endo, Muriel Lang, Sandra Hirche, Urban Fiber, and Dana Klückmann. Data augmentation of wearable sensor data for Parkinson’s disease monitoring using convolutional neural networks. In *Proceedings of the 19th ACM International Conference on Multimodal Interaction (ICMI)*, pages 216–220. ACM, 2017.
- [39] Shaohua Wan, Lianyong Qi, Xiaolong Xu, Chenguang Tong, and Zongming Gu. Deep learning models for real-time human activity recognition with smartphones. *Mobile Networks and Applications*, 25:743–755, 2020.
- [40] An Wang, Guangyu Chen, Chuang Shang, Mingli Zhang, and Likun Liu. Human activity recognition in a smart home environment with stacked denoising autoencoders. *IEEE Transactions on Industrial Informatics*, 19(2):2071–2080, 2023.

- [41] An Wang, Guangyu Chen, Jian Yang, Shihua Zhao, and Ching-Yao Chang. A comparative study on human activity recognition using inertial sensors in a smart-phone. *IEEE Sensors Journal*, 24(3):3234–3247, 2024.
- [42] Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O’Reilly Media, 2019.
- [43] Michael W. Whittle. *Gait Analysis: An Introduction*. Butterworth-Heinemann, 5th edition, 2014.
- [44] Ke Xia, Jianguang Huang, and Hanyu Wang. Lstm-cnn architecture for human activity recognition. *IEEE Access*, 8:56855–56866, 2020.
- [45] Matteo Zago, Matteo Luzzago, Tommaso Marangoni, Mariolino De Cecco, Marco Tarabini, and Manuela Galli. 3d tracking of human motion using visual skeletonization and stereoscopic vision. *Machine Learning Approaches to Human Movement Analysis*, March 2020.
- [46] Huiyu Zhou and Huosheng Hu. Human motion tracking for rehabilitation—a survey. *Biomedical Signal Processing and Control*, 3:1–18, 01 2008.